FULL NAME:	[Enter full name]
DATE:	[Enter submission date]
REGISTRATION NUMBER:	[Enter registration number]
COURSE:	COS 202 (Programming II)

1.0 SMART CLINIC APPLICATION PROJECT REPORT

1.1 PROJECT DESCRIPTION AND FEATURES

The Smart Clinic Appointment and Patient Management System (SCAPMS 3.0) is a C++17 desktop application for the COS 202 lab assessment. Its Qt Widgets interface provides patient registration, appointment management, a FIFO queue, searching, sorting, reporting and persistent storage. Eight panels separate overview, patient records, appointments, queue, algorithms, reports, staff activity and resource calendars.

Scheduled and walk-in visits have persistent consultation records. Check-in joins the queue; Call next starts consultation; completion saves notes. Rescheduling checks conflicts, cancellation and no-show require reasons, and corrections retain previous notes. Clinician and room allocations reject overlapping active bookings. History and archiving preserve previous visits. A welcome screen opens recent workspaces or a fictional demonstration copy.

1.2 CLASS HIERARCHY AND RESPONSIBILITIES

PatientRecord is abstract, with pure virtual category(), consultationSummary() and clone() operations. GeneralPatient, EmergencyPatient and FollowUpPatient provide category-specific behaviour. Appointable defines scheduling and cancellation policies. Processing a PatientRecord reference dispatches to the concrete subtype at runtime, while clone() preserves that subtype in undo snapshots.

ClinicDatabase owns patient and appointment collections. ClinicManager applies validation, permissions and transactions. Separate panels present data; FileHandler and RecoveryService handle persistence. WorkspaceGuard detects competing file access. AccessControl and PortableBackup provide authentication and encryption. ReportService and Benchmark calculate reports and algorithm comparisons. Header/source files and namespaces separate these responsibilities.

1.3 DATA STRUCTURES

A std::vector of unique_ptr<PatientRecord> owns patients without object slicing. Appointments refer to immutable patient IDs and optional clinician/room names. A std::queue of QueueEntry objects preserves arrival order and links waiting patients to visits. A fixed std::array of ten CacheEntry objects tracks frequent access, replacing the least-used entry with an oldest-access tie-break.

A std::stack of UndoSnapshot objects stores up to fifty deep snapshots. Candidate transactions are validated before committing; rejected changes preserve live state and undo history. Undo restores the latest record state while retaining existing audit events and adding an Undo entry. Account changes and switching users clear the undo stack.

1.4 SEARCHING SORTING AND RECURSION

Linear search scans IDs and supports name/category filtering. Binary search uses an ID-sorted view prepared by the handwritten merge sort. Lookup costs O(log n); preparing the order costs O(n log n), reported separately in the interface.

Insertion sort has O(n) best-case and O(n squared) worst-case time. Recursive merge sort is stable with O(n log n) time and O(n) temporary storage. Ranges smaller than two items form its base case; recursive calls sort both halves before merging. Equal keys preserve their original order. Required searching and sorting algorithms are implemented manually.

Both sorts support name, ID and age. The comparison runs sorted, reversed and seeded shuffled input, identical for each algorithm. One warm-up is excluded; five measured runs provide a median. Comparisons, moves and recursion depth supplement timing. A 2,000-record fictional dataset supports demonstrations. Background calculation keeps the interface responsive.

1.5 GUI DESIGN AND EVENTS

VBox, HBox, Grid and Form layouts arrange controls. A QAbstractTableModel adapter presents patients. Signals and slots handle actions, selections and timers. Dynamic subtype fields, date/resource filters, duplicate warnings, validation messages, role-aware tooltips, printing and recovery dialogs support daily workflows. Text scaling, visible empty states and keyboard shortcuts improve usability. Search, sort, selection and scrolling survive updates.

1.6 VALIDATION SECURITY AND PERSISTENCE

Custom exceptions reject invalid contacts, IDs, ages, times, overlapping bookings, duplicate queue entries and malformed data. New appointments are future-dated; check-in requires the appointment date; completion requires an active consultation and notes. No-show requires a scheduled visit whose end time has passed. Follow-up selection uses a completed visit from the patient's history.

Administrator, clinician, receptionist and viewer roles govern actions. Clinical forms and tooltips restrict sensitive fields; report exports/printing require clinical or administrator access, and portable backups require an administrator. Staff passwords use salted PBKDF2-HMAC-SHA256. Windows DPAPI protects normal workspaces and recovery copies. Separate portable backups use a passphrase-derived key with AES-256-GCM and authenticated format metadata.

QSaveFile commits writes atomically. WorkspaceGuard reserves a canonical path with QLockFile and checks a SHA-256 fingerprint before saving. Autosave runs every thirty seconds, with unique files for unsaved sessions. Manual saves retain five protected previous versions. Recovery previews show timestamps and record counts; imported or recovered data opens unsaved for Save As.

1.7 CHALLENGES AND RESOLUTIONS

Polymorphic undo requires deep cloning of unique_ptr records. Views release stale pointers before transactions and restore selection by ID afterward. Walk-ins now create visit records so notes are retained. Resource validation checks patient, clinician and room conflicts together. An empty-state callback initially outlived its table viewport; parent ownership and guarded pointers corrected the teardown failure.

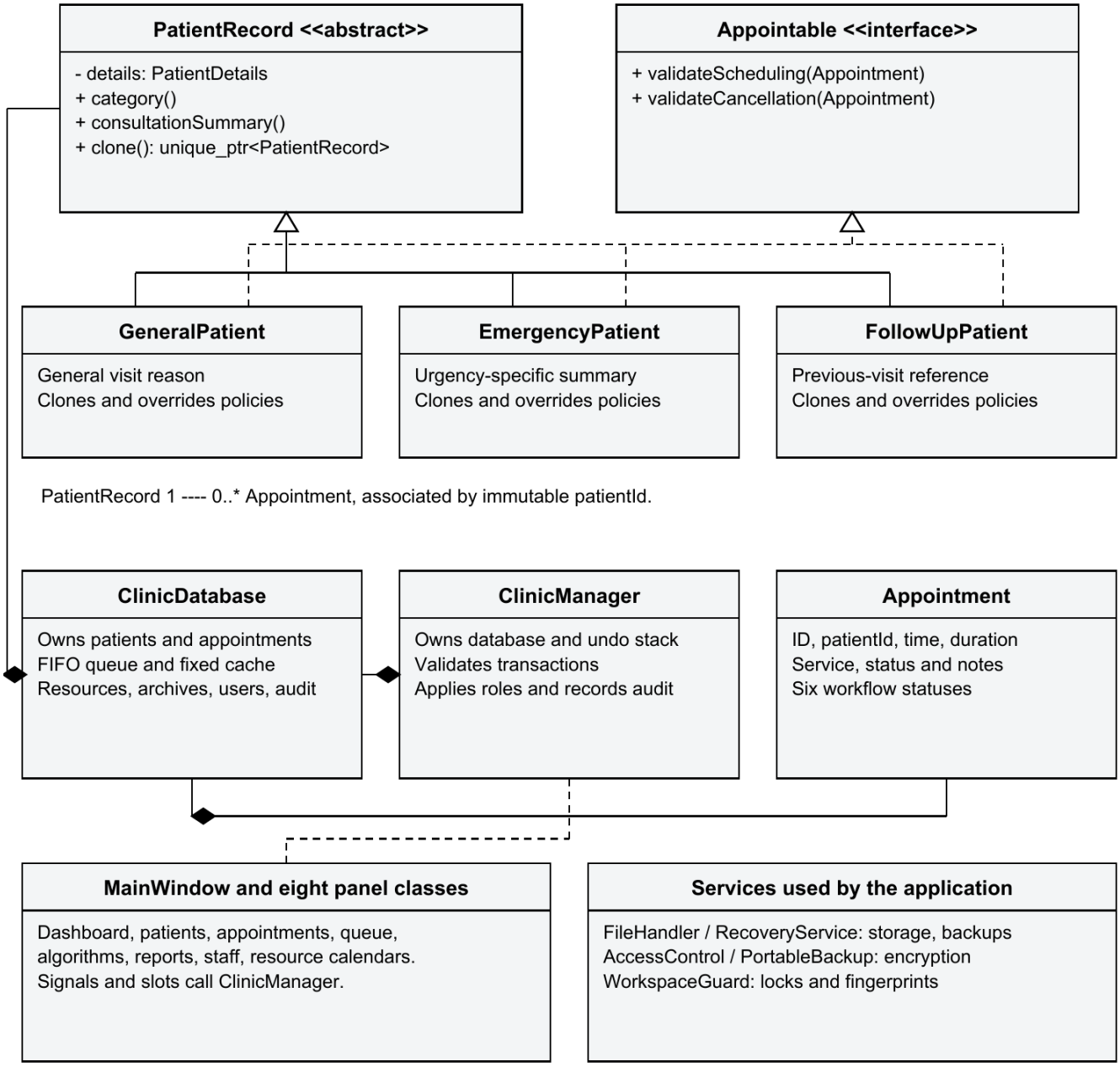
Portable encryption delegates AES to Windows CNG. Authentication, tamper rejection and an independent AES-GCM fixture verify the backup format. Normal DPAPI files remain bound to the Windows account; portable backups require their passphrase and staff administrator credentials. These local controls are an assessment implementation, not an independently evaluated clinical production system.

1.8 VERIFICATION

The final Qt Test run reports 31 results, 0 failures and 0 errors. Tests cover validation, failed-operation atomicity, visit transitions, permission enforcement, file locking, recovery, encryption, resource conflicts, repeated benchmarks and GUI state. Actual application captures and editable source listings follow.

2.0 UML AND SYSTEM DESIGN

The diagram shows inheritance, interface realization, composition and the main service dependencies. Each patient can be associated with zero or more appointments through a stable patient ID.



General, emergency and follow-up classes share an abstract contract. The database owns records and resource catalogs. Controllers coordinate validated state changes; panels use controllers and services for persistence, authorization and backup protection.

3.0 GRAPHICAL USER INTERFACE EVIDENCE

3.1 DASHBOARD

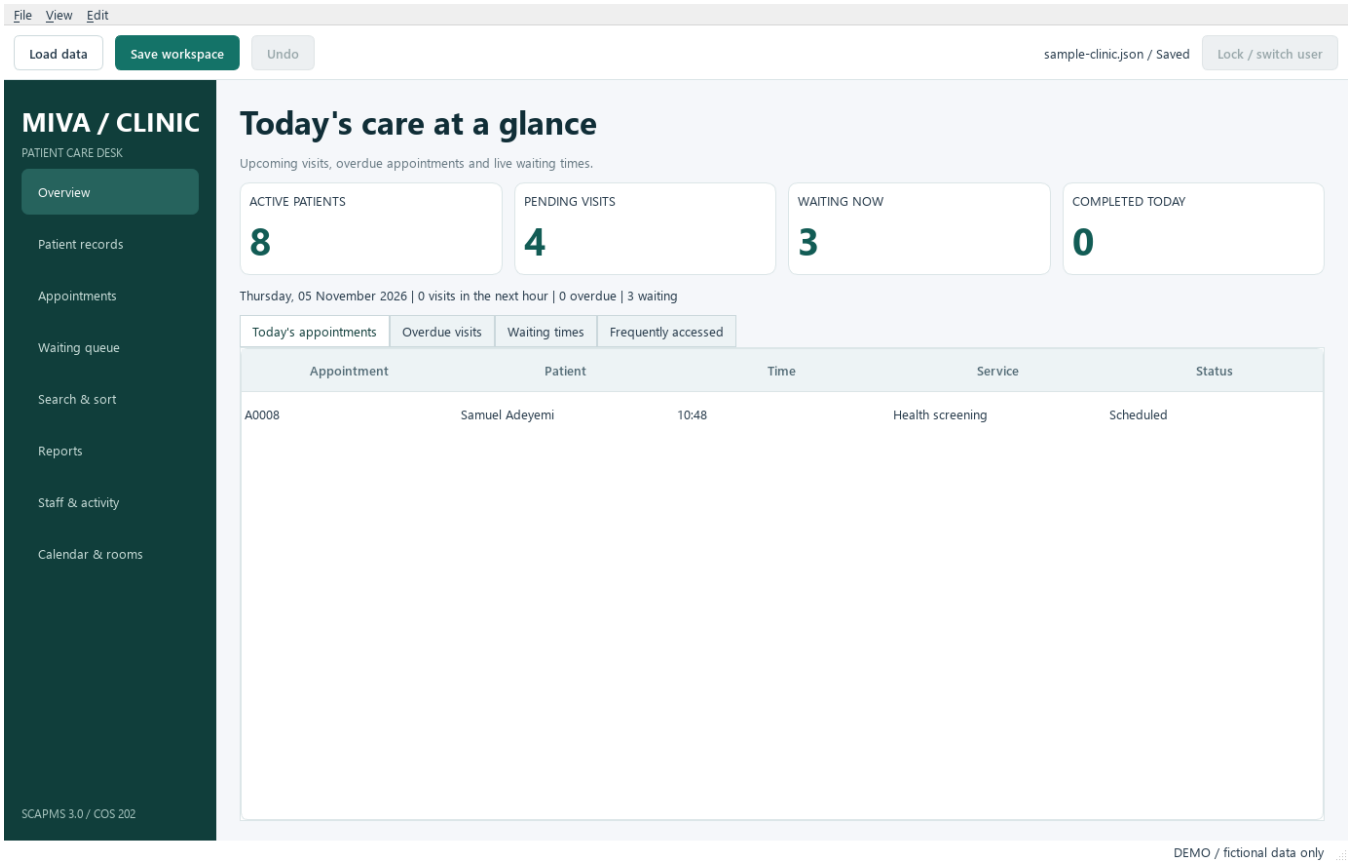


Figure 1. Dashboard in the compiled SCAPMS application.

3.2 PATIENTS

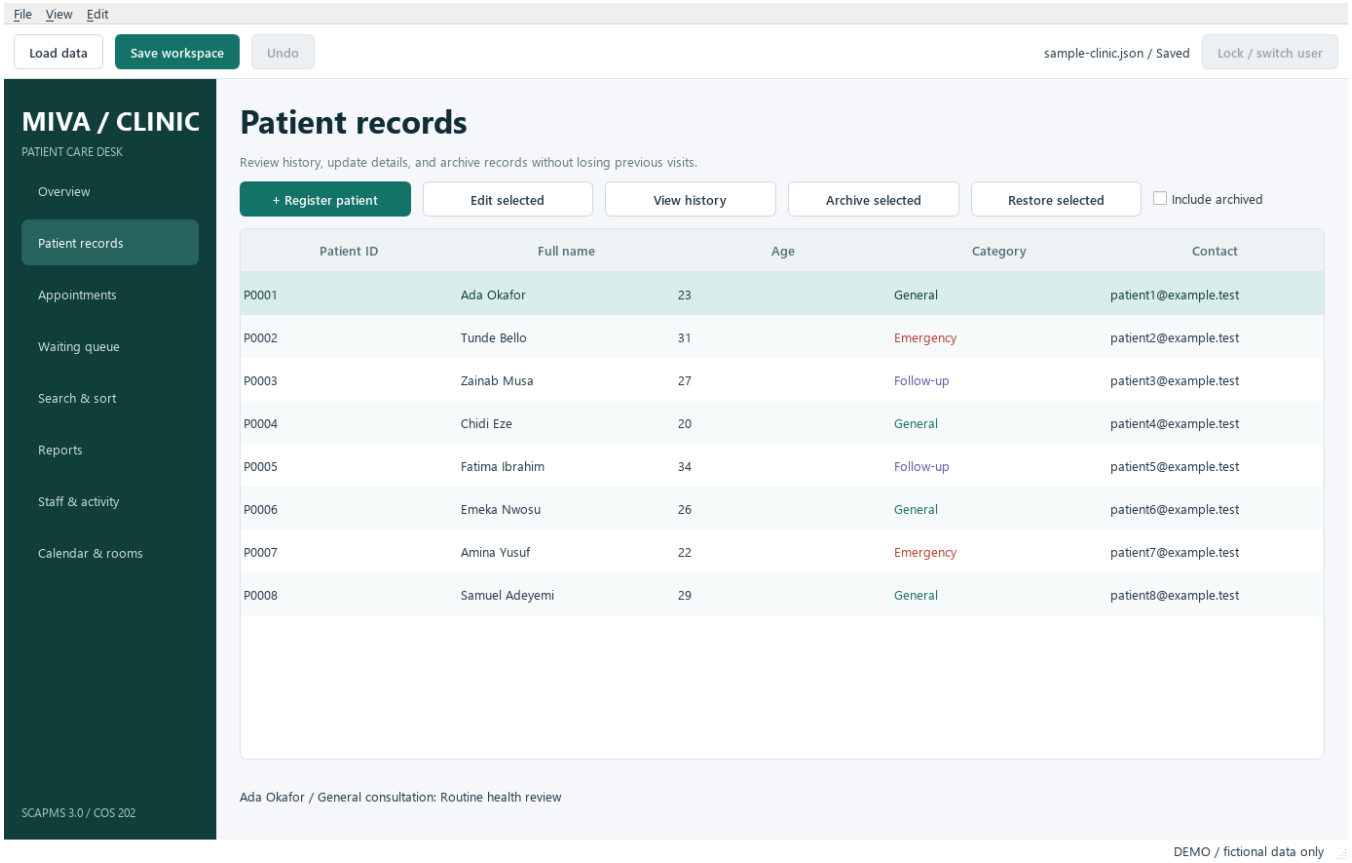


Figure 2. Patients in the compiled SCAPMS application.

3.3 REGISTRATION

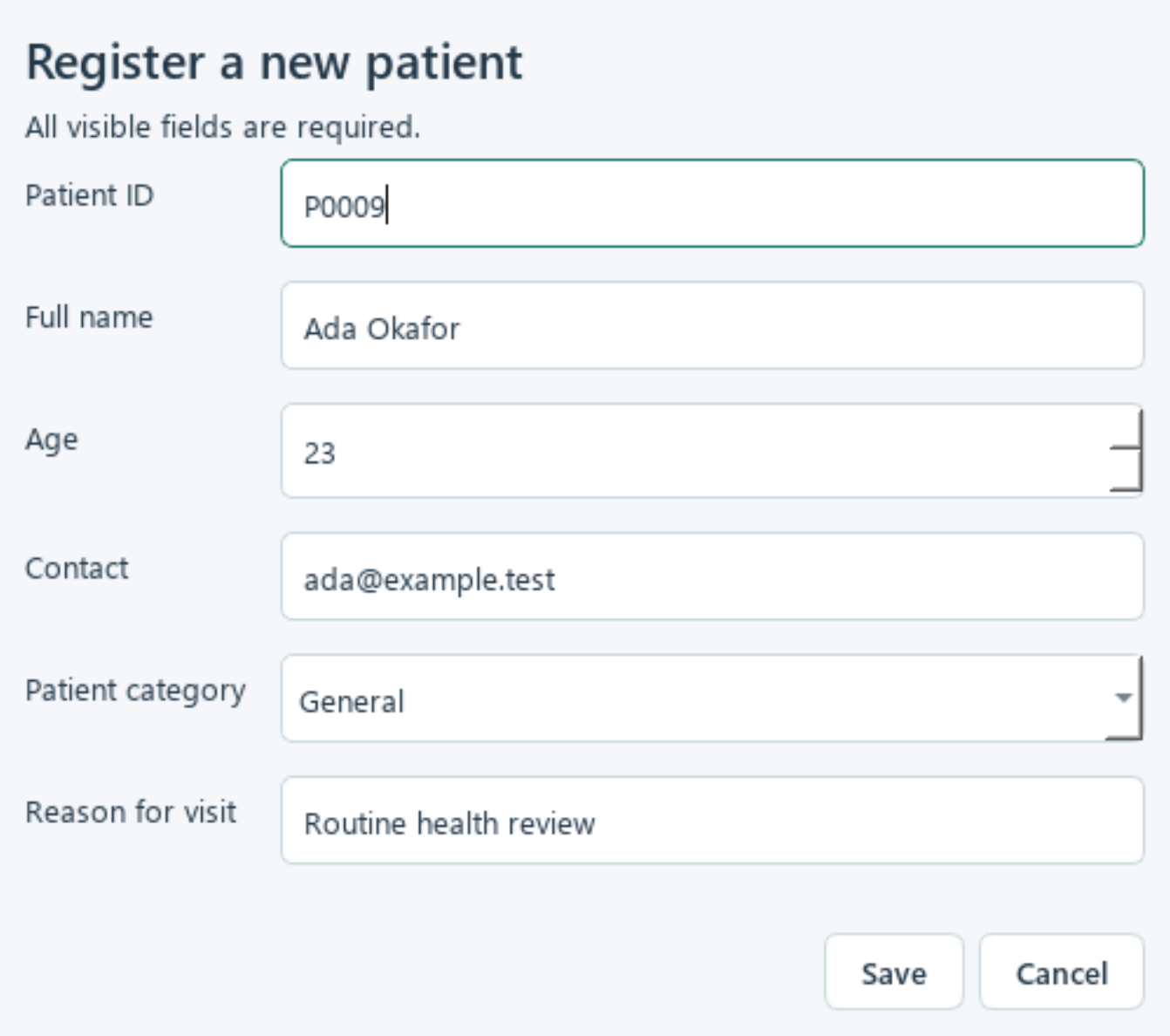


Figure 3. Registration in the compiled SCAPMS application.

3.4 APPOINTMENTS

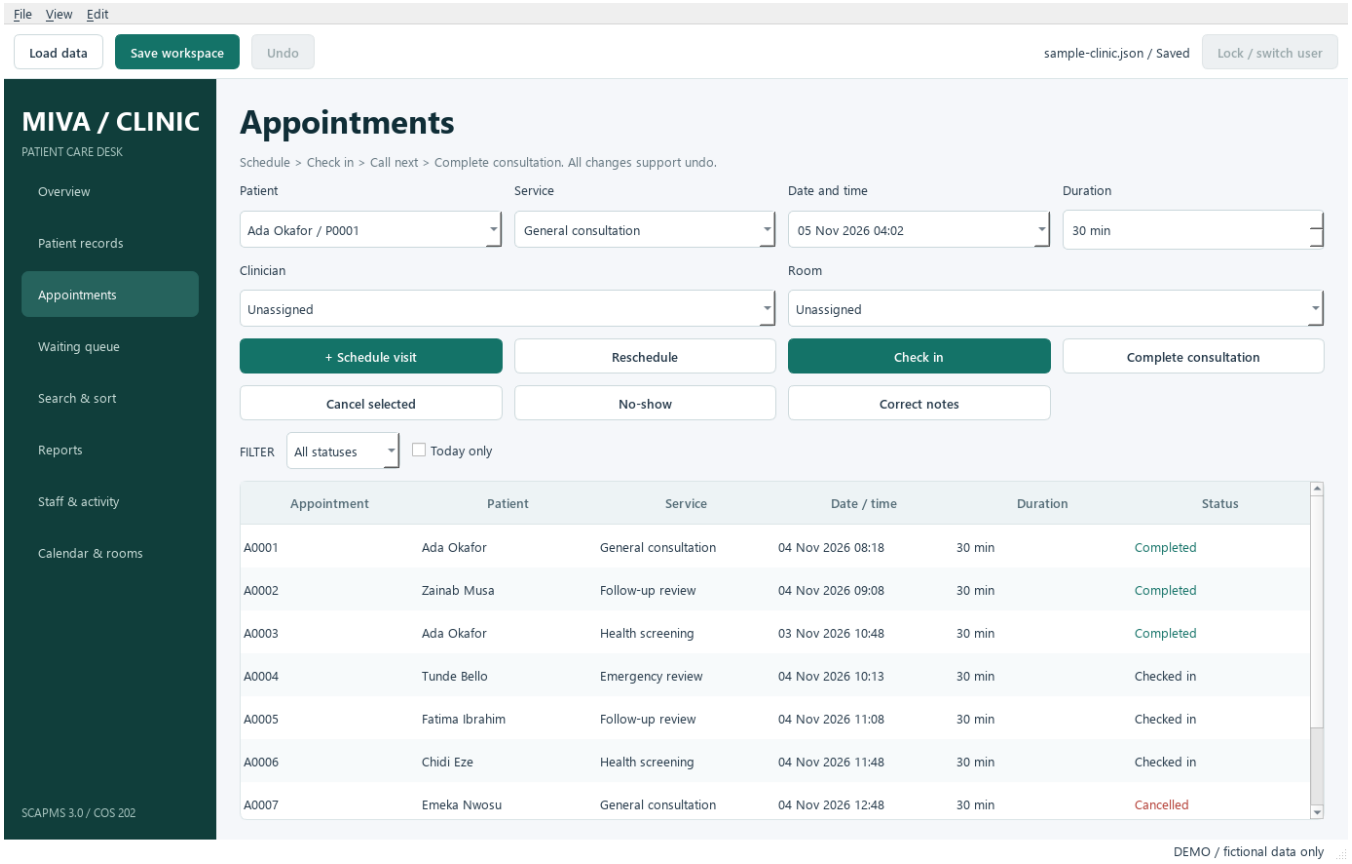


Figure 4. Appointments in the compiled SCAPMS application.

3.5 WAITING QUEUE

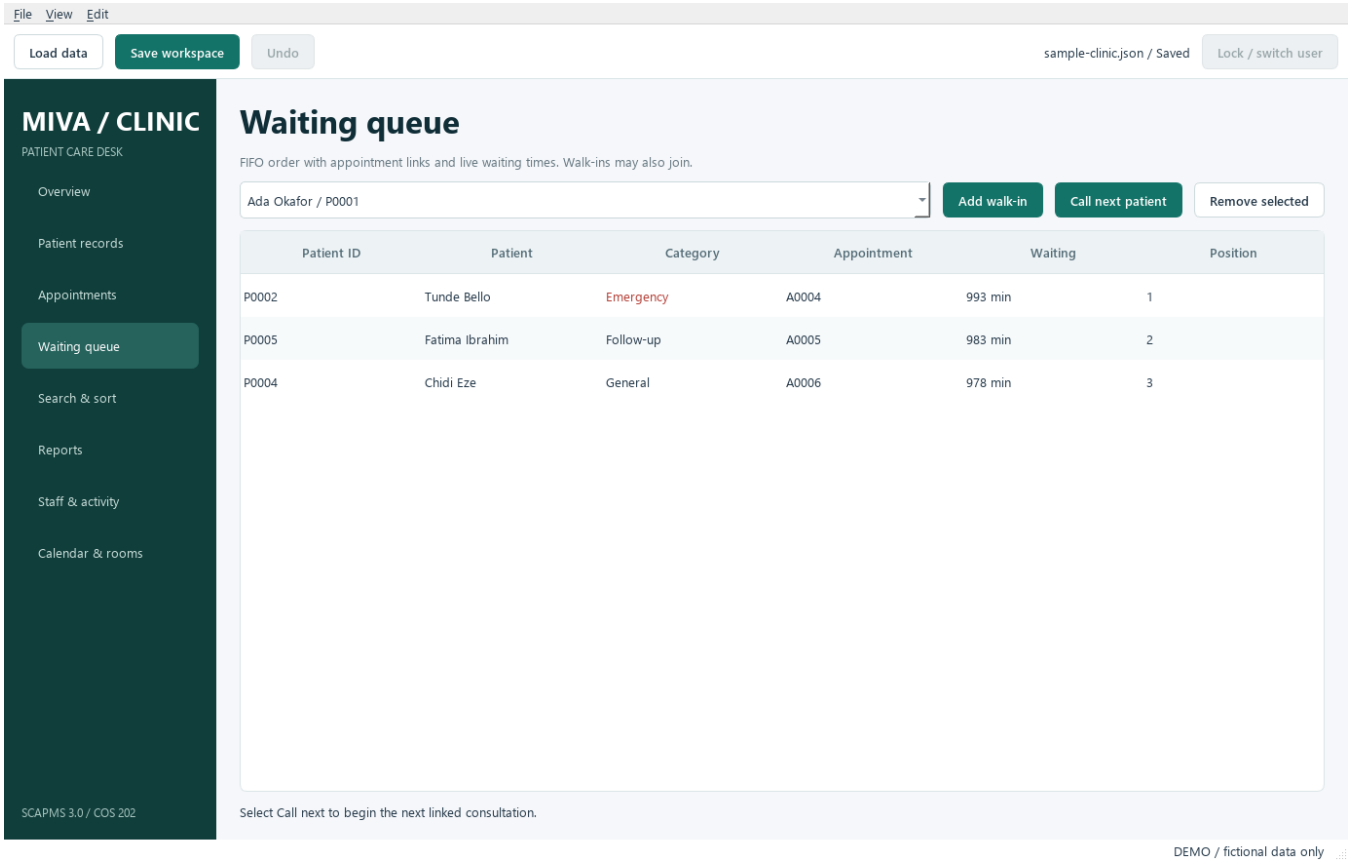


Figure 5. Waiting queue in the compiled SCAPMS application.

3.6 SEARCH SORT

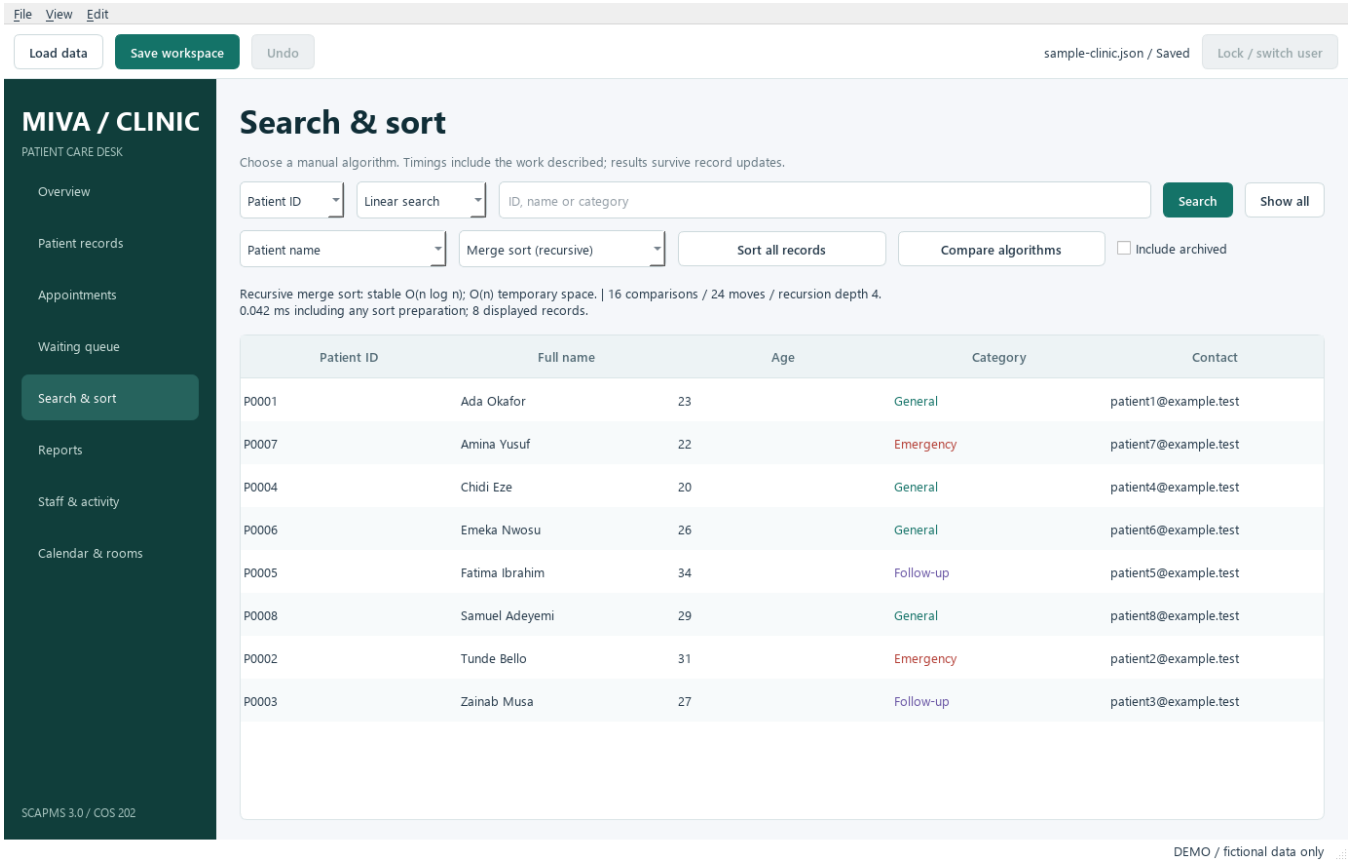


Figure 6. Search sort in the compiled SCAPMS application.

3.7 REPORTS

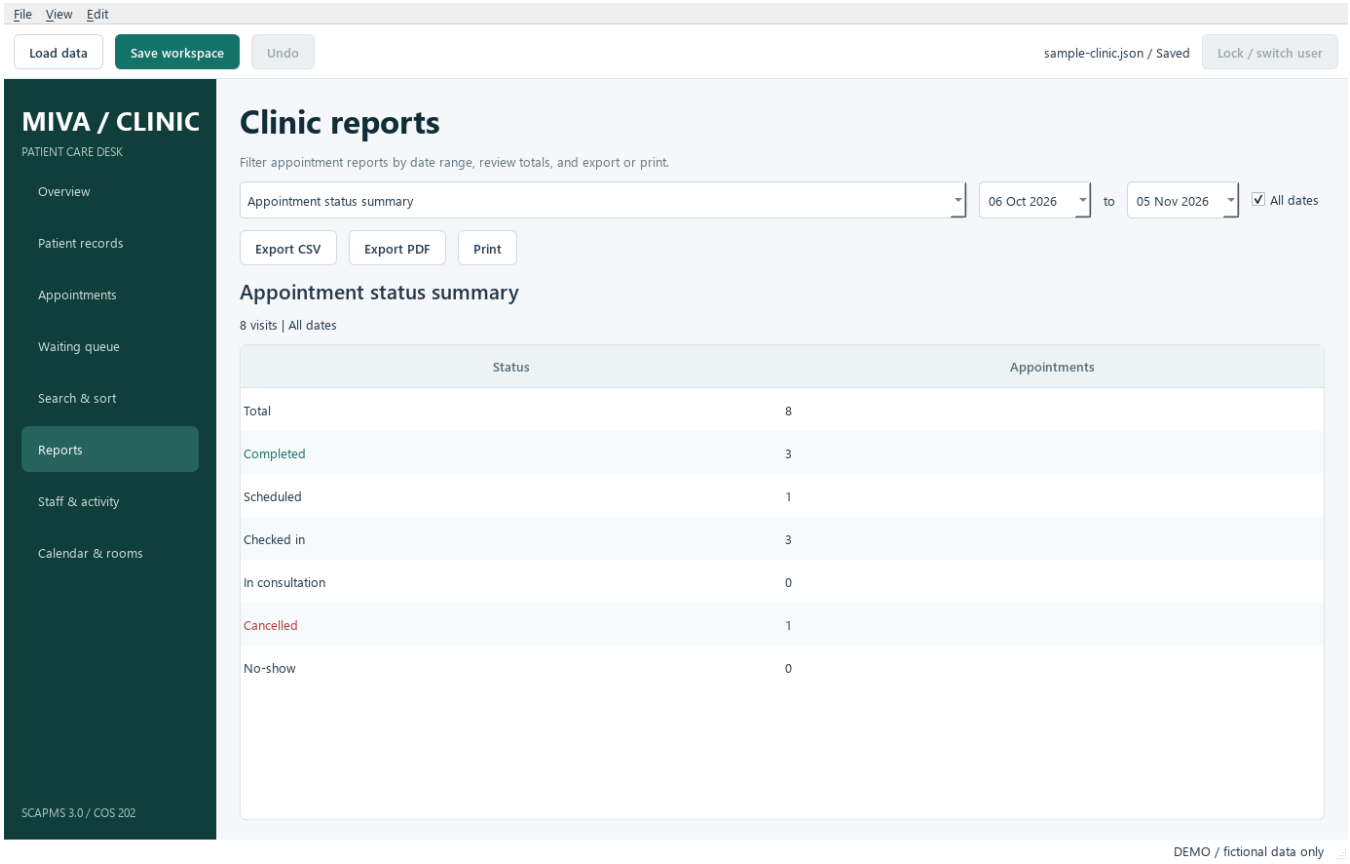


Figure 7. Reports in the compiled SCAPMS application.

3.8 ACCESS ACTIVITY

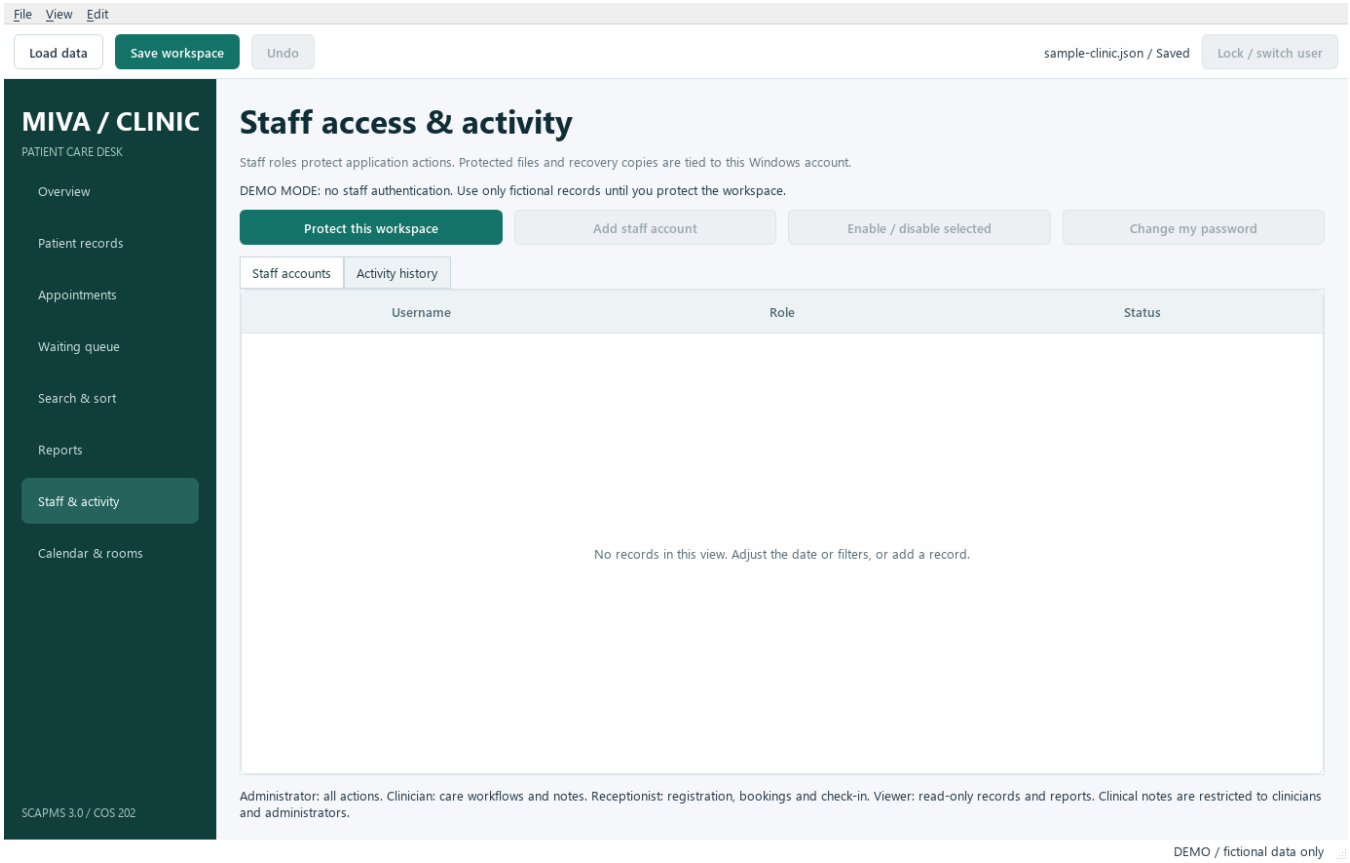


Figure 8. Access activity in the compiled SCAPMS application.

3.9 PATIENT HISTORY

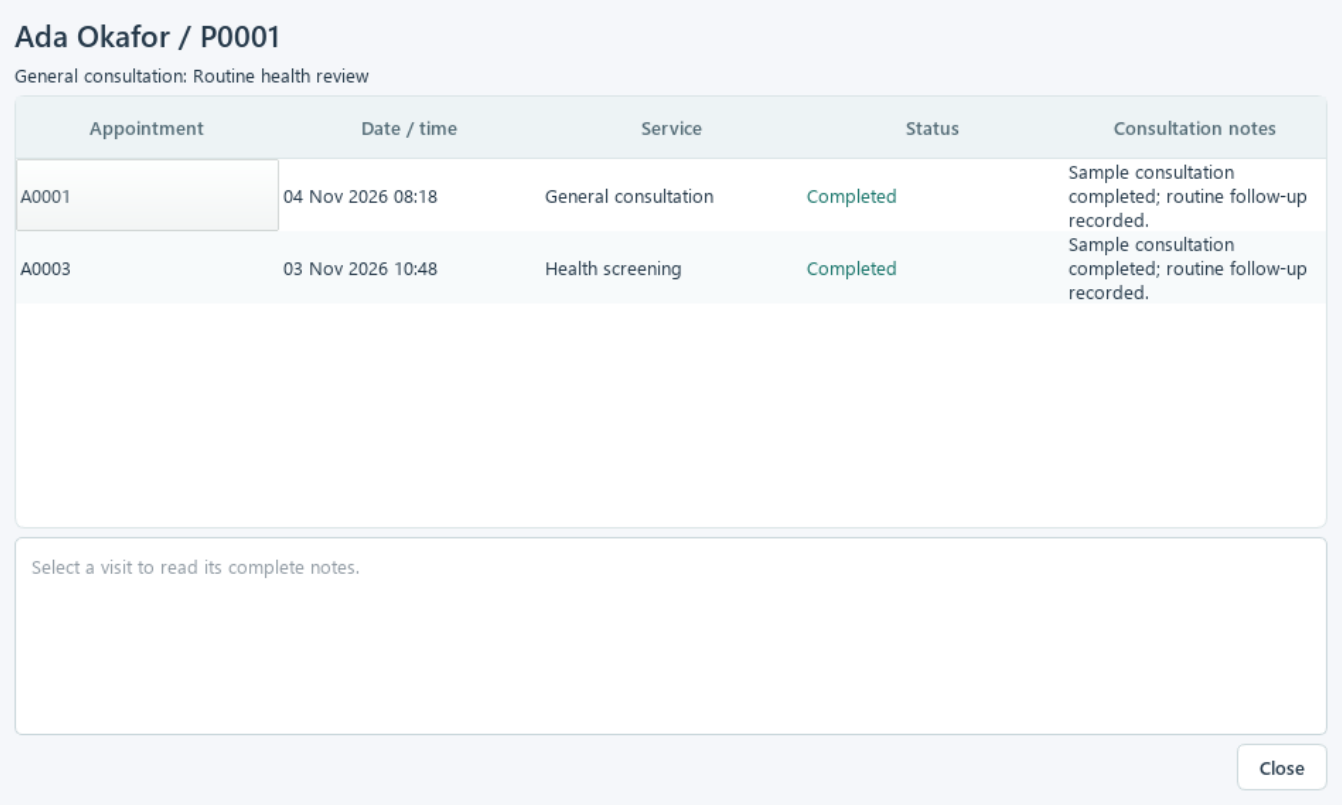
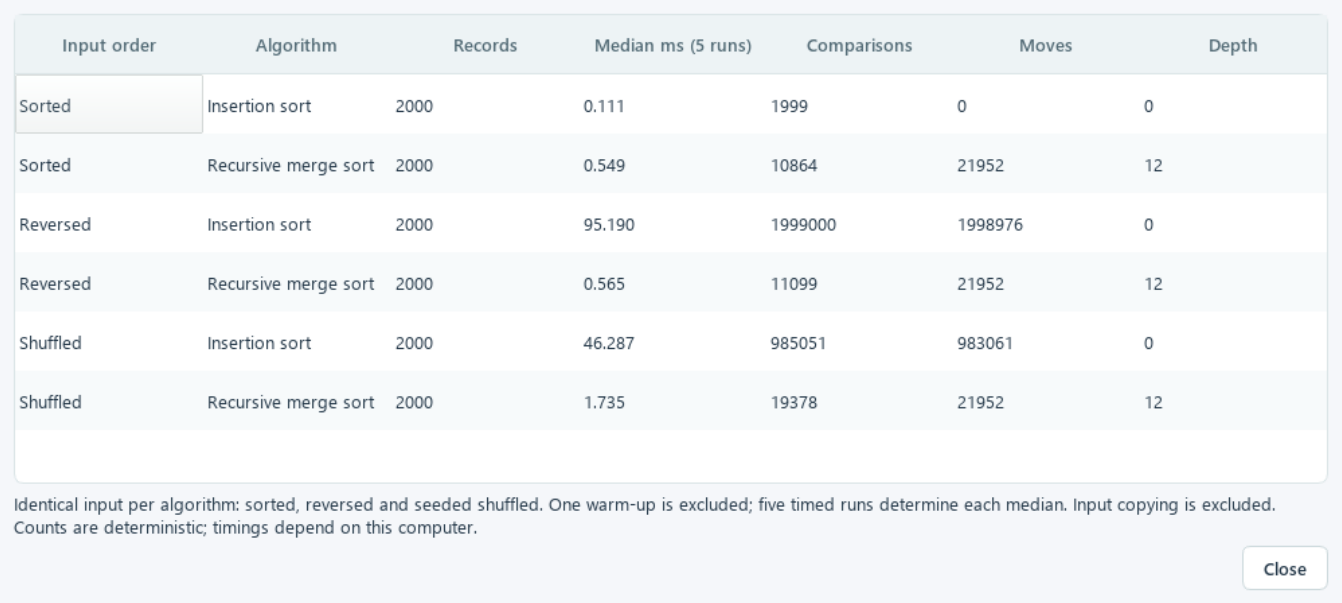


Figure 9. Patient history in the compiled SCAPMS application.

3.10 ALGORITHM COMPARISON



Input order	Algorithm	Records	Median ms (5 runs)	Comparisons	Moves	Depth
Sorted	Insertion sort	2000	0.111	1999	0	0
Sorted	Recursive merge sort	2000	0.549	10864	21952	12
Reversed	Insertion sort	2000	95.190	1999000	1998976	0
Reversed	Recursive merge sort	2000	0.565	11099	21952	12
Shuffled	Insertion sort	2000	46.287	985051	983061	0
Shuffled	Recursive merge sort	2000	1.735	19378	21952	12

Figure 10. Algorithm comparison in the compiled SCAPMS application. Both algorithms process the same 2,000 fictional records in three orders, with five measured runs per case.

3.11 CALENDAR ROOMS

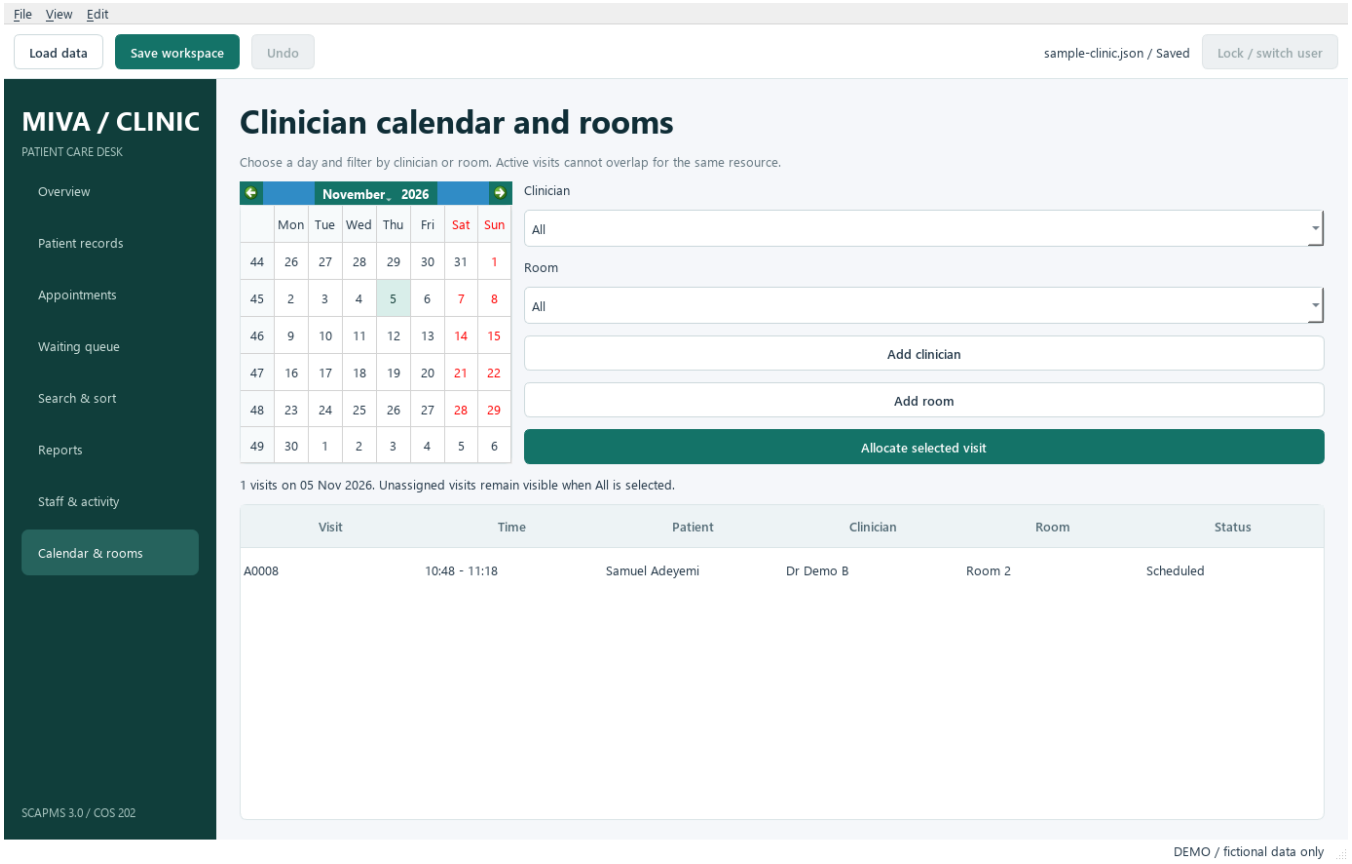


Figure 11. Calendar rooms in the compiled SCAPMS application.

3.12 WELCOME

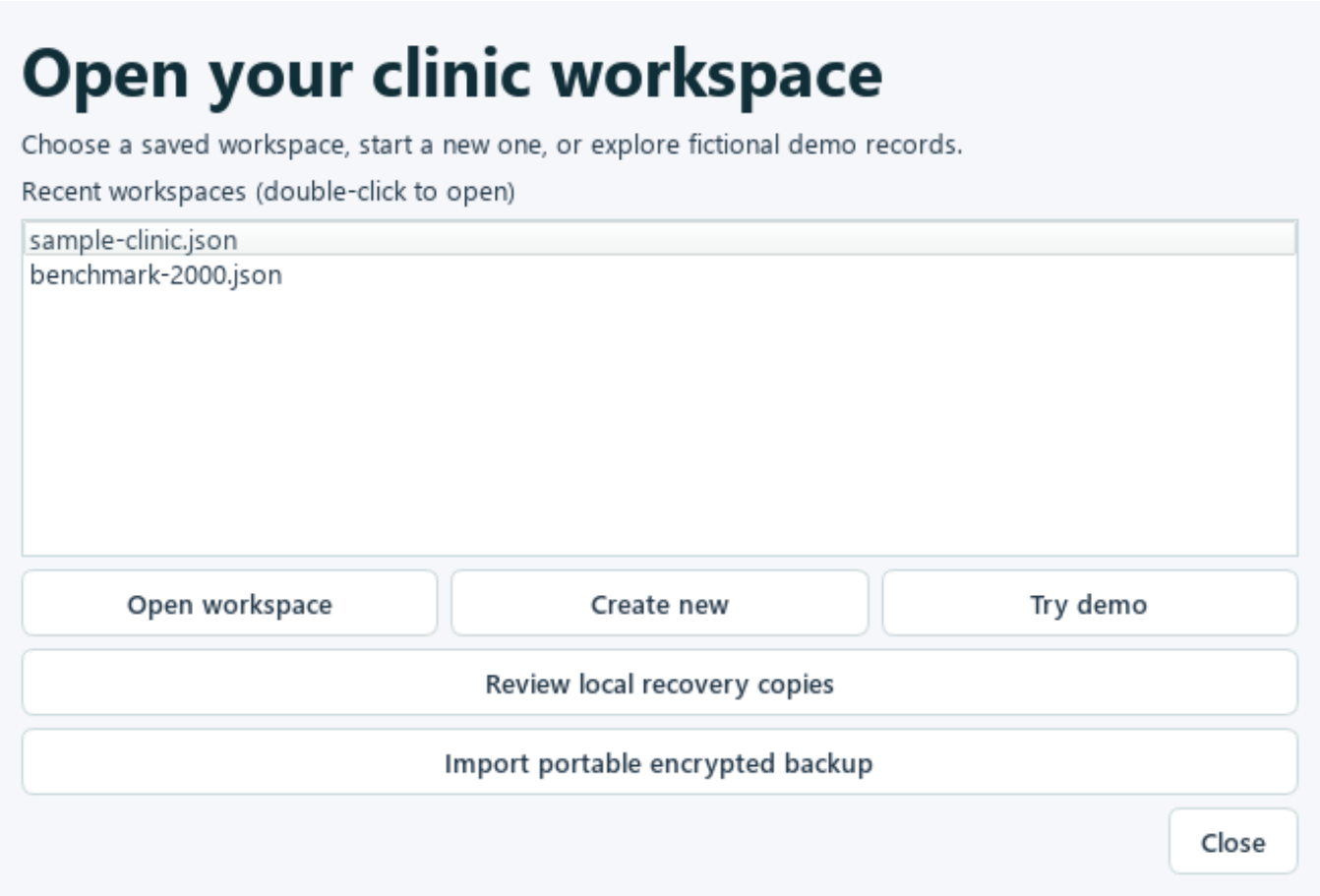


Figure 12. Welcome in the compiled SCAPMS application.

4.0 DEMONSTRATION AND REQUIREMENT COVERAGE

4.1 PATIENT RECORDS

Register each category; show dynamic fields, edit a record and open its history.

4.2 APPOINTMENTS AND QUEUE

Schedule and reschedule; reject an overlap; check in today's visit, call next, complete with notes and verify history.

4.3 UNDO AND ARCHIVE

Archive a patient, show retained historical visits, undo the archive and inspect the additional audit entry.

4.4 ALGORITHMS

Select manual searches and sorts. Compare five-run median timings and operation counts across three input orders using benchmark-2000.json.

4.5 REPORTS

Select a date range, review totals and export CSV/PDF or open the print dialog.

4.6 ACCESS AND RECOVERY

Protect a workspace and demonstrate staff restrictions, file locks and recovery previews. Export/import a portable backup and allocate clinicians/rooms in the calendar.

Shortcuts: Ctrl+N New, Ctrl+O Load, Ctrl+S Save, Ctrl+Shift+S Save As, Ctrl+Z Undo, Ctrl+F Search, Ctrl+Q Exit and Alt+1 through Alt+8 for panels.

5.0 SOURCE CODE AND RESOURCES

The following editable listings contain the complete application source, tests, build scripts and required demonstration data. Long lines are wrapped with a continuation marker. Original compilable files are also provided in the source archive. The large benchmark dataset is reproducible from its included generator.

5.1 SOURCE OR RESOURCE FILE

CMakeLists.txt

```
cmake_minimum_required(VERSION 3.21)
project(SCAPMS VERSION 3.0 LANGUAGES CXX)
set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_AUTOMOC ON)
find_package(Qt6 REQUIRED COMPONENTS Core Widgets Test Network PrintSupport)
add_library(clinic_core
    src/model/PatientRecord.cpp
    src/model/PatientTypes.cpp
    src/model/ClinicDatabase.cpp
    src/algorithms/Algorithms.cpp
    src/controller/ClinicManager.cpp
    src/controller/ReportService.cpp
    src/persistence/FileHandler.cpp
    src/persistence/RecoveryService.cpp
    src/persistence/WorkspaceGuard.cpp
    src/security/PortableBackup.cpp
    src/algorithms/Benchmark.cpp
    src/security/AccessControl.cpp)
target_include_directories(clinic_core PUBLIC src)
target_link_libraries(clinic_core PUBLIC Qt6::Core Qt6::Network)
if(WIN32)
    target_link_libraries(clinic_core PRIVATE crypt32 bcrypt)
endif()
file(GLOB GUI_SOURCES CONFIGURE_DEPENDS src/gui/*.cpp src/gui/panels/*.cpp)
add_library(clinic_gui ${GUI_SOURCES})
target_link_libraries(clinic_gui PUBLIC clinic_core Qt6::Widgets Qt6::PrintSupport)
add_executable(scapms WIN32 src/main.cpp)
target_link_libraries(scapms PRIVATE clinic_gui)
if(MSVC)
    target_compile_options(clinic_core PRIVATE /W4 /utf-8)
    target_compile_options(clinic_gui PRIVATE /W4 /utf-8)
else()
    target_compile_options(clinic_core PRIVATE -Wall -Wextra -Wpedantic)
    target_compile_options(clinic_gui PRIVATE -Wall -Wextra -Wpedantic)
endif()
enable_testing()
add_executable(clinic_tests tests/ClinicTests.cpp)
target_link_libraries(clinic_tests PRIVATE clinic_gui Qt6::Test)
target_compile_definitions(clinic_tests PRIVATE PROJECT_DIR="${CMAKE_CURRENT_SOURCE_DIR}")
add_test(NAME clinic_tests COMMAND clinic_tests -platform offscreen -o ${CMAKE_CURRENT_BINARY_DIR}/
 > test-results.txt,txt -o ${CMAKE_CURRENT_BINARY_DIR}/test-results.xml,junitxml)
set_tests_properties(clinic_tests PROPERTIES TIMEOUT 600 WORKING_DIRECTORY "${CMAKE_CURRENT_SOURCE_
 > DIR}")
install(TARGETS scapms RUNTIME DESTINATION .)


if(WIN32)
    set_tests_properties(clinic_tests PROPERTIES ENVIRONMENT "QT_QPA_FONTDIR=$ENV{WINDIR}/Fonts")
endif()
```

5.2 HEADER FILE

src/algorithms/Algorithms.h

```
#pragma once
#include "model/PatientRecord.h"
#include <functional>
```

```
#include <vector>

namespace clinic::algorithms {
using PatientView = std::vector<const model::PatientRecord *>;
enum class SortField { Name, Id, Age };
enum class SortAlgorithm { Insertion, Merge };
struct Metrics {
    size_t comparisons = 0;
    size_t moves = 0;
    size_t maxRecursionDepth = 0;
};
const model::PatientRecord *
linearSearchById(const std::vector<std::unique_ptr<model::PatientRecord>> &, const QString &,
                 Metrics * = nullptr);
const model::PatientRecord *binarySearchById(const PatientView &, const QString &,
                                             Metrics * = nullptr);
PatientView filterPatients(const PatientView &, const QString &text, bool byCategory);
PatientView viewOf(const std::vector<std::unique_ptr<model::PatientRecord>> &);
void sortPatients(PatientView &, SortField, SortAlgorithm, Metrics * = nullptr);
// Explicit implementations, also reusable by attendance reports.
template <class T, class Less>
void insertionSort(std::vector<T> &data, Less less, Metrics *metrics = nullptr) {
    for (size_t i = 1; i < data.size(); ++i) {
        T value = std::move(data[i]);
        size_t j = i;
        while (j > 0) {
            if (metrics)
                ++metrics->comparisons;
            if (!less(value, data[j - 1]))
                break;
            data[j] = std::move(data[j - 1]);
            --j;
            if (metrics)
                ++metrics->moves;
        }
        data[j] = std::move(value);
    }
}
template <class T, class Less>
void mergeRange(std::vector<T> &data, std::vector<T> &buffer, size_t begin, size_t end, Less less,
                Metrics *metrics, size_t depth) {
    if (metrics && depth > metrics->maxRecursionDepth)
        metrics->maxRecursionDepth = depth;
    if (end - begin < 2)
        return; // Recursive base case: zero or one item.
    const size_t middle = begin + (end - begin) / 2;
    mergeRange(data, buffer, begin, middle, less, metrics, depth + 1);
    mergeRange(data, buffer, middle, end, less, metrics, depth + 1);
    size_t left = begin, right = middle, out = begin;
    while (left < middle && right < end) {
        if (metrics)
            ++metrics->comparisons;
        // Left wins ties, preserving stability.
        buffer[out++] = less(data[right], data[left]) ? data[right++] : data[left++];
    }
    while (left < middle)
        buffer[out++] = data[left++];
    while (right < end)
        buffer[out++] = data[right++];
    for (size_t i = begin; i < end; ++i) {
        data[i] = buffer[i];
        if (metrics)
            ++metrics->moves;
    }
}
template <class T, class Less>
void mergeSort(std::vector<T> &data, Less less, Metrics *metrics = nullptr) {
    if (data.empty())
        return;
    auto buffer = data;
    mergeRange(data, buffer, 0, data.size(), less, metrics, 1);
}
} // namespace clinic::algorithms
```

5.3 HEADER FILE

src/algorithms/Benchmark.h

```
#pragma once
#include "Algorithms.h"
#include <QString>
namespace clinic::algorithms {
struct BenchmarkResult {
    QString order;
    SortAlgorithm algorithm;
    int records = 0;
    double medianMs = 0;
    Metrics metrics;
};
std::vector<BenchmarkResult> benchmarkSorts(const PatientView &, SortField, int repeats = 5);
} // namespace clinic::algorithms
```

5.4 HEADER FILE

src/controller/ClinicManager.h

```
#pragma once
#include "model/ClinicDatabase.h"
#include <functional>
#include <stack>
namespace clinic::controller {
struct UndoSnapshot {
    model::ClinicDatabase state;
    QString description;
};
class ClinicManager {
  public:
    const model::ClinicDatabase &database() const {
        return database_;
    }
    QString suggestedPatientId() const;
    void addPatient(model::PatientCategory, model::PatientDetails);
    void editPatient(const QString &, model::PatientCategory, model::PatientDetails);
    void deletePatient(const QString &); // Archives; never erases history.
    void restorePatient(const QString &);
    QString schedule(const QString &, const QString &, const QDateTime &, int,
                     const QString &clinician = {}, const QString &room = {});
    void reschedule(const QString &, const QDateTime &, int);
    void checkIn(const QString &);
    void cancel(const QString &, const QString &reason = {});
    void noShow(const QString &, const QString &);
    void correctNotes(const QString &, const QString &, const QString &);
    void assignResources(const QString &, const QString &, const QString &);
    void addResource(const QString &, bool room);
    QStringList possibleDuplicates(const model::PatientDetails &,
                                   const QString &exclude = {}) const;
    bool canExportReports() const {
        return canViewClinicalNotes();
    }
    void requireReportExport() const;
    void recordExport(const QString &);
    void complete(const QString &, const QString &);
    void enqueue(const QString &);
    QString callNext();
    void removeFromQueue(const QString &);
    bool canUndo() const {
        return !history_.empty() && canEdit();
    }
    QString undoDescription() const {
        return canUndo() ? history_.top().description : QString();
    }
    void undo();
    void replace(model::ClinicDatabase, const QString &username = {}, const QString &password = {})
 > ;
    void newDatabase();
    void access(const QString &id) {
        database_.recordAccess(id);
```

```
    }
    bool dirty() const {
        return dirty_;
    }
    void markSaved() {
        dirty_ = false;
    }
    void markUnsaved() {
        dirty_ = true;
    }
    bool demoMode() const {
        return database_.users.empty();
    }
    bool authenticated() const {
        return demoMode() || !username_.isEmpty();
    }
    QString username() const {
        return demoMode() ? "Demo operator" : username_;
    }
    security::Role role() const {
        return demoMode() ? security::Role::Administrator : role_;
    }
    bool canEdit() const {
        return authenticated() && role() != security::Role::Viewer;
    }
    bool isAdmin() const {
        return authenticated() && role() == security::Role::Administrator;
    }
    bool canViewClinicalNotes() const {
        return authenticated() && (isAdmin() || role() == security::Role::Clinician);
    }
    void configureAdministrator(const QString &, const QString &);
    void addUser(const QString &, const QString &, security::Role);
    void setUserActive(const QString &, bool);
    void changePassword(const QString &current, const QString &replacement);
    void login(const QString &, const QString &);
    void lock();

  private:
    model::ClinicDatabase database_;
    std::stack<UndoSnapshot> history_;
    bool dirty_ = false;
    QString username_;
    security::Role role_ = security::Role::Viewer;
    int failedLogins_ = 0;
    QDateTime lockedUntil_;
    void transact(const QString &, const std::function<void(model::ClinicDatabase &)> &);
    void requireEdit() const;
    void audit(const QString &);
};
} // namespace clinic::controller
```

5.5 HEADER FILE

src/controller/ReportService.h

```
#pragma once
#include "model/ClinicDatabase.h"
#include <QStringList>
namespace clinic::controller {
enum class ReportType { Attendance, Pending, Categories, Daily };
struct Report {
    QString title;
    QStringList headers;
    QList<QStringList> rows;
    QString summary;
};
Report generateReport(const model::ClinicDatabase &, ReportType, QDate from = {}, QDate to = {});
QString reportCsv(const Report &);
QString reportHtml(const Report &);
} // namespace clinic::controller
```

5.6 HEADER FILE

src/gui/MainWindow.h

```
#pragma once
#include "controller/ClinicManager.h"
#include <QElapsedTimer>
#include <QMainWindow>
#include <functional>
#include <vector>
#include "persistence/WorkspaceGuard.h"
class QListWidget;
class QStackedWidget;
class QLabel;
class QPushButton;
class QToolBar;
class QAction;
namespace clinic::gui {
class PanelBase;
class MainWindow final : public QMainWindow {
  public:
    explicit MainWindow(QWidget *parent = nullptr);
    ~MainWindow() override;
    void loadPath(const QString &, const QString &username = {}, const QString &password = {});
    bool openWorkspace(const QString &);
    void captureScreenshots(const QString &);
    controller::ClinicManager &manager() {
        return manager_;
    }
    void refresh();
    void autosaveNow();
    bool welcome();
    void offerRecovery();

  protected:
    void closeEvent(QCloseEvent *) override;
    bool eventFilter(QObject *, QEvent *) override;

  private:
    controller::ClinicManager manager_;
    std::unique_ptr<persistence::WorkspaceGuard> workspaceGuard_;
    void rememberWorkspace();
    void portableBackup(bool importing);
    void setTextSize(int);
    QString currentPath_, recoveryError_;
    QListWidget *navigation_;
    QStackedWidget *pages_;
    QLabel *fileLabel_, *sessionLabel_;
    QPushButton *undoButton_, *lockButton_;
    QAction *undoAction_;
    QToolBar *toolbar_;
    std::vector<PanelBase *> panels_;
    QElapsedTimer idle_;
    bool locking_ = false;
    void run(const std::function<void()> &, const QString &);
    void prepareChange();
    void updateTitle();
    bool save(bool choosePath = false);
    bool confirmDiscard();
    void openFile();
    void recover();
    void protectWorkspace();
    void lockWorkspace();
};
} // namespace clinic::gui
```

5.7 HEADER FILE

src/gui/panels/AccessPanel.h

```
#pragma once
#include "PanelBase.h"
namespace clinic::gui {
```

```
 4 class AccessPanel final : public PanelBase {
 5   public:
 6     AccessPanel(controller::ClinicManager &, MutationHandler, std::function<void()> secureWorkspace
 > ,
 7                 QWidget *parent = nullptr);
 8     void refresh() override;
 9
10   private:
11     QLabel *state_;
12     QTableWidget *users_, *audit_;
13     void addAccount();
14     void changePassword();
15 };
16 } // namespace clinic::gui
```

5.8 HEADER FILE

src/gui/panels/AppointmentsPanel.h

```
 1 #pragma once
 2 #include "PanelBase.h"
 3 namespace clinic::gui {
 4 class AppointmentsPanel final : public PanelBase {
 5   public:
 6     AppointmentsPanel(controller::ClinicManager &, MutationHandler, QWidget *parent = nullptr);
 7     void refresh() override;
 8
 9   private:
10     QComboBox *patient_, *service_, *status_, *clinician_, *room_;
11     QDateTimeEdit *time_;
12     QSpinBox *duration_;
13     QCheckBox *today_;
14     QTableWidget *table_;
15     void reschedule();
16 };
17 } // namespace clinic::gui
```

5.9 HEADER FILE

src/gui/panels/CalendarPanel.h

```
 1 #pragma once
 2 #include "PanelBase.h"
 3 namespace clinic::gui {
 4 class CalendarPanel final : public PanelBase {
 5   public:
 6     CalendarPanel(controller::ClinicManager &, MutationHandler, QWidget * = nullptr);
 7     void refresh() override;
 8
 9   private:
10     QCalendarWidget *calendar_;
11     QComboBox *clinician_, *room_;
12     QTableWidget *table_;
13     QLabel *summary_;
14     void allocate();
15 };
16 } // namespace clinic::gui
```

5.10 HEADER FILE

src/gui/panels/DashboardPanel.h

```
 1 #pragma once
 2 #include "PanelBase.h"
 3 namespace clinic::gui {
 4 class DashboardPanel final : public PanelBase {
 5   public:
 6     DashboardPanel(controller::ClinicManager &, MutationHandler, QWidget *parent = nullptr);
 7     void refresh() override;
 8
 9   private:
10     QLabel *patients_, *pending_, *waiting_, *completed_, *reminder_;
```

```
11      QTableWidget *today_, *overdue_, *waits_, *cache_;
12  };
13  } // namespace clinic::gui
```

5.11 HEADER FILE

src/gui/panels/PanelBase.h

```
 1  #pragma once
 2  #include "controller/ClinicManager.h"
 3  #include <QtWidgets>
 4  #include <functional>
 5  namespace clinic::gui {
 6  using MutationHandler = std::function<void(const std::function<void()> &, const QString &)>;
 7  class PanelBase : public QWidget {
 8    public:
 9      PanelBase(controller::ClinicManager &manager, MutationHandler mutate, QWidget *parent = nullptr
   > )
10          : QWidget(parent), manager_(manager), mutate_(std::move(mutate)) {}
11      virtual void refresh() = 0;
12      virtual void beforeChange() {}
13
14    protected:
15      controller::ClinicManager &manager_;
16      MutationHandler mutate_;
17      bool updating_ = false;
18  };
19  QVBoxLayout *panelLayout(QWidget *, const QString &, const QString &);
20  QPushButton *actionButton(const QString &, const QString &, bool primary = false);
21  QTableWidget *makeTable(const QStringList &, const QString &);
22  void configureTable(QTableView *);
23  void fillTable(QTableWidget *, const QList<QStringList> &);
24  QString selectedId(QTableWidget *);
25  void selectId(QTableWidget *, const QString &);
26  void fillPatientChoices(QComboBox *, const model::ClinicDatabase &);
27  void showPatientHistory(QWidget *, controller::ClinicManager &, const QString &);
28  } // namespace clinic::gui
```

5.12 HEADER FILE

src/gui/panels/PatientsPanel.h

```
 1  #pragma once
 2  #include "PanelBase.h"
 3  #include "gui/PatientTableModel.h"
 4  namespace clinic::gui {
 5  class PatientsPanel final : public PanelBase {
 6    public:
 7      PatientsPanel(controller::ClinicManager &, MutationHandler, QWidget *parent = nullptr);
 8      void refresh() override;
 9      void beforeChange() override;
10      void editPatient(bool edit);
11
12    private:
13      PatientTableModel *model_;
14      QTableView *table_;
15      QCheckBox *archived_;
16      QLabel *details_;
17      QString selected_;
18      int scroll_ = 0;
19      QString selection() const;
20  };
21  } // namespace clinic::gui
```

5.13 HEADER FILE

src/gui/panels/QueuePanel.h

```
 1  #pragma once
 2  #include "PanelBase.h"
 3  namespace clinic::gui {
 4  class QueuePanel final : public PanelBase {
```

```
  public:
    QueuePanel(controller::ClinicManager &, MutationHandler, QWidget *parent = nullptr);
    void refresh() override;
    void resetMessage();

  private:
    QComboBox *patient_;
    QTableWidget *table_;
    QLabel *lastCalled_;
};
} // namespace clinic::gui
```

5.14 HEADER FILE

src/gui/panels/ReportsPanel.h

```
#pragma once
#include "PanelBase.h"
#include "controller/ReportService.h"
namespace clinic::gui {
class ReportsPanel final : public PanelBase {
  public:
    ReportsPanel(controller::ClinicManager &, MutationHandler, QWidget *parent = nullptr);
    void refresh() override;

  private:
    QComboBox *type_;
    QDateEdit *from_, *to_;
    QCheckBox *allDates_;
    QLabel *title_, *summary_;
    QTableWidget *table_;
    controller::Report current_;
    void exportPdf();
    void print();
};
} // namespace clinic::gui
```

5.15 HEADER FILE

src/gui/panels/SearchPanel.h

```
#pragma once
#include "PanelBase.h"
#include "gui/PatientTableModel.h"
namespace clinic::gui {
class SearchPanel final : public PanelBase {
  public:
    SearchPanel(controller::ClinicManager &, MutationHandler, QWidget *parent = nullptr);
    void beforeChange() override;
    void refresh() override;

  private:
    PatientTableModel *model_;
    QTableView *table_;
    QComboBox *field_, *algorithm_, *sortField_, *sortAlgorithm_;
    QLineEdit *text_;
    QLabel *info_;
    QCheckBox *archived_;
    enum class ViewMode { All, Search, Sort };
    ViewMode mode_ = ViewMode::All;
    QString committedQuery_, selected_;
    int committedField_ = 0, committedAlgorithm_ = 0, committedSortField_ = 0,
        committedSortAlgorithm_ = 0, scroll_ = 0;
    void rebuild();
    void benchmark();
};
} // namespace clinic::gui
```

5.16 HEADER FILE

src/gui/PatientDialog.h

```
#pragma once
#include "model/PatientRecord.h"
#include "model/ClinicDatabase.h"
#include <QDialog>
class QLineEdit;
class QSpinBox;
class QComboBox;
class QLabel;
namespace clinic::gui {
class PatientDialog final : public QDialog {
  public:
    explicit PatientDialog(const QString &suggestedId,
                           const model::PatientRecord *patient = nullptr,
                           QWidget *parent = nullptr);
    model::PatientDetails details() const;
    model::PatientCategory category() const;
    void configureAccess(bool, const model::ClinicDatabase &);

  private:
    QLineEdit *id_, *name_, *contact_, *reason_, *previous_;
    QSpinBox *age_, *urgency_;
    QComboBox *category_;
    QComboBox *previousChoices_ = nullptr;
    bool clinicalAccess_ = true;
    QLabel *urgencyLabel_, *previousLabel_, *error_;
    void updateFields();
};
} // namespace clinic::gui
```

5.17 HEADER FILE

src/gui/PatientTableModel.h

```
#pragma once
#include "algorithms/Algorithms.h"
#include <QAbstractTableModel>
namespace clinic::gui {
class PatientTableModel final : public QAbstractTableModel {
  public:
    explicit PatientTableModel(QObject *parent = nullptr) : QAbstractTableModel(parent) {}
    int rowCount(const QModelIndex &parent = {}) const override;
    int columnCount(const QModelIndex &parent = {}) const override;
    QVariant data(const QModelIndex &, int role = Qt::DisplayRole) const override;
    QVariant headerData(int, Qt::Orientation, int role = Qt::DisplayRole) const override;
    void setPatients(algorithms::PatientView patients);
    QString idAt(int row) const;
    void setClinicalAccess(bool allowed) {
        clinicalAccess_ = allowed;
    }

  private:
    algorithms::PatientView patients_;
    bool clinicalAccess_ = false;
};
} // namespace clinic::gui
```

5.18 HEADER FILE

src/gui/Theme.h

```
#pragma once
namespace clinic::gui {
inline const char *theme = R"(
QMainWindow,QDialog { background:#f5f7fa; color:#243849; }
QWidget { font-family:'Segoe UI'; font-size:10pt; color:#243849; }
QLabel#pageTitle { font-size:25pt; font-weight:650; color:#102e37; }
QLabel#sectionTitle { font-size:16pt; font-weight:600; }
QLabel#caption { color:#657781; }
```

```
 9 QLabel#metric { font-size:29pt; font-weight:650; color:#125c55; }
10 QLabel#brand { color:white; font-size:20pt; font-weight:700; }
11 QLabel#brandCaption { color:#adc9c4; font-size:9pt; }
12 QFrame#sidebar { background:#103e3b; }
13 QListWidget { background:white; color:#243849; border:1px solid #cbd8dc; selection-background-color
 > :#d9eeea; selection-color:#133e39; }
14 QListWidget#navigation { background:transparent; border:0; color:#cee1dd; outline:0; }
15 QListWidget#navigation::item { padding:15px 14px; margin:3px 0; border-radius:7px; }
16 QListWidget#navigation::item:selected { background:#26635d; color:white; }
17 QListWidget#navigation::item:hover { background:#20524d; }
18 QFrame#card { background:white; border:1px solid #dce6e8; border-radius:10px; }
19 QPushButton { background:white; border:1px solid #cedadd; border-radius:6px; padding:7px 15px; font
 > -weight:600; }
20 QPushButton:hover { background:#e8f2f0; border-color:#719d95; }
21 QPushButton[primary="true"] { background:#147368; border-color:#147368; color:white; }
22 QPushButton[primary="true"]:hover { background:#105d54; }
23 QPushButton:disabled { color:#98a7aa; background:#edf0f2; border-color:#e0e5e7; }
24 QLineEdit,QSpinBox,QComboBox,QDateTimeEdit,QDateEdit,QTextEdit {
25 background:white; border:1px solid #cbd8dc; border-radius:5px; padding:7px; min-height:22px; }
26 QLineEdit:focus,QComboBox:focus { border:1px solid #147368; }
27 QTableView { background:white; alternate-background-color:#f7fafb; border:1px solid #dce6e8; border
 > -radius:7px;
28 selection-background-color:#d9eeea; selection-color:#133e39; }
29 QHeaderView::section { background:#edf3f5; color:#536b76; border:0; border-bottom:1px solid #dce6e8
 > ; padding:11px 6px; font-weight:600; }
30 QMenu { background:white; color:#243849; border:1px solid #cbd8dc; }
31 QMenu::item:selected { background:#d9eeea; color:#133e39; }
32 QTabBar::tab { background:#edf3f5; color:#243849; padding:7px 12px; border:1px solid #cbd8dc; }
33 QTabBar::tab:selected { background:white; color:#103e3b; }
34 QTabWidget::pane { border:1px solid #dce6e8; background:white; }
35 QComboBox QAbstractItemView { background:white; color:#243849; selection-background-color:#d9eeea;
 > selection-color:#133e39; }
36 QCalendarWidget QToolButton { color:white; background:#147368; }
37 QStatusBar { background:white; color:#657781; }
38 QToolBar { background:white; border-bottom:1px solid #dce6e8; spacing:12px; padding:10px; }
39 )";
40 }
```

5.19 HEADER FILE

src/model/Appointable.h

```
 1 #pragma once
 2 #include "Appointment.h"
 3 namespace clinic::model {
 4 // Scheduling/cancellation policy is polymorphic; the controller owns transactions.
 5 class Appointable {
 6   public:
 7     virtual ~Appointable() = default;
 8     virtual void validateScheduling(const Appointment &appointment) const = 0;
 9     virtual void validateCancellation(const Appointment &appointment) const = 0;
10 };
11 } // namespace clinic::model
```

5.20 HEADER FILE

src/model/Appointment.h

```
 1 #pragma once
 2 #include <QDateTime>
 3 #include <QString>
 4 #include <QStringList>
 5
 6 namespace clinic::model {
 7 enum class AppointmentStatus { Scheduled, Completed, Cancelled, CheckedIn, InConsultation, NoShow }
 > ;
 8 inline QString statusName(AppointmentStatus status) {
 9     switch (status) {
10     case AppointmentStatus::NoShow:
11         return "No-show";
12     case AppointmentStatus::CheckedIn:
13         return "Checked in";
14     case AppointmentStatus::InConsultation:
```

```
        return "In consultation";
    case AppointmentStatus::Scheduled:
        return "Scheduled";
    case AppointmentStatus::Completed:
        return "Completed";
    case AppointmentStatus::Cancelled:
        return "Cancelled";
    }
    return "Unknown";
}
inline bool isPending(AppointmentStatus s) {
    return s == AppointmentStatus::Scheduled || s == AppointmentStatus::CheckedIn ||
           s == AppointmentStatus::InConsultation;
}
struct Appointment {
    QString id;
    QString patientId;
    QString service;
    QDateTime start;
    int durationMinutes = 30;
    AppointmentStatus status = AppointmentStatus::Scheduled;
    QString consultationNotes;
    QString changeReason, clinician, room;
    QStringList corrections;
    bool walkIn = false;
    QDateTime consultationStarted, completedAt;
    QDateTime end() const {
        return start.addSecs(durationMinutes * 60);
    }
};
struct QueueEntry {
    QString patientId;
    QDateTime arrivedAt;
    QString appointmentId;
};
} // namespace clinic::model
```

5.21 HEADER FILE

src/model/ClinicDatabase.h

```
#pragma once
#include "Appointment.h"
#include "PatientRecord.h"
#include "security/AccessControl.h"
#include <QSet>
#include <array>
#include <queue>
#include <vector>

namespace clinic::model {
struct CacheEntry {
    QString patientId;
    int accesses = 0;
    quint64 lastAccess = 0;
};
class ClinicDatabase {
  public:
    std::vector<std::unique_ptr<PatientRecord>> patients;
    std::vector<Appointment> appointments;
    std::queue<QueueEntry> waiting;
    std::array<CacheEntry, 10> frequent{};
    QSet<QString> archivedPatients;
    QStringList clinicians, rooms;
    std::vector<security::UserAccount> users;
    std::vector<security::AuditEntry> audit;
    quint64 accessClock = 0;
    int nextPatientId = 1;
    int nextAppointmentId = 1;
    ClinicDatabase() = default;
    ClinicDatabase(const ClinicDatabase &other);
    ClinicDatabase &operator=(const ClinicDatabase &other);
    ClinicDatabase(ClinicDatabase &&) noexcept = default;
    ClinicDatabase &operator=(ClinicDatabase &&) noexcept = default;
```

```
34      const PatientRecord *patient(const QString &id) const;
35      Appointment *appointment(const QString &id);
36      const Appointment *appointment(const QString &id) const;
37      void recordAccess(const QString &id);
38      void validate() const;
39  };
40  } // namespace clinic::model
```

5.22 HEADER FILE

src/model/EmergencyPatient.h

```
 1  #pragma once
 2  #include "Appointable.h"
 3  #include "PatientRecord.h"
 4  namespace clinic::model {
 5  class EmergencyPatient final : public PatientRecord, public Appointable {
 6    public:
 7      explicit EmergencyPatient(PatientDetails details);
 8      PatientCategory category() const override {
 9          return PatientCategory::Emergency;
10      }
11      QString consultationSummary() const override;
12      std::unique_ptr<PatientRecord> clone() const override;
13      void validateScheduling(const Appointment &) const override;
14      void validateCancellation(const Appointment &) const override;
15  };
16  } // namespace clinic::model
```

5.23 HEADER FILE

src/model/Errors.h

```
 1  #pragma once
 2  #include <QString>
 3  #include <stdexcept>
 4  namespace clinic::model {
 5  class ClinicError : public std::runtime_error {
 6    public:
 7      explicit ClinicError(const QString &message) : std::runtime_error(message.toStdString()) {}
 8  };
 9  class ValidationError : public ClinicError {
10      using ClinicError::ClinicError;
11  };
12  class FileError : public ClinicError {
13      using ClinicError::ClinicError;
14  };
15  } // namespace clinic::model
```

5.24 HEADER FILE

src/model/FollowUpPatient.h

```
 1  #pragma once
 2  #include "Appointable.h"
 3  #include "PatientRecord.h"
 4  namespace clinic::model {
 5  class FollowUpPatient final : public PatientRecord, public Appointable {
 6    public:
 7      explicit FollowUpPatient(PatientDetails details);
 8      PatientCategory category() const override {
 9          return PatientCategory::FollowUp;
10      }
11      QString consultationSummary() const override;
12      std::unique_ptr<PatientRecord> clone() const override;
13      void validateScheduling(const Appointment &) const override;
14      void validateCancellation(const Appointment &) const override;
15  };
16  } // namespace clinic::model
```

5.25 HEADER FILE

src/model/GeneralPatient.h

```
#pragma once
#include "Appointable.h"
#include "PatientRecord.h"
namespace clinic::model {
class GeneralPatient final : public PatientRecord, public Appointable {
  public:
    explicit GeneralPatient(PatientDetails details);
    PatientCategory category() const override {
        return PatientCategory::General;
    }
    QString consultationSummary() const override;
    std::unique_ptr<PatientRecord> clone() const override;
    void validateScheduling(const Appointment &) const override;
    void validateCancellation(const Appointment &) const override;
};
} // namespace clinic::model
```

5.26 HEADER FILE

src/model/PatientRecord.h

```
#pragma once
#include <QString>
#include <memory>

namespace clinic::model {
enum class PatientCategory { General, Emergency, FollowUp };
QString categoryName(PatientCategory category);
PatientCategory parseCategory(const QString &value);
struct PatientDetails {
    QString id;
    QString name;
    int age = 0;
    QString contact;
    QString reason;
    int urgency = 1;
    QString previousVisit;
};
class PatientRecord {
  public:
    explicit PatientRecord(PatientDetails details);
    virtual ~PatientRecord() = default;
    const PatientDetails &details() const {
        return details_;
    }
    const QString &id() const {
        return details_.id;
    }
    virtual PatientCategory category() const = 0;
    virtual QString consultationSummary() const = 0;
    virtual std::unique_ptr<PatientRecord> clone() const = 0;

  protected:
    PatientDetails details_;
};
std::unique_ptr<PatientRecord> makePatient(PatientCategory category, PatientDetails details);
QString processPatient(const PatientRecord &patient);
} // namespace clinic::model
```

5.27 HEADER FILE

src/persistence/FileHandler.h

```
#pragma once
#include "model/ClinicDatabase.h"
#include <QJsonObject>
namespace clinic::persistence {
class FileHandler {
```

```
  public:
    static model::ClinicDatabase load(const QString &path);
    static void save(const model::ClinicDatabase &, const QString &path);
    static QJsonObject serialize(const model::ClinicDatabase &);
    static model::ClinicDatabase deserialize(const QJsonObject &);
    static void writeText(const QString &path, const QString &text);
};
} // namespace clinic::persistence
```

5.28 HEADER FILE

src/persistence/RecoveryService.h

```
#pragma once
#include "FileHandler.h"
namespace clinic::persistence {
class RecoveryService {
  public:
    static QString recoveryPath(const QString &workspace);
    static void autosave(const model::ClinicDatabase &, const QString &workspace);
    static void saveWithBackup(const model::ClinicDatabase &, const QString &workspace);
    static QStringList copies(const QString &workspace);
    static void discardRecovery(const QString &workspace);
};
} // namespace clinic::persistence
```

5.29 HEADER FILE

src/persistence/WorkspaceGuard.h

```
#pragma once
#include <QByteArray>
#include <QLockFile>
#include <QString>
#include <memory>
namespace clinic::persistence {
// One writer per canonical file, plus a fingerprint against non-cooperating editors.
class WorkspaceGuard {
  public:
    explicit WorkspaceGuard(const QString &);
    void verifyUnchanged() const;
    void checkpoint();
    QString path() const {
        return path_;
    }

  private:
    QString path_;
    std::unique_ptr<QLockFile> lock_;
    QByteArray fingerprint_;
    bool existed_ = false;
    QByteArray digest() const;
};
} // namespace clinic::persistence
```

5.30 HEADER FILE

src/security/AccessControl.h

```
#pragma once
#include <QByteArray>
#include <QDateTime>
#include <QString>
namespace clinic::security {
enum class Role { Administrator, Clinician, Receptionist, Viewer };
QString roleName(Role);
Role parseRole(const QString &);
struct UserAccount {
    QString username;
    Role role = Role::Viewer;
    QByteArray salt;
    QByteArray passwordHash;
```

```
14     int iterations = 210000;
15     bool active = true;
16 };
17 struct AuditEntry {
18     QDateTime time;
19     QString actor;
20     QString action;
21 };
22 UserAccount createAccount(const QString &, const QString &, Role);
23 bool verifyPassword(const UserAccount &, const QString &);
24 QByteArray protect(const QByteArray &);
25 QByteArray unprotect(const QByteArray &);
26 } // namespace clinic::security
```

5.31 HEADER FILE

src/security/PortableBackup.h

```
1 #pragma once
2 #include <QByteArray>
3 #include <QString>
4 namespace clinic::security {
5 // Versioned AES-256-GCM backup; independent of DPAPI and the Windows account.
6 QByteArray encryptBackup(const QByteArray &, const QString &);
7 QByteArray decryptBackup(const QByteArray &, const QString &);
8 } // namespace clinic::security
```

5.32 SOURCE OR RESOURCE FILE

src/algorithms/Algorithms.cpp

```
#include "Algorithms.h"

namespace clinic::algorithms {
const model::PatientRecord *
linearSearchById(const std::vector<std::unique_ptr<model::PatientRecord>> &data, const QString &id,
                 Metrics *metrics) {
    for (const auto &patient : data) {
        if (metrics)
            ++metrics->comparisons;
        if (patient->id() == id)
            return patient.get();
    }
    return nullptr;
}
const model::PatientRecord *binarySearchById(const PatientView &sorted, const QString &id,
                                             Metrics *metrics) {
    // Precondition: ascending, case-sensitive ID order; GUI creates this view explicitly.
    size_t low = 0, high = sorted.size();
    while (low < high) {
        const size_t middle = low + (high - low) / 2;
        if (metrics)
            ++metrics->comparisons;
        const int comparison = QString::compare(sorted[middle]->id(), id, Qt::CaseSensitive);
        if (comparison == 0)
            return sorted[middle];
        if (comparison < 0)
            low = middle + 1;
        else
            high = middle;
    }
    return nullptr;
}
PatientView filterPatients(const PatientView &data, const QString &text, bool byCategory) {
    PatientView matches;
    for (const auto *patient : data) {
        const auto field =
            byCategory ? model::categoryName(patient->category()) : patient->details().name;
        if (field.contains(text, Qt::CaseInsensitive))
            matches.push_back(patient);
    }
    return matches;
}
PatientView viewOf(const std::vector<std::unique_ptr<model::PatientRecord>> &data) {
    PatientView result;
    for (const auto &patient : data)
        result.push_back(patient.get());
    return result;
}
void sortPatients(PatientView &data, SortField field, SortAlgorithm algorithm, Metrics *metrics) {
    const auto less = [field](const model::PatientRecord *a, const model::PatientRecord *b) {
        switch (field) {
        case SortField::Name:
            return QString::compare(a->details().name, b->details().name, Qt::CaseInsensitive) < 0;
        case SortField::Age:
            return a->details().age < b->details().age;
        case SortField::Id:
            return QString::compare(a->id(), b->id(), Qt::CaseSensitive) < 0;
        }
        return false;
    };
    if (algorithm == SortAlgorithm::Insertion)
        insertionSort(data, less, metrics);
    else
        mergeSort(data, less, metrics);
}
} // namespace clinic::algorithms
```

5.33 SOURCE OR RESOURCE FILE

src/algorithms/Benchmark.cpp

```
#include "Benchmark.h"
#include "model/Errors.h"
#include <QElapsedTimer>
#include <algorithm>
#include <random>
namespace clinic::algorithms {
std::vector<BenchmarkResult> benchmarkSorts(const PatientView &input, SortField field,
                                            int repeats) {
    if (input.empty() || input.size() > 10000 || repeats < 3 || repeats > 9 || repeats % 2 == 0)
        throw model::ValidationError("Use 1-10000 records and an odd repeat count from 3 to 9.");
    std::vector<BenchmarkResult> rows;
    auto sorted = input;
    sortPatients(sorted, field, SortAlgorithm::Merge);
    for (const auto &order : {QString("Sorted"), QString("Reversed"), QString("Shuffled")}) {
        auto base = sorted;
        if (order == "Reversed")
            std::reverse(base.begin(), base.end());
        if (order == "Shuffled") {
            std::mt19937 rng(2026);
            std::shuffle(base.begin(), base.end(), rng);
        }
        for (auto algorithm : {SortAlgorithm::Insertion, SortAlgorithm::Merge}) {
            // Warm up once; input preparation and copying are outside measured time.
            auto warmup = base;
            sortPatients(warmup, field, algorithm);
            std::vector<double> timings;
            Metrics metrics;
            for (int run = 0; run < repeats; ++run) {
                auto copy = base;
                Metrics current;
                QElapsedTimer timer;
                timer.start();
                sortPatients(copy, field, algorithm, &current);
                timings.push_back(timer.nsecsElapsed() / 1000000.0);
                metrics = current;
            }
            insertionSort(timings, [](double a, double b) { return a < b; });
            rows.push_back(
                {order, algorithm, static_cast<int>(base.size()), timings[repeats / 2], metrics});
        }
    }
    return rows;
}
} // namespace clinic::algorithms
```

5.34 SOURCE OR RESOURCE FILE

src/controller/ClinicManager.cpp

```
#include "ClinicManager.h"
#include "model/Appointable.h"
#include "model/Errors.h"
#include <limits>
namespace clinic::controller {
using namespace model;
namespace {
QString generateId(const QString &prefix, int &next,
                   const std::function<bool(const QString &)> &exists) {
    while (next < std::numeric_limits<int>::max()) {
        const QString value = prefix + QString("%1").arg(next++, 4, 10, QChar('0'));
        if (!exists(value))
            return value;
    }
    throw ValidationError("ID space exhausted.");
}
const PatientRecord &requirePatient(const ClinicDatabase &db, const QString &id,
                                    bool allowArchived = false) {
    const auto *patient = db.patient(id);
    if (!patient)
        throw ValidationError("Select an existing patient.");
    if (!allowArchived && db.archivedPatients.contains(id))
        throw ValidationError(
            "Restore this archived patient before changing their record or booking a visit.");
    return *patient;
}
Appointment &requireAppointment(ClinicDatabase &db, const QString &id) {
    auto *appointment = db.appointment(id);
    if (!appointment)
        throw ValidationError("Select an existing appointment.");
    return *appointment;
}
void removeQueued(ClinicDatabase &db, const QString &id) {
    std::queue<QueueEntry> updated;
    while (!db.waiting.empty()) {
        if (db.waiting.front().patientId != id)
            updated.push(db.waiting.front());
        db.waiting.pop();
    }
    db.waiting = std::move(updated);
}
} // namespace
void ClinicManager::requireEdit() const {
    if (!canEdit())
        throw ValidationError("Your current role cannot modify clinic records.");
}
void ClinicManager::audit(const QString &action) {
    database_.audit.push_back({QDateTime::currentDateTimeUtc(), username(), action});
    dirty_ = true;
}
void ClinicManager::transact(const QString &description,
                             const std::function<void(ClinicDatabase &)> &operation) {
    requireEdit();
    ClinicDatabase candidate(database_);
    operation(candidate);
    candidate.validate();
    // Bound memory while retaining the explicitly required stack container.
    if (history_.size() >= 50) {
        std::vector<UndoSnapshot> newest;
        while (!history_.empty()) {
            newest.push_back(std::move(history_.top()));
            history_.pop();
        }
        newest.pop_back();
        for (auto it = newest.rbegin(); it != newest.rend(); ++it)
            history_.push(std::move(*it));
    }
    auto snapshot = database_;
    snapshot.audit.clear(); // Audit is append-only across undo.
```

```
    history_.push({std::move(snapshot), description});
    database_ = std::move(candidate);
    audit(description);
}
QString ClinicManager::suggestedPatientId() const {
    int next = database_.nextPatientId;
    return generateId("P", next,
                      [this](const QString &id) { return database_.patient(id) != nullptr; });
}
void ClinicManager::addPatient(PatientCategory category, PatientDetails details) {
    if (!canViewClinicalNotes()) {
        category = PatientCategory::General;
        details.reason = "Awaiting clinician assessment";
        details.urgency = 1;
        details.previousVisit.clear();
    }
    transact("Register patient " + details.name.trimmed(), [&](ClinicDatabase &db) {
        if (details.id.trimmed().isEmpty())
            details.id = generateId("P", db.nextPatientId,
                                    [&](const QString &id) { return db.patient(id) != nullptr; });
        db.patients.push_back(makePatient(category, details));
    });
}
void ClinicManager::editPatient(const QString &id, PatientCategory category,
                                PatientDetails details) {
    transact("Edit patient " + id, [&](ClinicDatabase &db) {
        const auto &old = requirePatient(db, id);
        if (!canViewClinicalNotes()) {
            category = old.category();
            details.reason = old.details().reason;
            details.urgency = old.details().urgency;
            details.previousVisit = old.details().previousVisit;
        }
        details.id = id;
        for (auto &patient : db.patients)
            if (patient->id() == id) {
                patient = makePatient(category, details);
                break;
            }
    });
}
void ClinicManager::deletePatient(const QString &id) {
    if (!isAdmin())
        throw ValidationError("Only an administrator can archive a patient.");
    transact("Archive patient " + id, [&](ClinicDatabase &db) {
        requirePatient(db, id);
        for (const auto &a : db.appointments)
            if (a.patientId == id && a.status == AppointmentStatus::InConsultation)
                throw ValidationError(
                    "Finish the active consultation before archiving this patient.");
        db.archivedPatients.insert(id);
        for (auto &a : db.appointments)
            if (a.patientId == id && isPending(a.status)) {
                a.status = AppointmentStatus::Cancelled;
                a.changeReason = "Patient archived";
            }
        removeQueued(db, id);
        for (auto &e : db.frequent)
            if (e.patientId == id)
                e = {};
    });
}
void ClinicManager::restorePatient(const QString &id) {
    if (!isAdmin())
        throw ValidationError("Only an administrator can restore archived patients.");
    transact("Restore patient " + id, [&](ClinicDatabase &db) {
        requirePatient(db, id, true);
        if (!db.archivedPatients.remove(id))
            throw ValidationError("This patient is not archived.");
    });
}
QString ClinicManager::schedule(const QString &patientId, const QString &service,
                                const QDateTime &start, int minutes, const QString &clinician,
                                const QString &room) {
    QString result;
```

```
    transact("Schedule visit for " + patientId, [&](ClinicDatabase &db) {
        const auto &patient = requirePatient(db, patientId);
        if (!start.isValid() || start < QDateTime::currentDateTime())
            throw ValidationError("New appointments must be scheduled in the future.");
        Appointment a;
        a.id = generateId("A", db.nextAppointmentId,
                          [&](const QString &id) { return db.appointment(id) != nullptr; });
        a.patientId = patientId;
        a.service = service.trimmed();
        a.start = start;
        a.durationMinutes = minutes;
        a.clinician = clinician;
        a.room = room;
        const auto *policy = dynamic_cast<const Appointable *>(&patient);
        if (!policy)
            throw ValidationError("This patient cannot be scheduled.");
        policy->validateScheduling(a);
        result = a.id;
        db.appointments.push_back(a);
    });
    return result;
}
void ClinicManager::reschedule(const QString &id, const QDateTime &start, int minutes) {
    transact("Reschedule appointment " + id, [&](ClinicDatabase &db) {
        auto &a = requireAppointment(db, id);
        requirePatient(db, a.patientId);
        if (a.status != AppointmentStatus::Scheduled)
            throw ValidationError(
                "Only scheduled, not-yet-checked-in appointments can be rescheduled.");
        if (!start.isValid() || start < QDateTime::currentDateTime())
            throw ValidationError("Choose a future appointment time.");
        a.start = start;
        a.durationMinutes = minutes;
    });
}
void ClinicManager::checkIn(const QString &id) {
    transact("Check in appointment " + id, [&](ClinicDatabase &db) {
        auto &a = requireAppointment(db, id);
        requirePatient(db, a.patientId);
        if (a.status != AppointmentStatus::Scheduled)
            throw ValidationError("Only a scheduled appointment can be checked in.");
        if (a.start.toLocalTime().date() != QDate::currentDate())
            throw ValidationError("Check in is available on the appointment date.");
        a.status = AppointmentStatus::CheckedIn;
        db.waiting.push({a.patientId, QDateTime::currentDateTime(), a.id});
    });
}
void ClinicManager::cancel(const QString &id, const QString &reason) {
    if (reason.trimmed().isEmpty() || reason.size() > 1000)
        throw ValidationError("Enter a cancellation reason (1-1000 characters).");
    transact("Cancel appointment " + id, [&](ClinicDatabase &db) {
        auto &a = requireAppointment(db, id);
        if (a.status != AppointmentStatus::Scheduled && a.status != AppointmentStatus::CheckedIn)
            throw ValidationError("Only scheduled or checked-in visits can be cancelled.");
        if (a.status == AppointmentStatus::Scheduled) {
            auto *policy = dynamic_cast<const Appointable *>(&requirePatient(db, a.patientId));
            if (policy)
                policy->validateCancellation(a);
        }
        if (a.status == AppointmentStatus::CheckedIn)
            removeQueued(db, a.patientId);
        a.status = AppointmentStatus::Cancelled;
        a.changeReason = reason.trimmed();
    });
}
void ClinicManager::complete(const QString &id, const QString &notes) {
    if (!canViewClinicalNotes())
        throw ValidationError("Only clinicians and administrators can complete consultations.");
    transact("Complete consultation " + id, [&](ClinicDatabase &db) {
        auto &a = requireAppointment(db, id);
        if (a.status != AppointmentStatus::InConsultation)
            throw ValidationError(
                "Call this patient from the queue to start consultation before completing it.");
        if (a.consultationStarted.isValid() && a.consultationStarted > QDateTime::currentDateTime()
  > )
```

```
            throw ValidationError("Consultation start time is in the future.");
        if (notes.trimmed().isEmpty() || notes.size() > 10000)
            throw ValidationError("Enter consultation notes (1-10000 characters).");
        a.consultationNotes = notes.trimmed();
        a.completedAt = QDateTime::currentDateTime();
        a.status = AppointmentStatus::Completed;
        removeQueued(db, a.patientId);
    });
}
void ClinicManager::enqueue(const QString &patientId) {
    transact("Add patient " + patientId + " to waiting queue", [&](ClinicDatabase &db) {
        requirePatient(db, patientId);
        for (const auto &a : db.appointments)
            if (a.patientId == patientId && a.status == AppointmentStatus::InConsultation)
                throw ValidationError("This patient is already in consultation.");
        Appointment a;
        a.id = generateId("A", db.nextAppointmentId,
                          [&](const QString &v) { return db.appointment(v); });
        a.patientId = patientId;
        a.service = "Walk-in consultation";
        a.start = QDateTime::currentDateTime();
        a.status = AppointmentStatus::CheckedIn;
        a.walkIn = true;
        db.appointments.push_back(a);
        db.waiting.push({patientId, a.start, a.id});
    });
}
QString ClinicManager::callNext() {
    if (!canViewClinicalNotes())
        throw ValidationError("Only clinicians and administrators can call the next patient.");
    QString summary;
    transact("Call next patient", [&](ClinicDatabase &db) {
        if (db.waiting.empty())
            throw ValidationError("The waiting queue is empty.");
        auto entry = db.waiting.front();
        const auto &patient = requirePatient(db, entry.patientId);
        if (entry.appointmentId.isEmpty()) {
            Appointment a;
            a.id = generateId("A", db.nextAppointmentId,
                              [&](const QString &v) { return db.appointment(v); });
            a.patientId = entry.patientId;
            a.start = QDateTime::currentDateTime();
            a.service = "Walk-in consultation";
            a.walkIn = true;
            db.appointments.push_back(a);
            entry.appointmentId = a.id;
        }
        auto &visit = requireAppointment(db, entry.appointmentId);
        visit.status = AppointmentStatus::InConsultation;
        visit.consultationStarted = QDateTime::currentDateTime();
        summary = patient.details().name + " (" + patient.id() + ")\n" + processPatient(patient);
        db.waiting.pop();
    });
    return summary;
}
void ClinicManager::removeFromQueue(const QString &patientId) {
    transact("Remove " + patientId + " from waiting queue", [&](ClinicDatabase &db) {
        auto queue = db.waiting;
        bool found = false;
        while (!queue.empty()) {
            const auto entry = queue.front();
            if (entry.patientId == patientId) {
                found = true;
                if (!entry.appointmentId.isEmpty())
                    requireAppointment(db, entry.appointmentId).status =
                        AppointmentStatus::Scheduled;
            }
            queue.pop();
        }
        if (!found)
            throw ValidationError("Select a waiting patient.");
        removeQueued(db, patientId);
    });
}
void ClinicManager::undo() {
```

```
    requireEdit();
    if (!canUndo())
        throw ValidationError("There is no operation to undo.");
    const auto description = history_.top().description;
    auto auditTrail = std::move(database_.audit);
    database_ = std::move(history_.top().state);
    history_.pop();
    database_.audit = std::move(auditTrail);
    audit("Undo: " + description);
}
void ClinicManager::replace(ClinicDatabase database, const QString &username,
                            const QString &password) {
    database.validate();
    QString accepted;
    auto role = security::Role::Viewer;
    if (!database.users.empty()) {
        for (const auto &account : database.users)
            if (account.username == username.trimmed().toLower() &&
                security::verifyPassword(account, password)) {
                accepted = account.username;
                role = account.role;
                break;
            }
        if (accepted.isEmpty())
            throw ValidationError("Invalid username or password.");
    }
    database_ = std::move(database);
    username_ = accepted;
    role_ = role;
    history_ = {};
    dirty_ = false;
    failedLogins_ = 0;
}
void ClinicManager::newDatabase() {
    replace(ClinicDatabase{});
}
void ClinicManager::configureAdministrator(const QString &username, const QString &password) {
    if (!demoMode())
        throw ValidationError("Access control is already configured.");
    auto account = security::createAccount(username, password, security::Role::Administrator);
    database_.users.push_back(account);
    username_ = account.username;
    role_ = account.role;
    history_ = {};
    audit("Enable staff access control");
}
void ClinicManager::addUser(const QString &username, const QString &password, security::Role role)
  > {
    if (!isAdmin())
        throw ValidationError("Administrator access is required.");
    if (demoMode())
        throw ValidationError("Create the initial administrator first.");
    auto account = security::createAccount(username, password, role);
    for (const auto &user : database_.users)
        if (user.username == account.username)
            throw ValidationError("Username already exists.");
    database_.users.push_back(account);
    history_ = {};
    audit("Add staff account " + account.username + " (" + security::roleName(role) + ")");
}
void ClinicManager::setUserActive(const QString &username, bool active) {
    if (!isAdmin())
        throw ValidationError("Administrator access is required.");
    if (username == username_)
        throw ValidationError("You cannot disable your current account.");
    for (auto &user : database_.users)
        if (user.username == username) {
            user.active = active;
            history_ = {};
            audit(QString(active ? "Enable staff account " : "Disable staff account ") + username);
            return;
        }
    throw ValidationError("Staff account not found.");
}
void ClinicManager::changePassword(const QString &current, const QString &replacement) {
```

```
    if (!authenticated() || demoMode())
        throw ValidationError("Sign in to a staff account first.");
    for (auto &user : database_.users)
        if (user.username == username_) {
            if (!security::verifyPassword(user, current))
                throw ValidationError("Current password is incorrect.");
            user = security::createAccount(user.username, replacement, user.role);
            history_ = {};
            audit("Change own password");
            return;
        }
}
void ClinicManager::login(const QString &name, const QString &password) {
    if (lockedUntil_ > QDateTime::currentDateTime())
        throw ValidationError("Too many sign-in attempts. Wait 30 seconds.");
    for (const auto &account : database_.users)
        if (account.username == name.trimmed().toLower() &&
            security::verifyPassword(account, password)) {
            username_ = account.username;
            role_ = account.role;
            failedLogins_ = 0;
            history_ = {};
            audit("Staff sign in");
            return;
        }
    if (++failedLogins_ >= 5) {
        lockedUntil_ = QDateTime::currentDateTime().addSecs(30);
        failedLogins_ = 0;
    }
    audit("Failed sign-in attempt");
    throw ValidationError("Invalid username or password.");
}
void ClinicManager::lock() {
    if (demoMode())
        return;
    audit("Lock workspace");
    username_.clear();
    role_ = security::Role::Viewer;
    history_ = {};
}

void ClinicManager::requireReportExport() const {
    if (!canExportReports())
        throw ValidationError("Only clinicians and administrators may export or print reports.");
}
void ClinicManager::recordExport(const QString &what) {
    requireReportExport();
    audit(what);
}
QStringList ClinicManager::possibleDuplicates(const PatientDetails &d,
                                              const QString &exclude) const {
    QStringList matches;
    for (const auto &p : database_.patients) {
        if (p->id() == exclude)
            continue;
        const auto &x = p->details();
        if ((x.name.simplified().compare(d.name.simplified(), Qt::CaseInsensitive) == 0 &&
             x.age == d.age) ||
            x.contact.trimmed().compare(d.contact.trimmed(), Qt::CaseInsensitive) == 0)
            matches << x.id + " / " + x.name +
                           (database_.archivedPatients.contains(x.id) ? " (archived)" : "");
    }
    return matches;
}
void ClinicManager::noShow(const QString &id, const QString &reason) {
    transact("Mark no-show " + id, [&](ClinicDatabase &db) {
        auto &a = requireAppointment(db, id);
        if (a.status != AppointmentStatus::Scheduled || a.end() > QDateTime::currentDateTime())
            throw ValidationError(
                "Only scheduled visits whose end time has passed can be marked no-show.");
        if (reason.trimmed().isEmpty() || reason.size() > 1000)
            throw ValidationError("Enter a no-show reason (1-1000 characters).");
        a.status = AppointmentStatus::NoShow;
        a.changeReason = reason.trimmed();
    });
```

```
}
void ClinicManager::correctNotes(const QString &id, const QString &notes, const QString &reason) {
    if (!canViewClinicalNotes())
        throw ValidationError("Clinical access required.");
    transact("Correct consultation notes " + id, [&](ClinicDatabase &db) {
        auto &a = requireAppointment(db, id);
        if (a.status != AppointmentStatus::Completed)
            throw ValidationError("Select a completed consultation.");
        if (notes.trimmed().isEmpty() || notes.size() > 10000 || reason.trimmed().isEmpty() ||
            reason.size() > 1000)
            throw ValidationError(
                "Enter notes (1-10000 characters) and a correction reason (1-1000 characters).");
        a.corrections << QDateTime::currentDateTimeUtc().toString(Qt::ISODate) + " / " +
                             username() + " / " + reason.trimmed() +
                             "\nPrevious notes: " + a.consultationNotes;
        a.consultationNotes = notes.trimmed();
        a.changeReason = reason.trimmed();
    });
}
void ClinicManager::assignResources(const QString &id, const QString &clinician,
                                    const QString &room) {
    transact("Allocate resources " + id, [&](ClinicDatabase &db) {
        auto &a = requireAppointment(db, id);
        if (!isPending(a.status))
            throw ValidationError("Only active visits can be allocated.");
        a.clinician = clinician;
        a.room = room;
    });
}
void ClinicManager::addResource(const QString &name, bool room) {
    if (!isAdmin())
        throw ValidationError("Only administrators can manage resources.");
    transact("Add " + QString(room ? "room " : "clinician ") + name,
             [&](ClinicDatabase &db) { (room ? db.rooms : db.clinicians).append(name.trimmed()); })
  > ;
}

} // namespace clinic::controller
```

5.35 SOURCE OR RESOURCE FILE

src/controller/ReportService.cpp

```
#include "ReportService.h"
#include "algorithms/Algorithms.h"
#include "model/Errors.h"
namespace clinic::controller {
using namespace model;
Report generateReport(const ClinicDatabase &db, ReportType type, QDate from, QDate to) {
    if (from.isValid() && !to.isValid())
        to = from;
    if (from.isValid() && to < from)
        throw ValidationError("The end date cannot be earlier than the start date.");
    auto included = [&](const Appointment &a) {
        const auto d = a.start.toLocalTime().date();
        return !from.isValid() || (d >= from && d <= to);
    };
    Report report;
    const auto range = from.isValid()
                           ? from.toString("dd MMM yyyy") + " - " + to.toString("dd MMM yyyy")
                           : "All dates";
    if (type == ReportType::Attendance) {
        report.title = "Most frequently attended patients";
        report.headers = {"Patient ID", "Patient", "Completed visits", "Record"};
        struct Count {
            QString id, name, state;
            int count;
        };
        std::vector<Count> rows;
        int visits = 0;
        for (const auto &p : db.patients) {
            int count = 0;
            for (const auto &a : db.appointments)
                if (a.patientId == p->id() && a.status == AppointmentStatus::Completed &&
                    included(a))
                    ++count;
            if (count)
                rows.push_back({p->id(), p->details().name,
                                db.archivedPatients.contains(p->id()) ? "Archived" : "Active",
                                count});
            visits += count;
        }
        algorithms::mergeSort(rows,
                              [](const Count &a, const Count &b) { return a.count > b.count; });
        for (const auto &r : rows)
            report.rows.append({r.id, r.name, QString::number(r.count), r.state});
        report.summary = QString("%1 completed visits across %2 patients | %3")
                             .arg(visits)
                             .arg(rows.size())
                             .arg(range);
    } else if (type == ReportType::Pending) {
        report.title = "Patients with pending appointments";
        report.headers = {"Patient ID", "Patient",       "Appointment",
                          "Service",    "Scheduled for", "Status"};
        for (const auto &a : db.appointments)
            if (isPending(a.status) && included(a))
                report.rows.append({a.patientId, db.patient(a.patientId)->details().name, a.id,
                                    a.service, a.start.toLocalTime().toString("yyyy-MM-dd HH:mm"),
                                    statusName(a.status)});
        report.summary = QString("%1 pending visits | %2").arg(report.rows.size()).arg(range);
    } else if (type == ReportType::Categories) {
        report.title = "Patient category distribution";
        report.headers = {"Category", "Active patients", "Archived patients", "Total"};
        for (auto category :
             {PatientCategory::General, PatientCategory::Emergency, PatientCategory::FollowUp}) {
            int active = 0, archived = 0;
            for (const auto &p : db.patients)
                if (p->category() == category) {
                    if (db.archivedPatients.contains(p->id()))
                        ++archived;
                    else
                        ++active;
```

```
                }
            report.rows.append({categoryName(category), QString::number(active),
                                QString::number(archived), QString::number(active + archived)});
        }
        report.summary = QString("%1 registered records | Category counts are current, independent
 > "
                                 "of the date range.")
                             .arg(db.patients.size());
    } else {
        report.title = "Appointment status summary";
        report.headers = {"Status", "Appointments"};
        int total = 0, scheduled = 0, checked = 0, consulting = 0, completed = 0, cancelled = 0,
            noShow = 0;
        for (const auto &a : db.appointments)
            if (included(a)) {
                ++total;
                switch (a.status) {
                case AppointmentStatus::Scheduled:
                    ++scheduled;
                    break;
                case AppointmentStatus::CheckedIn:
                    ++checked;
                    break;
                case AppointmentStatus::InConsultation:
                    ++consulting;
                    break;
                case AppointmentStatus::Completed:
                    ++completed;
                    break;
                case AppointmentStatus::NoShow:
                    ++noShow;
                    break;
                case AppointmentStatus::Cancelled:
                    ++cancelled;
                    break;
                }
            }
        report.rows = {{"Total", QString::number(total)},
                       {"Completed", QString::number(completed)},
                       {"Scheduled", QString::number(scheduled)},
                       {"Checked in", QString::number(checked)},
                       {"In consultation", QString::number(consulting)},
                       {"Cancelled", QString::number(cancelled)},
                       {"No-show", QString::number(noShow)}};
        report.summary = QString("%1 visits | %2").arg(total).arg(range);
    }
    return report;
}
QString reportCsv(const Report &report) {
    auto encode = [](QString value) {
        const auto trimmed = value.trimmed();
        if (!trimmed.isEmpty() && QString("=+-@").contains(trimmed.front()))
            value.prepend("'");
        value.replace("\"", "\"\"");
        return "\"" + value + "\"";
    };
    QString text;
    auto append = [&](const QStringList &row) {
        QStringList cells;
        for (const auto &c : row)
            cells.append(encode(c));
        text += cells.join(",") + "\r\n";
    };
    append(report.headers);
    for (const auto &row : report.rows)
        append(row);
    append({"Summary", report.summary});
    return text;
}
QString reportHtml(const Report &report) {
    QString html = "<h1>" + report.title.toHtmlEscaped() + "</h1><p>" +
                   report.summary.toHtmlEscaped() +
                   "</p><table border='1' cellspacing='0' cellpadding='6'><tr>";
    for (const auto &h : report.headers)
        html += "<th>" + h.toHtmlEscaped() + "</th>";
```

```
144     html += "</tr>";
145     for (const auto &row : report.rows) {
146         html += "<tr>";
147         for (const auto &c : row)
148             html += "<td>" + c.toHtmlEscaped() + "</td>";
149         html += "</tr>";
150     }
151     return html + "</table><p>SCAPMS - generated " +
152            QDateTime::currentDateTime().toString("yyyy-MM-dd HH:mm") + "</p>";
153 }
154 } // namespace clinic::controller
```

5.36 SOURCE OR RESOURCE FILE

src/gui/MainWindow.cpp

```
#include "MainWindow.h"
#include "PatientDialog.h"
#include "security/PortableBackup.h"
#include "panels/CalendarPanel.h"
#include <QSettings>
#include <QJsonDocument>
#include <QSaveFile>
#include "Theme.h"
#include "model/Errors.h"
#include "panels/AccessPanel.h"
#include "panels/AppointmentsPanel.h"
#include "panels/DashboardPanel.h"
#include "panels/PatientsPanel.h"
#include "panels/QueuePanel.h"
#include "panels/ReportsPanel.h"
#include "panels/SearchPanel.h"
#include "persistence/RecoveryService.h"
#include <QtWidgets>
namespace clinic::gui {
using persistence::FileHandler;
using persistence::RecoveryService;
namespace {
bool credentials(QWidget *parent, const QString &title, QString &username, QString &password,
                 bool create = false) {
    QDialog dialog(parent);
    dialog.setObjectName("credentialsDialog");
    dialog.setWindowTitle(title);
    dialog.setMinimumWidth(430);
    auto *form = new QFormLayout(&dialog);
    auto *info =
        new QLabel(create ? "Choose your administrator credentials. There is no default password."
                          : "Sign in with a staff account for this workspace.");
    info->setWordWrap(true);
    form->addRow(info);
    auto *name = new QLineEdit;
    name->setObjectName("loginUsername");
    name->setText(username);
    auto *secret = new QLineEdit;
    secret->setObjectName("loginPassword");
    secret->setEchoMode(QLineEdit::Password);
    form->addRow("Username", name);
    form->addRow(create ? "Password (12+ characters)" : "Password", secret);
    auto *buttons = new QDialogButtonBox(QDialogButtonBox::Ok | QDialogButtonBox::Cancel);
    form->addRow(buttons);
    QObject::connect(buttons, &QDialogButtonBox::accepted, &dialog, &QDialog::accept);
    QObject::connect(buttons, &QDialogButtonBox::rejected, &dialog, &QDialog::reject);
    if (dialog.exec() != QDialog::Accepted)
        return false;
    username = name->text();
    password = secret->text();
    return true;
}
} // namespace
MainWindow::MainWindow(QWidget *parent) : QMainWindow(parent) {
    setObjectName("mainWindow");
    setMinimumSize(1180, 740);
    resize(1380, 880);
    setStyleSheet(theme);
    auto *shell = new QWidget;
    auto *horizontal = new QHBoxLayout(shell);
    horizontal->setContentsMargins(0, 0, 0, 0);
    horizontal->setSpacing(0);
    auto *sidebar = new QFrame;
    sidebar->setObjectName("sidebar");
    sidebar->setFixedWidth(218);
    auto *side = new QVBoxLayout(sidebar);
    side->setContentsMargins(18, 24, 18, 20);
    auto *brand = new QLabel("MIVA / CLINIC");
    brand->setObjectName("brand");
```

```
    side->addWidget(brand);
    auto *caption = new QLabel("PATIENT CARE DESK");
    caption->setObjectName("brandCaption");
    side->addWidget(caption);
    navigation_ = new QListWidget;
    navigation_->setObjectName("navigation");
    navigation_->addItems({"Overview", "Patient records", "Appointments", "Waiting queue",
                           "Search & sort", "Reports", "Staff & activity", "Calendar & rooms"});
    side->addWidget(navigation_);
    auto *foot = new QLabel("SCAPMS 3.0 / COS 202");
    foot->setObjectName("brandCaption");
    side->addWidget(foot);
    horizontal->addWidget(sidebar);
    pages_ = new QStackedWidget;
    pages_->setObjectName("pages");
    MutationHandler mutation = [this](const std::function<void()> &action, const QString &success)
 > {
        run(action, success);
    };
    panels_ = {new DashboardPanel(manager_, mutation),
               new PatientsPanel(manager_, mutation),
               new AppointmentsPanel(manager_, mutation),
               new QueuePanel(manager_, mutation),
               new SearchPanel(manager_, mutation),
               new ReportsPanel(manager_, mutation),
               new AccessPanel(manager_, mutation, [this] { protectWorkspace(); }),
               new CalendarPanel(manager_, mutation)};
    for (auto *panel : panels_)
        pages_->addWidget(panel);
    horizontal->addWidget(pages_, 1);
    setCentralWidget(shell);
    connect(navigation_, &QListWidget::currentRowChanged, pages_, &QStackedWidget::setCurrentIndex)
 > ;
    navigation_->setCurrentRow(0);
    connect(navigation_, &QListWidget::currentRowChanged, this, [this](int i) {
        if (i >= 0 && i < static_cast<int>(panels_.size()))
            panels_[i]->refresh();
    });
    auto *file = menuBar()->addMenu("&File");
    auto *fresh = file->addAction("&New workspace");
    fresh->setObjectName("newWorkspaceAction");
    fresh->setShortcut(QKeySequence::New);
    connect(fresh, &QAction::triggered, this, [this] {
        if (confirmDiscard()) {
            prepareChange();
            manager_.newDatabase();
            currentPath_.clear();
            workspaceGuard_.reset();
            refresh();
        }
    });
    auto *load = file->addAction("&Load workspace...");
    load->setShortcut(QKeySequence::Open);
    connect(load, &QAction::triggered, this, &MainWindow::openFile);
    auto *saveAction = file->addAction("&Save");
    saveAction->setShortcut(QKeySequence::Save);
    connect(saveAction, &QAction::triggered, this, [this] { save(); });
    auto *saveAs = file->addAction("Save &as...");
    saveAs->setShortcut(QKeySequence::SaveAs);
    connect(saveAs, &QAction::triggered, this, [this] { save(true); });
    auto *exportBackup = file->addAction("Export portable encrypted backup...");
    exportBackup->setObjectName("exportPortableBackup");
    connect(exportBackup, &QAction::triggered, this, [this] { portableBackup(false); });
    auto *importBackup = file->addAction("Import portable encrypted backup...");
    connect(importBackup, &QAction::triggered, this, [this] { portableBackup(true); });
    auto *home = file->addAction("Welcome / recent workspaces...");
    connect(home, &QAction::triggered, this, [this] {
        if (confirmDiscard())
            welcome();
    });
    auto *view = menuBar()->addMenu("&View");
    for (int size : {10, 12, 14}) {
        auto *action = view->addAction(QString("Text size %1 pt").arg(size));
        connect(action, &QAction::triggered, this, [this, size] { setTextSize(size); });
    }
```

```
    for (int i = 0; i < 8; ++i) {
        auto *shortcut = new QShortcut(QKeySequence(QString("Alt+%1").arg(i + 1)), this);
        connect(shortcut, &QShortcut::activated, this,
                [this, i] { navigation_->setCurrentRow(i); });
    }
    auto *recovery = file->addAction("Recover autosave / backup...");
    recovery->setObjectName("recoverAction");
    connect(recovery, &QAction::triggered, this, &MainWindow::recover);
    file->addSeparator();
    auto *quit = file->addAction("E&xit");
    quit->setShortcut(QKeySequence("Ctrl+Q"));
    connect(quit, &QAction::triggered, this, &QWidget::close);
    auto *edit = menuBar()->addMenu("&Edit");
    undoAction_ = edit->addAction("&Undo");
    undoAction_->setShortcut(QKeySequence::Undo);
    connect(undoAction_, &QAction::triggered, this, [this] {
        run(
            [this] {
                manager_.undo();
                dynamic_cast<QueuePanel *>(panels_[3])->resetMessage();
            },
            "Operation undone.");
    });
    auto *find = edit->addAction("&Find patients");
    find->setShortcut(QKeySequence::Find);
    connect(find, &QAction::triggered, this, [this] {
        navigation_->setCurrentRow(4);
        findChild<QLineEdit *>("searchText")->setFocus();
    });
    toolbar_ = addToolBar("Workspace");
    toolbar_->setMovable(false);
    auto *open = actionButton("Load data", "loadButton");
    toolbar_->addWidget(open);
    connect(open, &QPushButton::clicked, this, &MainWindow::openFile);
    auto *saveButton = actionButton("Save workspace", "saveButton", true);
    saveButton->setToolTip("Save with a protected backup of the previous version (Ctrl+S)");
    toolbar_->addWidget(saveButton);
    connect(saveButton, &QPushButton::clicked, this, [this] { save(); });
    undoButton_ = actionButton("Undo", "undoButton");
    toolbar_->addWidget(undoButton_);
    connect(undoButton_, &QPushButton::clicked, undoAction_, &QAction::trigger);
    auto *spacer = new QWidget;
    spacer->setSizePolicy(QSizePolicy::Expanding, QSizePolicy::Preferred);
    toolbar_->addWidget(spacer);
    fileLabel_ = new QLabel;
    toolbar_->addWidget(fileLabel_);
    sessionLabel_ = new QLabel;
    statusBar()->addPermanentWidget(sessionLabel_);
    lockButton_ = actionButton("Lock / switch user", "lockButton");
    toolbar_->addWidget(lockButton_);
    connect(lockButton_, &QPushButton::clicked, this, &MainWindow::lockWorkspace);
    auto *timer = new QTimer(this);
    timer->setInterval(30000);
    connect(timer, &QTimer::timeout, this, [this] {
        if (locking_)
            return;
        if (!manager_.demoMode() && manager_.authenticated() && idle_.elapsed() >= 300000) {
            lockWorkspace();
            return;
        }
        panels_[0]->refresh();
        panels_[3]->refresh();
        autosaveNow();
    });
    timer->start();
    qApp->installEventFilter(this);
    idle_.start();
    refresh();
}
MainWindow::~MainWindow() {
    qApp->removeEventFilter(this);
}
bool MainWindow::eventFilter(QObject *watched, QEvent *event) {
    if (event->type() == QEvent::MouseButtonPress || event->type() == QEvent::KeyPress ||
        event->type() == QEvent::Wheel)
```

```
        idle_.restart();
    return QMainWindow::eventFilter(watched, event);
}
void MainWindow::prepareChange() {
    for (auto *panel : panels_)
        panel->beforeChange();
}
void MainWindow::run(const std::function<void()> &operation, const QString &success) {
    prepareChange();
    try {
        operation();
        refresh();
        statusBar()->showMessage(success, 6000);
    } catch (const std::exception &e) {
        refresh();
        QMessageBox::warning(this, "Unable to complete operation", QString::fromUtf8(e.what()));
    }
}
void MainWindow::refresh() {
    for (auto *panel : panels_)
        panel->refresh();
    updateTitle();
}
void MainWindow::updateTitle() {
    const auto name =
        currentPath_.isEmpty() ? "Untitled workspace" : QFileInfo(currentPath_).fileName();
    setWindowTitle(QString("SCAPMS 3.0 - %1%2").arg(name, manager_.dirty() ? " *" : ""));
    fileLabel_->setText(name + (manager_.dirty() ? " / Unsaved" : " / Saved"));
    sessionLabel_->setText(manager_.demoMode()
                               ? "DEMO / fictional data only"
                               : manager_.username() + " / " + security::roleName(manager_.role()))
  > ;
    undoAction_->setEnabled(manager_.canUndo());
    undoButton_->setEnabled(manager_.canUndo());
    undoAction_->setText(manager_.canUndo() ? "Undo: " + manager_.undoDescription() : "Undo");
    undoButton_->setText("Undo");
    undoButton_->setToolTip(manager_.canUndo() ? "Undo: " + manager_.undoDescription()
                                               : "No operation to undo");
    lockButton_->setEnabled(!manager_.demoMode());
    findChild<QPushButton *>("saveButton")->setEnabled(manager_.canEdit());
    findChild<QAction *>("exportPortableBackup")->setEnabled(manager_.isAdmin());
}
bool MainWindow::save(bool choosePath) {
    if (!manager_.canEdit()) {
        QMessageBox::warning(this, "Save", "Your role cannot save or copy a workspace.");
        return false;
    }
    auto target = currentPath_;
    if (!manager_.demoMode() && QFileInfo(target).suffix() == "json")
        choosePath = true;
    if (target.isEmpty() || choosePath) {
        const auto filter = manager_.demoMode()
                                ? "Protected workspace (*.clinic);;Demo JSON (*.json)"
                                : "Protected workspace (*.clinic)";
        target = QFileDialog::getSaveFileName(
            this, "Save workspace",
            manager_.demoMode() ? "clinic-data.clinic" : "clinic-workspace.clinic", filter);
    }
    if (target.isEmpty())
        return false;
    if (QFileInfo(target).suffix().isEmpty())
        target += ".clinic";
    try {
        std::unique_ptr<persistence::WorkspaceGuard> nextGuard;
        auto *guard = workspaceGuard_.get();
        const auto resolved = QFileInfo(target).exists() ? QFileInfo(target).canonicalFilePath()
                                                         : QFileInfo(target).absoluteFilePath();
        if (!guard || guard->path() != resolved) {
            nextGuard = std::make_unique<persistence::WorkspaceGuard>(target);
            guard = nextGuard.get();
        }
        guard->verifyUnchanged();
        const auto oldRecovery = RecoveryService::recoveryPath(currentPath_);
        RecoveryService::saveWithBackup(manager_.database(), target);
        if (oldRecovery != RecoveryService::recoveryPath(target))
```

```
            QFile::remove(oldRecovery);
        guard->checkpoint();
        currentPath_ = guard->path();
        if (nextGuard)
            workspaceGuard_ = std::move(nextGuard);
        rememberWorkspace();
        manager_.markSaved();
        updateTitle();
        statusBar()->showMessage(
            "Saved. Up to five previous versions are kept as protected backups.", 6000);
        return true;
    } catch (const std::exception &e) {
        QMessageBox::warning(this, "Save failed", e.what());
        return false;
    }
}
bool MainWindow::confirmDiscard() {
    if (!manager_.dirty())
        return true;
    if (!manager_.canEdit()) {
        autosaveNow();
        return true;
    }
    const auto choice = QMessageBox::question(
        this, "Unsaved changes", "Save changes to the current workspace?",
        QMessageBox::Save | QMessageBox::Discard | QMessageBox::Cancel, QMessageBox::Save);
    if (choice == QMessageBox::Save)
        return save();
    if (choice == QMessageBox::Discard) {
        try {
            RecoveryService::discardRecovery(currentPath_);
        } catch (const std::exception &e) {
            QMessageBox::warning(this, "Recovery copy", e.what());
            return false;
        }
        return true;
    }
    return false;
}
void MainWindow::loadPath(const QString &path, const QString &username, const QString &password) {
    std::unique_ptr<persistence::WorkspaceGuard> nextGuard;
    auto *guard = workspaceGuard_.get();
    if (!guard || guard->path() != QFileInfo(path).canonicalFilePath()) {
        nextGuard = std::make_unique<persistence::WorkspaceGuard>(path);
        guard = nextGuard.get();
    } else {
        // A reload intentionally accepts the current on-disk version.
        guard->checkpoint();
    }
    controller::ClinicManager next;
    next.replace(FileHandler::load(path), username, password);
    guard->verifyUnchanged();
    prepareChange();
    manager_ = std::move(next);
    if (nextGuard)
        workspaceGuard_ = std::move(nextGuard);
    currentPath_ = guard->path();
    rememberWorkspace();
    dynamic_cast<QueuePanel *>(panels_[3])->resetMessage();
    refresh();
    idle_.restart();
}
bool MainWindow::openWorkspace(const QString &path) {
    try {
        const auto candidate = FileHandler::load(path);
        if (candidate.users.empty()) {
            loadPath(path);
            return true;
        }
        QString name, password;
        for (int attempt = 0; attempt < 5; ++attempt) {
            if (!credentials(this, "Open protected workspace", name, password))
                return false;
            try {
                loadPath(path, name, password);
```

```
                return true;
            } catch (const model::ValidationError &e) {
                QMessageBox::warning(this, "Sign in", e.what());
            }
        }
        QMessageBox::warning(this, "Sign in",
                             "Too many attempts. Opening this workspace has been cancelled.");
    } catch (const std::exception &e) {
        QMessageBox::warning(this, "Load failed", e.what());
    }
    return false;
}
void MainWindow::openFile() {
    if (!confirmDiscard())
        return;
    const auto file =
        QFileDialog::getOpenFileName(this, "Load workspace", {}, "Clinic data (*.clinic *.json)");
    if (file.isEmpty())
        return;
    if (openWorkspace(file))
        offerRecovery();
}
void MainWindow::autosaveNow() {
    if (!manager_.dirty())
        return;
    try {
        if (workspaceGuard_)
            workspaceGuard_->verifyUnchanged();
        RecoveryService::autosave(manager_.database(), currentPath_);
        recoveryError_.clear();
        statusBar()->showMessage("Recovery copy updated; use Save to commit the workspace.", 4000);
    } catch (const std::exception &e) {
        recoveryError_ = QString::fromUtf8(e.what());
        statusBar()->showMessage("Recovery unavailable: " + recoveryError_);
    }
}
void MainWindow::protectWorkspace() {
    QString username, password;
    if (!credentials(this, "Protect workspace", username, password, true))
        return;
    auto file =
        QFileDialog::getSaveFileName(this, "Save protected workspace", "clinic-workspace.clinic",
                                     "Protected workspace (*.clinic)");
    if (file.isEmpty())
        return;
    if (!file.endsWith(".clinic", Qt::CaseInsensitive))
        file += ".clinic";
    run(
        [&] {
            controller::ClinicManager candidate;
            candidate.replace(manager_.database());
            candidate.configureAdministrator(username, password);
            auto guard = std::make_unique<persistence::WorkspaceGuard>(file);
            guard->verifyUnchanged();
            RecoveryService::saveWithBackup(candidate.database(), file);
            guard->checkpoint();
            manager_ = std::move(candidate);
            manager_.markSaved();
            currentPath_ = guard->path();
            workspaceGuard_ = std::move(guard);
            rememberWorkspace();
        },
        "Access control enabled. Keep backups with this Windows account.");
}
void MainWindow::lockWorkspace() {
    if (manager_.demoMode() || locking_)
        return;
    locking_ = true;
    manager_.lock();
    autosaveNow();
    centralWidget()->hide();
    toolbar_->hide();
    menuBar()->hide();
    sessionLabel_->setText("Workspace locked");
    QString username, password;
```

```
    bool unlocked = false;
    while (credentials(this, "Unlock workspace", username, password)) {
        try {
            manager_.login(username, password);
            unlocked = true;
            break;
        } catch (const std::exception &e) {
            QMessageBox::warning(this, "Sign in", e.what());
        }
    }
    if (!unlocked) {
        autosaveNow();
        qApp->quit();
        return;
    }
    centralWidget()->show();
    toolbar_->show();
    menuBar()->show();
    idle_.restart();
    locking_ = false;
    refresh();
}
void MainWindow::closeEvent(QCloseEvent *event) {
    if (confirmDiscard())
        event->accept();
    else
        event->ignore();
}
void MainWindow::captureScreenshots(const QString &directory) {
    QDir().mkpath(directory);
    for (const auto &p : manager_.database().patients)
        manager_.access(p->id());
    refresh();
    const QStringList names{"01-dashboard",       "02-patients",      "04-appointments",
                            "05-waiting-queue",   "06-search-sort",   "07-reports",
                            "08-access-activity", "11-calendar-rooms"};
    for (int i = 0; i < 8; ++i) {
        navigation_->setCurrentRow(i);
        if (i == 1 && findChild<QTableView *>("patientsTable")->model()->rowCount())
            findChild<QTableView *>("patientsTable")->selectRow(0);
        if (i == 4) {
            findChild<QComboBox *>("sortAlgorithm")->setCurrentIndex(1);
            findChild<QPushButton *>("sortButton")->click();
        }
        if (i == 5)
            findChild<QComboBox *>("reportType")->setCurrentIndex(3);
        QApplication::processEvents();
        if (!grab().save(QDir(directory).filePath(names[i] + ".png")))
            throw model::FileError("Could not save screenshot.");
    }
    PatientDialog registration(manager_.suggestedPatientId(), nullptr, this);
    registration.findChild<QLineEdit *>("patientName")->setText("Ada Okafor");
    registration.findChild<QLineEdit *>("patientContact")->setText("ada@example.test");
    registration.findChild<QLineEdit *>("patientReason")->setText("Routine health review");
    registration.findChild<QSpinBox *>("patientAge")->setValue(23);
    registration.show();
    QApplication::processEvents();
    registration.grab().save(QDir(directory).filePath("03-registration.png"));
    registration.close();
    navigation_->setCurrentRow(1);
    QTimer::singleShot(100, this, [this, directory] {
        if (auto *dialog = findChild<QDialog *>("historyDialog")) {
            dialog->grab().save(QDir(directory).filePath("09-patient-history.png"));
            dialog->reject();
        }
    });
    findChild<QPushButton *>("patientHistory")->click();
    navigation_->setCurrentRow(4);
    auto *captureTimer = new QTimer(this);
    connect(captureTimer, &QTimer::timeout, this, [this, directory, captureTimer] {
        if (auto *dialog = findChild<QDialog *>("benchmarkDialog")) {
            dialog->grab().save(QDir(directory).filePath("10-algorithm-comparison.png"));
            captureTimer->stop();
            captureTimer->deleteLater();
            dialog->reject();
```

```
517         }
518     });
519     captureTimer->start(100);
520     findChild<QPushButton *>("benchmarkButton")->click();
521     navigation_->setCurrentRow(0);
522     QTimer::singleShot(100, this, [this, directory] {
523         if (auto *dialog = findChild<QDialog *>("welcomeDialog")) {
524             dialog->grab().save(QDir(directory).filePath("12-welcome.png"));
525             dialog->reject();
526         }
527     });
528     welcome();
529 }
530 } // namespace clinic::gui
```

5.37 SOURCE OR RESOURCE FILE

src/gui/MainWindowExtras.cpp

```
#include "MainWindow.h"
#include "Theme.h"
#include "model/Errors.h"
#include "persistence/FileHandler.h"
#include "persistence/RecoveryService.h"
#include "security/PortableBackup.h"
#include <QtWidgets>
#include <QJsonDocument>
#include <QSaveFile>
namespace clinic::gui {
using persistence::FileHandler;
using persistence::RecoveryService;
void MainWindow::rememberWorkspace() {
    if (currentPath_.isEmpty() || currentPath_.endsWith(".recovery.clinic") ||
        currentPath_.contains(".backups/") || currentPath_.contains(".backups\\"))
        return;
    QSettings settings;
    auto recent = settings.value("recentWorkspaces").toStringList();
    recent.removeAll(currentPath_);
    recent.prepend(currentPath_);
    while (recent.size() > 10)
        recent.removeLast();
    settings.setValue("recentWorkspaces", recent);
}
void MainWindow::setTextSize(int size) {
    size = qBound(10, size, 14);
    QString css = QString::fromUtf8(theme);
    css.replace("font-size:10pt", QString("font-size:%1pt").arg(size));
    setStyleSheet(css);
    QSettings().setValue("textSize", size);
}
bool MainWindow::welcome() {
    QDialog dialog(this);
    dialog.setWindowTitle("Welcome to SCAPMS 3.0");
    dialog.setObjectName("welcomeDialog");
    dialog.resize(650, 440);
    auto *layout = new QVBoxLayout(&dialog);
    auto *title = new QLabel("Open your clinic workspace");
    title->setObjectName("pageTitle");
    layout->addWidget(title);
    auto *hint =
        new QLabel("Choose a saved workspace, start a new one, or explore fictional demo records.")
 > ;
    hint->setWordWrap(true);
    layout->addWidget(hint);
    auto *recent = new QListWidget;
    recent->setObjectName("recentWorkspaces");
    for (const auto &p : QSettings().value("recentWorkspaces").toStringList()) {
        if (!QFileInfo::exists(p))
            continue;
        auto *item = new QListWidgetItem(QFileInfo(p).fileName(), recent);
        item->setData(Qt::UserRole, p);
        item->setToolTip(p);
    }
    layout->addWidget(new QLabel(recent->count() ? "Recent workspaces (double-click to open)"
                                                 : "No recent workspaces yet."));
    layout->addWidget(recent);
    auto choose = [&](const QString &p) {
        if (!p.isEmpty() && openWorkspace(p)) {
            offerRecovery();
            dialog.accept();
        }
    };
    connect(recent, &QListWidget::itemDoubleClicked, &dialog,
            [&](QListWidgetItem *item) { choose(item->data(Qt::UserRole).toString()); });
    auto *row = new QHBoxLayout;
    auto *open = new QPushButton("Open workspace");
    auto *fresh = new QPushButton("Create new");
    auto *demo = new QPushButton("Try demo");
```

```
    open->setObjectName("welcomeOpen");
    fresh->setObjectName("welcomeCreate");
    demo->setObjectName("welcomeDemo");
    row->addWidget(open);
    row->addWidget(fresh);
    row->addWidget(demo);
    layout->addLayout(row);
    connect(open, &QPushButton::clicked, &dialog, [&] {
        choose(QFileDialog::getOpenFileName(&dialog, "Open workspace", {},
                                            "Clinic workspace (*.clinic *.json)"));
    });
    connect(fresh, &QPushButton::clicked, &dialog, [&] {
        prepareChange();
        manager_.newDatabase();
        currentPath_.clear();
        workspaceGuard_.reset();
        refresh();
        dialog.accept();
    });
    connect(demo, &QPushButton::clicked, &dialog, [&] {
        try {
            auto sample = QCoreApplication::applicationDirPath() + "/sample-clinic.json";
            if (!QFileInfo::exists(sample))
                sample = QCoreApplication::applicationDirPath() + "/../data/sample-clinic.json";
            auto db = FileHandler::load(sample);
            prepareChange();
            manager_.replace(std::move(db));
            currentPath_.clear();
            workspaceGuard_.reset();
            refresh();
            dialog.accept();
        } catch (const std::exception &e) {
            QMessageBox::warning(&dialog, "Demo unavailable", e.what());
        }
    });
    auto *recoverButton = new QPushButton("Review local recovery copies");
    auto *importButton = new QPushButton("Import portable encrypted backup");
    try {
        const auto count = RecoveryService::copies({}).size();
        if (count)
            layout->addWidget(new QLabel(
                QString("%1 local recovery copies are available for review.").arg(count)));
    } catch (const std::exception &e) {
        auto *message = new QLabel(QString("Recovery folder unavailable: %1").arg(e.what()));
        message->setWordWrap(true);
        layout->addWidget(message);
    }
    layout->addWidget(recoverButton);
    layout->addWidget(importButton);
    connect(recoverButton, &QPushButton::clicked, &dialog, [&] {
        recover();
        if (manager_.dirty())
            dialog.accept();
    });
    connect(importButton, &QPushButton::clicked, &dialog, [&] {
        portableBackup(true);
        if (manager_.dirty())
            dialog.accept();
    });
    auto *close = new QDialogButtonBox(QDialogButtonBox::Close);
    layout->addWidget(close);
    connect(close, &QDialogButtonBox::rejected, &dialog, &QDialog::reject);
    setTextSize(QSettings().value("textSize", 10).toInt());
    return dialog.exec() == QDialog::Accepted;
}
void MainWindow::offerRecovery() {
    try {
        const auto copies = RecoveryService::copies(currentPath_);
        if (copies.isEmpty())
            return;
        bool hasAutosave = false;
        for (const auto &p : copies)
            if (p.endsWith(".recovery.clinic") || QFileInfo(p).fileName().startsWith("unsaved"))
                hasAutosave = true;
        if (hasAutosave &&
```

```
144             QMessageBox::question(this, "Recovery available",
145                                   "An autosave is available. Review its date and record counts "
146                                   "before choosing whether to restore it?",
147                                   QMessageBox::Yes | QMessageBox::No) == QMessageBox::Yes)
148             recover();
149     } catch (const std::exception &e) {
150         QMessageBox::warning(this, "Recovery unavailable", e.what());
151     }
152 }
153 void MainWindow::recover() {
154     if (!confirmDiscard())
155         return;
156     try {
157         QDialog dialog(this);
158         dialog.setWindowTitle("Review recovery copies");
159         dialog.resize(850, 400);
160         auto *layout = new QVBoxLayout(&dialog);
161         layout->addWidget(new QLabel("Review the saved time and counts. Restoring opens an unsaved
  > "
162                                      "copy; the original is retained."));
163         auto *list = new QListWidget;
164         layout->addWidget(list);
165         auto add = [&](const QString &p) {
166             QString description =
167                 QFileInfo(p).lastModified().toLocalTime().toString("dd MMM yyyy HH:mm:ss") + " / "
  > +
168                 QFileInfo(p).fileName();
169             try {
170                 const auto db = FileHandler::load(p);
171                 description += QString(" / %1 patients / %2 visits")
172                                    .arg(db.patients.size())
173                                    .arg(db.appointments.size());
174             } catch (const std::exception &) {
175                 description += " / Cannot read: damaged or another Windows account";
176             }
177             auto *item = new QListWidgetItem(description, list);
178             item->setData(Qt::UserRole, p);
179             item->setToolTip(p);
180         };
181         for (const auto &p : RecoveryService::copies(currentPath_))
182             add(p);
183         if (list->count() == 0)
184             layout->addWidget(
185                 new QLabel("No automatic copies found. Browse to a protected recovery file."));
186         auto *browse = new QPushButton("Browse for a recovery file");
187         layout->addWidget(browse);
188         connect(browse, &QPushButton::clicked, &dialog, [&] {
189             auto p = QFileDialog::getOpenFileName(&dialog, "Choose recovery copy", {},
190                                                   "Clinic files (*.clinic *.json)");
191             if (!p.isEmpty()) {
192                 add(p);
193                 list->setCurrentRow(list->count() - 1);
194             }
195         });
196         auto *buttons =
197             new QDialogButtonBox(QDialogButtonBox::RestoreDefaults | QDialogButtonBox::Cancel);
198         buttons->button(QDialogButtonBox::RestoreDefaults)->setText("Restore selected");
199         layout->addWidget(buttons);
200         connect(buttons->button(QDialogButtonBox::RestoreDefaults), &QPushButton::clicked, &dialog,
201                 [&] {
202                     if (!list->currentItem()) {
203                         QMessageBox::information(&dialog, "Select a copy",
204                                                  "Select a recovery copy first.");
205                         return;
206                     }
207                     if (openWorkspace(list->currentItem()->data(Qt::UserRole).toString())) {
208                         currentPath_.clear();
209                         workspaceGuard_.reset();
210                         manager_.markUnsaved();
211                         refresh();
212                         dialog.accept();
213                     }
214                 });
215         connect(buttons, &QDialogButtonBox::rejected, &dialog, &QDialog::reject);
216         if (list->count())
```

```
            list->setCurrentRow(0);
        dialog.exec();
    } catch (const std::exception &e) {
        QMessageBox::warning(this, "Recovery failed", e.what());
    }
}
void MainWindow::portableBackup(bool importing) {
    if (!importing && !manager_.isAdmin()) {
        QMessageBox::warning(this, "Backup",
                             "Only administrators may export a full workspace backup.");
        return;
    }
    if (importing && !confirmDiscard())
        return;
    const auto file = importing ? QFileDialog::getOpenFileName(this, "Import portable backup", {},
                                                                "Encrypted backup (*.scapbackup)")
                                : QFileDialog::getSaveFileName(this, "Export portable backup",
                                                               "clinic-backup.scapbackup",
                                                               "Encrypted backup (*.scapbackup)");
    if (file.isEmpty())
        return;
    bool ok = false;
    auto pass =
        QInputDialog::getText(this, "Backup passphrase",
                              importing ? "Enter the backup passphrase"
                                        : "Choose a separate backup passphrase (12-256 "
                                          "characters). Keep it safely; it cannot be recovered.",
                              QLineEdit::Password, {}, &ok);
    if (!ok)
        return;
    if (!importing) {
        auto confirmation =
            QInputDialog::getText(this, "Confirm passphrase", "Repeat the backup passphrase",
                                  QLineEdit::Password, {}, &ok);
        if (!ok)
            return;
        if (pass != confirmation) {
            QMessageBox::warning(this, "Backup", "Passphrases do not match.");
            return;
        }
    }
    try {
        if (importing) {
            QFile input(file);
            if (!input.open(QIODevice::ReadOnly) || input.size() > 21 * 1024 * 1024)
                throw model::FileError("Cannot read backup or backup exceeds size limit.");
            const auto plain = security::decryptBackup(input.readAll(), pass);
            QJsonParseError error;
            const auto json = QJsonDocument::fromJson(plain, &error);
            if (error.error != QJsonParseError::NoError || !json.isObject())
                throw model::FileError("Invalid backup contents.");
            auto db = FileHandler::deserialize(json.object());
            controller::ClinicManager candidate;
            if (!db.users.empty()) {
                auto user = QInputDialog::getText(this, "Staff sign-in",
                                                  "Administrator username from the backup",
                                                  QLineEdit::Normal, {}, &ok);
                if (!ok)
                    return;
                auto secret = QInputDialog::getText(this, "Staff sign-in", "Staff password",
                                                    QLineEdit::Password, {}, &ok);
                if (!ok)
                    return;
                candidate.replace(std::move(db), user, secret);
                if (!candidate.isAdmin())
                    throw model::ValidationError(
                        "An administrator from the backup must restore it.");
            } else
                candidate.replace(std::move(db));
            prepareChange();
            manager_ = std::move(candidate);
            manager_.markUnsaved();
            currentPath_.clear();
            workspaceGuard_.reset();
            refresh();
```

```
            QMessageBox::information(this, "Backup restored",
                                     "The backup is open as an unsaved workspace. Save As to "
                                     "create a protected file on this Windows account.");
        } else {
            auto destination = file;
            if (!destination.endsWith(".scapbackup", Qt::CaseInsensitive))
                destination += ".scapbackup";
            const auto plain = QJsonDocument(FileHandler::serialize(manager_.database()))
                                   .toJson(QJsonDocument::Compact);
            const auto cipher = security::encryptBackup(plain, pass);
            QSaveFile output(destination);
            if (!output.open(QIODevice::WriteOnly) || output.write(cipher) != cipher.size() ||
                !output.commit())
                throw model::FileError("Could not save the encrypted backup.");
            manager_.recordExport("Export portable encrypted backup");
            updateTitle();
            QMessageBox::information(this, "Backup exported",
                                     "The backup can be imported on another Windows computer using
  > "
                                     "its backup passphrase and staff administrator credentials.");
        }
    } catch (const std::exception &e) {
        QMessageBox::warning(this, "Backup failed", e.what());
    }
}
} // namespace clinic::gui
```

5.38 SOURCE OR RESOURCE FILE

src/gui/panels/AccessPanel.cpp

```
#include "AccessPanel.h"
namespace clinic::gui {
AccessPanel::AccessPanel(controller::ClinicManager &m, MutationHandler f,
                         std::function<void()> secureWorkspace, QWidget *parent)
    : PanelBase(m, std::move(f), parent) {
    auto *layout = panelLayout(this, "Staff access & activity",
                               "Staff roles protect application actions. Protected files and "
                               "recovery copies are tied to this Windows account.");
    state_ = new QLabel;
    state_->setWordWrap(true);
    layout->addWidget(state_);
    auto *actions = new QHBoxLayout;
    auto *enable = actionButton("Protect this workspace", "enableSecurity", true);
    auto *add = actionButton("Add staff account", "addUser");
    auto *toggle = actionButton("Enable / disable selected", "toggleUser");
    auto *password = actionButton("Change my password", "changePassword");
    for (auto *b : {enable, add, toggle, password})
        actions->addWidget(b);
    layout->addLayout(actions);
    auto *tabs = new QTabWidget;
    users_ = makeTable({"Username", "Role", "Status"}, "staffTable");
    audit_ = makeTable({"Time", "Actor", "Action"}, "auditTable");
    tabs->addTab(users_, "Staff accounts");
    tabs->addTab(audit_, "Activity history");
    layout->addWidget(tabs, 1);
    auto *note =
        new QLabel("Administrator: all actions. Clinician: care workflows and notes. Receptionist:
 > "
                   "registration, bookings and check-in. Viewer: read-only records and reports. "
                   "Clinical notes are restricted to clinicians and administrators.");
    note->setWordWrap(true);
    layout->addWidget(note);
    connect(enable, &QPushButton::clicked, this, [secureWorkspace] { secureWorkspace(); });
    connect(add, &QPushButton::clicked, this, &AccessPanel::addAccount);
    connect(password, &QPushButton::clicked, this, &AccessPanel::changePassword);
    connect(toggle, &QPushButton::clicked, this, [this] {
        const auto name = selectedId(users_);
        bool enabled = true;
        for (const auto &u : manager_.database().users)
            if (u.username == name)
                enabled = u.active;
        mutate_([&] { manager_.setUserActive(name, !enabled); }, "Account status updated.");
    });
}
void AccessPanel::refresh() {
    const bool admin = manager_.isAdmin(), demo = manager_.demoMode();
    state_->setText(demo ? "DEMO MODE: no staff authentication. Use only fictional records until "
                           "you protect the workspace."
                         : "Signed in as " + manager_.username() + " / " +
                               security::roleName(manager_.role()) +
                               ". Workspace automatically locks after five minutes of inactivity.")
 > ;
    findChild<QPushButton *>("enableSecurity")->setEnabled(demo);
    findChild<QPushButton *>("addUser")->setEnabled(admin && !demo);
    findChild<QPushButton *>("toggleUser")->setEnabled(admin && !demo);
    findChild<QPushButton *>("changePassword")->setEnabled(!demo && manager_.authenticated());
    QList<QStringList> users, audit;
    if (admin) {
        for (const auto &u : manager_.database().users)
            users.append(
                {u.username, security::roleName(u.role), u.active ? "Enabled" : "Disabled"});
        for (auto it = manager_.database().audit.rbegin(); it != manager_.database().audit.rend();
             ++it)
            audit.append(
                {it->time.toLocalTime().toString("dd MMM yyyy HH:mm:ss"), it->actor, it->action});
    }
    fillTable(users_, users);
    fillTable(audit_, audit);
}
```

```
68 void AccessPanel::addAccount() {
69     QDialog dialog(this);
70     dialog.setWindowTitle("Add staff account");
71     auto *form = new QFormLayout(&dialog);
72     auto *name = new QLineEdit;
73     auto *password = new QLineEdit;
74     password->setEchoMode(QLineEdit::Password);
75     auto *role = new QComboBox;
76     role->addItems({"Administrator", "Clinician", "Receptionist", "Viewer"});
77     form->addRow("Username", name);
78     form->addRow("Password (12+ characters)", password);
79     form->addRow("Role", role);
80     auto *buttons = new QDialogButtonBox(QDialogButtonBox::Save | QDialogButtonBox::Cancel);
81     form->addRow(buttons);
82     connect(buttons, &QDialogButtonBox::accepted, &dialog, &QDialog::accept);
83     connect(buttons, &QDialogButtonBox::rejected, &dialog, &QDialog::reject);
84     if (dialog.exec() == QDialog::Accepted)
85         mutate_(
86             [&] {
87                 manager_.addUser(name->text(), password->text(),
88                                  security::parseRole(role->currentText()));
89             },
90             "Staff account added. Save the workspace.");
91 }
92 void AccessPanel::changePassword() {
93     QDialog dialog(this);
94     dialog.setWindowTitle("Change password");
95     auto *form = new QFormLayout(&dialog);
96     auto *current = new QLineEdit;
97     current->setEchoMode(QLineEdit::Password);
98     auto *replacement = new QLineEdit;
99     replacement->setEchoMode(QLineEdit::Password);
100    form->addRow("Current password", current);
101    form->addRow("New password (12+ characters)", replacement);
102    auto *buttons = new QDialogButtonBox(QDialogButtonBox::Save | QDialogButtonBox::Cancel);
103    form->addRow(buttons);
104    connect(buttons, &QDialogButtonBox::accepted, &dialog, &QDialog::accept);
105    connect(buttons, &QDialogButtonBox::rejected, &dialog, &QDialog::reject);
106    if (dialog.exec() == QDialog::Accepted)
107        mutate_([&] { manager_.changePassword(current->text(), replacement->text()); },
108                "Password changed. Save the workspace.");
109 }
110 } // namespace clinic::gui
```

5.39 SOURCE OR RESOURCE FILE

src/gui/panels/AppointmentsPanel.cpp

```
#include "AppointmentsPanel.h"
namespace clinic::gui {
AppointmentsPanel::AppointmentsPanel(controller::ClinicManager &m, MutationHandler f,
                                     QWidget *parent)
    : PanelBase(m, std::move(f), parent) {
    auto *layout = panelLayout(
        this, "Appointments",
        "Schedule > Check in > Call next > Complete consultation. All changes support undo.");
    auto *form = new QGridLayout;
    patient_ = new QComboBox;
    patient_->setObjectName("appointmentPatient");
    service_ = new QComboBox;
    service_->setObjectName("appointmentService");
    service_->setEditable(true);
    service_->addItems(
        {"General consultation", "Follow-up review", "Emergency review", "Health screening"});
    time_ = new QDateTimeEdit(QDateTime::currentDateTime().addSecs(3600));
    time_->setObjectName("appointmentTime");
    time_->setCalendarPopup(true);
    time_->setDisplayFormat("dd MMM yyyy HH:mm");
    duration_ = new QSpinBox;
    duration_->setObjectName("appointmentDuration");
    duration_->setRange(5, 240);
    duration_->setValue(30);
    duration_->setSuffix(" min");
    const QStringList labels{"Patient", "Service", "Date and time", "Duration"};
    for (int i = 0; i < 4; ++i)
        form->addWidget(new QLabel(labels[i]), 0, i);
    form->addWidget(patient_, 1, 0);
    form->addWidget(service_, 1, 1);
    form->addWidget(time_, 1, 2);
    form->addWidget(duration_, 1, 3);
    clinician_ = new QComboBox;
    room_ = new QComboBox;
    clinician_->setObjectName("appointmentClinician");
    room_->setObjectName("appointmentRoom");
    form->addWidget(new QLabel("Clinician"), 2, 0);
    form->addWidget(new QLabel("Room"), 2, 2);
    form->addWidget(clinician_, 3, 0, 1, 2);
    form->addWidget(room_, 3, 2, 1, 2);
    layout->addLayout(form);
    auto *actions = new QGridLayout;
    auto *book = actionButton("+ Schedule visit", "scheduleAppointment", true);
    auto *move = actionButton("Reschedule", "rescheduleAppointment");
    auto *check = actionButton("Check in", "checkInAppointment", true);
    auto *complete = actionButton("Complete consultation", "completeAppointment");
    auto *cancel = actionButton("Cancel selected", "cancelAppointment");
    int actionIndex = 0;
    for (auto *b : {book, move, check, complete, cancel}) {
        actions->addWidget(b, actionIndex / 4, actionIndex % 4);
        ++actionIndex;
    }
    auto *noShow = actionButton("No-show", "noShowAppointment");
    auto *correct = actionButton("Correct notes", "correctAppointment");
    actions->addWidget(noShow, 1, 1);
    actions->addWidget(correct, 1, 2);
    layout->addLayout(actions);
    auto *filters = new QHBoxLayout;
    status_ = new QComboBox;
    status_->setObjectName("appointmentStatusFilter");
    status_->addItems({"All statuses", "Scheduled", "Checked in", "In consultation", "Completed",
                       "Cancelled", "No-show"});
    today_ = new QCheckBox("Today only");
    today_->setObjectName("appointmentsTodayOnly");
    filters->addWidget(new QLabel("FILTER"));
    filters->addWidget(status_);
    filters->addWidget(today_);
    filters->addStretch();
    layout->addLayout(filters);
```

```
    table_ = makeTable({"Appointment", "Patient", "Service", "Date / time", "Duration", "Status"},
                       "appointmentsTable");
    layout->addWidget(table_, 1);
    connect(book, &QPushButton::clicked, this, [this] {
        const auto id = patient_->currentData().toString();
        mutate_(
            [&] {
                manager_.schedule(id, service_->currentText(), time_->dateTime(),
                                  duration_->value(), clinician_->currentData().toString(),
                                  room_->currentData().toString());
            },
            "Appointment scheduled.");
    });
    connect(move, &QPushButton::clicked, this, &AppointmentsPanel::reschedule);
    connect(check, &QPushButton::clicked, this, [this] {
        const auto id = selectedId(table_);
        mutate_([&] { manager_.checkIn(id); }, "Patient checked in and added to waiting queue.");
    });
    connect(cancel, &QPushButton::clicked, this, [this] {
        const auto id = selectedId(table_);
        if (id.isEmpty()) {
            QMessageBox::information(this, "Cancel visit", "Select an appointment first.");
            return;
        }
        bool ok = false;
        const auto reason = QInputDialog::getMultiLineText(this, "Cancel appointment",
                                                           "Cancellation reason", {}, &ok);
        if (ok)
            mutate_([&] { manager_.cancel(id, reason); }, "Appointment cancelled.");
    });
    connect(complete, &QPushButton::clicked, this, [this] {
        const auto id = selectedId(table_);
        if (id.isEmpty())
            return;
        bool accepted = false;
        const auto notes = QInputDialog::getMultiLineText(this, "Complete consultation",
                                                          "Consultation notes", {}, &accepted);
        if (accepted)
            mutate_([&] { manager_.complete(id, notes); }, "Consultation completed.");
    });
    connect(noShow, &QPushButton::clicked, this, [this] {
        const auto id = selectedId(table_);
        bool ok = false;
        if (id.isEmpty()) {
            QMessageBox::information(this, "No-show", "Select an appointment first.");
            return;
        }
        const auto reason = QInputDialog::getMultiLineText(this, "Mark no-show",
                                                           "Reason / contact attempt", {}, &ok);
        if (ok)
            mutate_([&] { manager_.noShow(id, reason); }, "Visit marked no-show.");
    });
    connect(correct, &QPushButton::clicked, this, [this] {
        if (!manager_.canViewClinicalNotes())
            return;
        const auto id = selectedId(table_);
        const auto *a = manager_.database().appointment(id);
        if (!a || a->status != model::AppointmentStatus::Completed) {
            QMessageBox::information(this, "Correct notes", "Select a completed consultation.");
            return;
        }
        bool ok = false;
        auto notes = QInputDialog::getMultiLineText(this, "Correct consultation notes",
                                                    "Replacement notes", a->consultationNotes, &ok)
  > ;
        if (!ok)
            return;
        auto reason = QInputDialog::getMultiLineText(this, "Correction reason",
                                                     "Why are the notes being corrected?", {}, &ok)
  > ;
        if (ok)
            mutate_([&] { manager_.correctNotes(id, notes, reason); },
                    "Notes corrected; previous notes retained in history.");
    });
    connect(status_, &QComboBox::currentIndexChanged, this, [this] { refresh(); });
```

```
    connect(today_, &QCheckBox::toggled, this, [this] { refresh(); });
    connect(table_, &QTableWidget::doubleClicked, this, [this] {
        const auto *a = manager_.database().appointment(selectedId(table_));
        if (a)
            showPatientHistory(this, manager_, a->patientId);
    });
}
void AppointmentsPanel::refresh() {
    fillPatientChoices(patient_, manager_.database());
    auto fill = [](QComboBox *combo, const QStringList &values) {
        const auto old = combo->currentData().toString();
        combo->clear();
        combo->addItem("Unassigned", {});
        for (const auto &v : values)
            combo->addItem(v, v);
        combo->setCurrentIndex(qMax(0, combo->findData(old)));
    };
    fill(clinician_, manager_.database().clinicians);
    fill(room_, manager_.database().rooms);
    QList<QStringList> rows;
    for (const auto &a : manager_.database().appointments) {
        if (today_->isChecked() && a.start.toLocalTime().date() != QDate::currentDate())
            continue;
        if (status_->currentIndex() > 0 && model::statusName(a.status) != status_->currentText())
            continue;
        rows.append({a.id, manager_.database().patient(a.patientId)->details().name, a.service,
                     a.start.toLocalTime().toString("dd MMM yyyy HH:mm"),
                     QString::number(a.durationMinutes) + " min", model::statusName(a.status)});
    }
    fillTable(table_, rows);
    for (const auto &name : {"scheduleAppointment", "rescheduleAppointment", "checkInAppointment",
                             "cancelAppointment", "noShowAppointment"})
        findChild<QPushButton *>(name)->setEnabled(manager_.canEdit());
    findChild<QPushButton *>("completeAppointment")->setEnabled(manager_.canViewClinicalNotes());
    findChild<QPushButton *>("correctAppointment")->setEnabled(manager_.canViewClinicalNotes());
}
void AppointmentsPanel::reschedule() {
    const auto id = selectedId(table_);
    const auto *a = manager_.database().appointment(id);
    if (!a) {
        QMessageBox::information(this, "Reschedule", "Select an appointment first.");
        return;
    }
    QDialog dialog(this);
    dialog.setObjectName("rescheduleDialog");
    dialog.setWindowTitle("Reschedule " + id);
    auto *form = new QFormLayout(&dialog);
    auto *time = new QDateTimeEdit(a->start);
    time->setObjectName("rescheduleTime");
    time->setCalendarPopup(true);
    time->setDisplayFormat("dd MMM yyyy HH:mm");
    auto *minutes = new QSpinBox;
    minutes->setObjectName("rescheduleDuration");
    minutes->setRange(5, 240);
    minutes->setValue(a->durationMinutes);
    form->addRow("New date / time", time);
    form->addRow("Duration (minutes)", minutes);
    auto *buttons = new QDialogButtonBox(QDialogButtonBox::Save | QDialogButtonBox::Cancel);
    form->addRow(buttons);
    connect(buttons, &QDialogButtonBox::accepted, &dialog, &QDialog::accept);
    connect(buttons, &QDialogButtonBox::rejected, &dialog, &QDialog::reject);
    if (dialog.exec() == QDialog::Accepted)
        mutate_([&] { manager_.reschedule(id, time->dateTime(), minutes->value()); },
                "Appointment rescheduled.");
}
} // namespace clinic::gui
```

5.40 SOURCE OR RESOURCE FILE

src/gui/panels/CalendarPanel.cpp

```
#include "CalendarPanel.h"
#include "algorithms/Algorithms.h"
namespace clinic::gui {
CalendarPanel::CalendarPanel(controller::ClinicManager &m, MutationHandler f, QWidget *parent)
    : PanelBase(m, std::move(f), parent) {
    auto *layout = panelLayout(this, "Clinician calendar and rooms",
                               "Choose a day and filter by clinician or room. Active visits cannot
> "
                               "overlap for the same resource.");
    auto *top = new QHBoxLayout;
    calendar_ = new QCalendarWidget;
    calendar_->setObjectName("resourceCalendar");
    calendar_->setGridVisible(true);
    top->addWidget(calendar_);
    auto *options = new QVBoxLayout;
    clinician_ = new QComboBox;
    room_ = new QComboBox;
    clinician_->setObjectName("calendarClinician");
    room_->setObjectName("calendarRoom");
    options->addWidget(new QLabel("Clinician"));
    options->addWidget(clinician_);
    options->addWidget(new QLabel("Room"));
    options->addWidget(room_);
    auto *addClinician = actionButton("Add clinician", "addClinician");
    auto *addRoom = actionButton("Add room", "addRoom");
    auto *assign = actionButton("Allocate selected visit", "allocateResources", true);
    options->addWidget(addClinician);
    options->addWidget(addRoom);
    options->addWidget(assign);
    options->addStretch();
    top->addLayout(options, 1);
    layout->addLayout(top);
    summary_ = new QLabel;
    summary_->setWordWrap(true);
    layout->addWidget(summary_);
    table_ =
        makeTable({"Visit", "Time", "Patient", "Clinician", "Room", "Status"}, "calendarTable");
    layout->addWidget(table_, 1);
    connect(calendar_, &QCalendarWidget::selectionChanged, this, [this] { refresh(); });
    connect(clinician_, &QComboBox::currentIndexChanged, this, [this] { refresh(); });
    connect(room_, &QComboBox::currentIndexChanged, this, [this] { refresh(); });
    for (auto pair : {qMakePair(addClinician, false), qMakePair(addRoom, true)}) {
        connect(pair.first, &QPushButton::clicked, this, [this, isRoom = pair.second] {
            bool ok = false;
            auto name = QInputDialog::getText(this, isRoom ? "Add room" : "Add clinician", "Name",
                                              QLineEdit::Normal, {}, &ok);
            if (ok)
                mutate_([&] { manager_.addResource(name, isRoom); }, "Resource added.");
        });
    }
    connect(assign, &QPushButton::clicked, this, &CalendarPanel::allocate);
    connect(table_, &QTableWidget::doubleClicked, this, [this] {
        if (auto *a = manager_.database().appointment(selectedId(table_)))
            showPatientHistory(this, manager_, a->patientId);
    });
}
void CalendarPanel::refresh() {
    const auto &db = manager_.database();
    auto fill = [](QComboBox *combo, const QStringList &names) {
        const QSignalBlocker blocker(combo);
        auto current = combo->currentData().toString();
        combo->clear();
        combo->addItem("All", {});
        for (const auto &name : names)
            combo->addItem(name, name);
        auto index = combo->findData(current);
        combo->setCurrentIndex(index < 0 ? 0 : index);
    };
    fill(clinician_, db.clinicians);
```

```
    fill(room_, db.rooms);
    std::vector<const model::Appointment *> visits;
    for (const auto &a : db.appointments) {
        if (a.start.toLocalTime().date() != calendar_->selectedDate())
            continue;
        if (!clinician_->currentData().toString().isEmpty() &&
            a.clinician != clinician_->currentData().toString())
            continue;
        if (!room_->currentData().toString().isEmpty() && a.room != room_->currentData().toString()
 > )
            continue;
        visits.push_back(&a);
    }
    algorithms::mergeSort(visits, [](auto a, auto b) { return a->start < b->start; });
    QList<QStringList> rows;
    for (auto *a : visits)
        rows.append({a->id,
                     a->start.toLocalTime().toString("HH:mm") + " - " +
                         a->end().toLocalTime().toString("HH:mm"),
                     db.patient(a->patientId)->details().name,
                     a->clinician.isEmpty() ? "Unassigned" : a->clinician,
                     a->room.isEmpty() ? "Unassigned" : a->room, model::statusName(a->status)});
    fillTable(table_, rows);
    summary_->setText(
        rows.isEmpty()
            ? "No visits match this day and resource filter. Schedule a visit on the Appointments "
              "page."
            : QString("%1 visits on %2. Unassigned visits remain visible when All is selected.")
                  .arg(rows.size())
                  .arg(calendar_->selectedDate().toString("dd MMM yyyy")));
    findChild<QPushButton *>("addClinician")->setEnabled(manager_.isAdmin());
    findChild<QPushButton *>("addRoom")->setEnabled(manager_.isAdmin());
    findChild<QPushButton *>("allocateResources")
        ->setEnabled(manager_.canEdit() && !rows.isEmpty());
}
void CalendarPanel::allocate() {
    const auto id = selectedId(table_);
    const auto *a = manager_.database().appointment(id);
    if (!a) {
        QMessageBox::information(this, "Choose a visit", "Select a visit in the calendar first.");
        return;
    }
    QDialog dialog(this);
    dialog.setWindowTitle("Allocate " + id);
    auto *form = new QFormLayout(&dialog);
    auto *clinician = new QComboBox;
    auto *room = new QComboBox;
    for (auto *c : {clinician, room})
        c->addItem("Unassigned", {});
    for (const auto &x : manager_.database().clinicians)
        clinician->addItem(x, x);
    for (const auto &x : manager_.database().rooms)
        room->addItem(x, x);
    clinician->setCurrentIndex(qMax(0, clinician->findData(a->clinician)));
    room->setCurrentIndex(qMax(0, room->findData(a->room)));
    form->addRow("Clinician", clinician);
    form->addRow("Room", room);
    auto *buttons = new QDialogButtonBox(QDialogButtonBox::Save | QDialogButtonBox::Cancel);
    form->addRow(buttons);
    connect(buttons, &QDialogButtonBox::accepted, &dialog, &QDialog::accept);
    connect(buttons, &QDialogButtonBox::rejected, &dialog, &QDialog::reject);
    if (dialog.exec() == QDialog::Accepted)
        mutate_(
            [&] {
                manager_.assignResources(id, clinician->currentData().toString(),
                                         room->currentData().toString());
            },
            "Resources allocated.");
}
} // namespace clinic::gui
```

5.41 SOURCE OR RESOURCE FILE

src/gui/panels/DashboardPanel.cpp

```
#include "DashboardPanel.h"
#include "algorithms/Algorithms.h"
namespace clinic::gui {
DashboardPanel::DashboardPanel(controller::ClinicManager &m, MutationHandler f, QWidget *parent)
    : PanelBase(m, std::move(f), parent) {
    auto *layout = panelLayout(this, "Today's care at a glance",
                               "Upcoming visits, overdue appointments and live waiting times.");
    auto *metrics = new QHBoxLayout;
    auto card = [&](const QString &name, QLabel *&count) {
        auto *frame = new QFrame;
        frame->setObjectName("card");
        auto *body = new QVBoxLayout(frame);
        body->addWidget(new QLabel(name));
        count = new QLabel("0");
        count->setObjectName("metric");
        body->addWidget(count);
        metrics->addWidget(frame);
    };
    card("ACTIVE PATIENTS", patients_);
    card("PENDING VISITS", pending_);
    card("WAITING NOW", waiting_);
    card("COMPLETED TODAY", completed_);
    layout->addLayout(metrics);
    reminder_ = new QLabel;
    reminder_->setWordWrap(true);
    reminder_->setObjectName("dashboardReminder");
    layout->addWidget(reminder_);
    auto *tabs = new QTabWidget;
    tabs->setObjectName("dashboardLists");
    today_ = makeTable({"Appointment", "Patient", "Time", "Service", "Status"}, "todayTable");
    overdue_ = makeTable({"Appointment", "Patient", "Scheduled for", "Overdue"}, "overdueTable");
    waits_ = makeTable({"Patient ID", "Patient", "Appointment", "Waiting"}, "dashboardWaiting");
    cache_ = makeTable({"Patient ID", "Patient", "Category", "Record views"}, "cacheTable");
    tabs->addTab(today_, "Today's appointments");
    tabs->addTab(overdue_, "Overdue visits");
    tabs->addTab(waits_, "Waiting times");
    tabs->addTab(cache_, "Frequently accessed");
    layout->addWidget(tabs, 1);
    for (auto *table : {today_, overdue_})
        connect(table, &QTableWidget::doubleClicked, this, [this, table] {
            const auto *a = manager_.database().appointment(selectedId(table));
            if (a)
                showPatientHistory(this, manager_, a->patientId);
        });
    connect(cache_, &QTableWidget::doubleClicked, this,
            [this] { showPatientHistory(this, manager_, selectedId(cache_)); });
}
void DashboardPanel::refresh() {
    const auto &db = manager_.database();
    const auto now = QDateTime::currentDateTime();
    patients_->setText(QString::number(db.patients.size() - db.archivedPatients.size()));
    waiting_->setText(QString::number(db.waiting.size()));
    int pending = 0, completed = 0, soon = 0;
    QList<QStringList> today, overdue, waits, frequent;
    std::vector<const model::Appointment *> appointments;
    for (const auto &a : db.appointments)
        appointments.push_back(&a);
    algorithms::mergeSort(appointments,
                          [](const auto *a, const auto *b) { return a->start < b->start; });
    for (const auto *a : appointments) {
        if (model::isPending(a->status))
            ++pending;
        if (a->status == model::AppointmentStatus::Scheduled && a->start >= now &&
            a->start <= now.addSecs(3600))
            ++soon;
        if (a->start.toLocalTime().date() == QDate::currentDate()) {
            today.append({a->id, db.patient(a->patientId)->details().name,
                          a->start.toLocalTime().toString("HH:mm"), a->service,
                          model::statusName(a->status)});
```

```
            if (a->status == model::AppointmentStatus::Completed)
                ++completed;
        }
        if (a->status == model::AppointmentStatus::Scheduled && a->start < now)
            overdue.append({a->id, db.patient(a->patientId)->details().name,
                            a->start.toLocalTime().toString("dd MMM HH:mm"),
                            QString::number(a->start.secsTo(now) / 60) + " min"});
    }
    auto queue = db.waiting;
    while (!queue.empty()) {
        const auto &e = queue.front();
        waits.append({e.patientId, db.patient(e.patientId)->details().name,
                      e.appointmentId.isEmpty() ? "Walk-in" : e.appointmentId,
                      QString::number(qMax<qint64>(0, e.arrivedAt.secsTo(now) / 60)) + " min"});
        queue.pop();
    }
    std::vector<model::CacheEntry> entries;
    for (const auto &e : db.frequent)
        if (!e.patientId.isEmpty())
            entries.push_back(e);
    algorithms::insertionSort(entries,
                              [](const auto &a, const auto &b) { return a.accesses > b.accesses; })
 > ;
    for (const auto &e : entries)
        if (const auto *p = db.patient(e.patientId))
            frequent.append({p->id(), p->details().name, model::categoryName(p->category()),
                             QString::number(e.accesses)});
    pending_->setText(QString::number(pending));
    completed_->setText(QString::number(completed));
    reminder_->setText(QString("%1 | %2 visits in the next hour | %3 overdue | %4 waiting")
                           .arg(now.toString("dddd, dd MMMM yyyy"))
                           .arg(soon)
                           .arg(overdue.size())
                           .arg(waits.size()));
    fillTable(today_, today);
    fillTable(overdue_, overdue);
    fillTable(waits_, waits);
    fillTable(cache_, frequent);
}
} // namespace clinic::gui
```

5.42 SOURCE OR RESOURCE FILE

src/gui/panels/PanelBase.cpp

```
#include "PanelBase.h"
namespace clinic::gui {
namespace {
class EmptyState final : public QObject {
    QPointer<QWidget> viewport_;
    QPointer<QAbstractItemModel> model_;
    QPointer<QLabel> label_;

  public:
    explicit EmptyState(QTableView *table)
        : QObject(table->viewport()), viewport_(table->viewport()), model_(table->model()),
          label_(new QLabel(table->viewport())) {
        const auto name = table->objectName();
        label_->setText(
            name == "patientsTable"
                ? "No patient records in this view. Register a patient or include archived records.
 > "
                : name == "searchResults"
                ? "No matching patients. Change your search or choose Show all."
                : "No records in this view. Adjust the date or filters, or add a record.");
        label_->setAlignment(Qt::AlignCenter);
        label_->setWordWrap(true);
        label_->setAttribute(Qt::WA_TransparentForMouseEvents);
        label_->setStyleSheet("color:#536b76; background:transparent; padding:24px;");
        table->viewport()->installEventFilter(this);
        QObject::connect(table->model(), &QAbstractItemModel::modelReset, this,
                         [this] { refresh(); });
        QObject::connect(table->model(), &QAbstractItemModel::rowsInserted, this,
                         [this] { refresh(); });
        QObject::connect(table->model(), &QAbstractItemModel::rowsRemoved, this,
                         [this] { refresh(); });
        refresh();
    }
    void refresh() {
        if (!viewport_ || !model_ || !label_)
            return;
        label_->setGeometry(viewport_->rect());
        label_->setVisible(model_->rowCount() == 0);
    }
    bool eventFilter(QObject *object, QEvent *event) override {
        if (event->type() == QEvent::Resize)
            refresh();
        return QObject::eventFilter(object, event);
    }
};
} // namespace

QVBoxLayout *panelLayout(QWidget *widget, const QString &title, const QString &subtitle) {
    auto *layout = new QVBoxLayout(widget);
    layout->setContentsMargins(24, 20, 24, 20);
    layout->setSpacing(12);
    auto *heading = new QLabel(title);
    heading->setObjectName("pageTitle");
    layout->addWidget(heading);
    auto *caption = new QLabel(subtitle);
    caption->setObjectName("caption");
    caption->setWordWrap(true);
    layout->addWidget(caption);
    return layout;
}
QPushButton *actionButton(const QString &title, const QString &name, bool primary) {
    auto *b = new QPushButton(title);
    b->setObjectName(name);
    b->setAccessibleName(title);
    b->setToolTip(title);
    b->setProperty("primary", primary);
    b->setMinimumHeight(36);
    b->setCursor(Qt::PointingHandCursor);
    return b;
```

```
}
void configureTable(QTableView *table) {
    table->setSelectionBehavior(QAbstractItemView::SelectRows);
    table->setSelectionMode(QAbstractItemView::SingleSelection);
    table->setEditTriggers(QAbstractItemView::NoEditTriggers);
    table->setAlternatingRowColors(true);
    table->horizontalHeader()->setSectionResizeMode(QHeaderView::Stretch);
    table->verticalHeader()->setVisible(false);
    table->verticalHeader()->setDefaultSectionSize(38);
    table->setShowGrid(false);
    table->setSortingEnabled(false);
    new EmptyState(table);
    auto *open = new QShortcut(QKeySequence(Qt::Key_Return), table);
    open->setContext(Qt::WidgetWithChildrenShortcut);
    QObject::connect(open, &QShortcut::activated, table, [table] {
        if (table->currentIndex().isValid())
            emit table->doubleClicked(table->currentIndex());
    });
    table->setAccessibleName(
        "Records table. Use arrow keys to select a row and Enter to open details.");
    table->setWordWrap(true);
    table->verticalHeader()->setSectionResizeMode(QHeaderView::ResizeToContents);
}
QTableWidget *makeTable(const QStringList &headers, const QString &name) {
    auto *table = new QTableWidget(0, headers.size());
    table->setHorizontalHeaderLabels(headers);
    table->setObjectName(name);
    configureTable(table);
    return table;
}
QString selectedId(QTableWidget *table) {
    const int row = table->currentRow();
    return row >= 0 && table->item(row, 0) ? table->item(row, 0)->text() : QString();
}
void selectId(QTableWidget *table, const QString &id) {
    for (int i = 0; i < table->rowCount(); ++i)
        if (table->item(i, 0)->text() == id) {
            table->selectRow(i);
            return;
        }
}
void fillTable(QTableWidget *table, const QList<QStringList> &rows) {
    const auto selected = selectedId(table);
    const auto scroll = table->verticalScrollBar()->value();
    table->setAccessibleDescription(rows.isEmpty() ? "No records match this view."
                                                   : QString("%1 records").arg(rows.size()));
    table->setToolTip(rows.isEmpty() ? "No records match this view. Adjust filters or add a record.
  > "
                                     : "");
    table->setRowCount(rows.size());
    for (int r = 0; r < rows.size(); ++r)
        for (int c = 0; c < rows[r].size(); ++c) {
            auto *item = new QTableWidgetItem(rows[r][c]);
            table->setItem(r, c, item);
            if (rows[r][c] == "Cancelled" || rows[r][c] == "Emergency" || rows[r][c] == "Archived")
                item->setForeground(QColor("#b43d34"));
            if (rows[r][c] == "Completed")
                item->setForeground(QColor("#147368"));
        }
    selectId(table, selected);
    table->verticalScrollBar()->setValue(scroll);
}
void fillPatientChoices(QComboBox *combo, const model::ClinicDatabase &db) {
    const auto selected = combo->currentData().toString();
    combo->clear();
    for (const auto &p : db.patients)
        if (!db.archivedPatients.contains(p->id()))
            combo->addItem(p->details().name + " / " + p->id(), p->id());
    const int index = combo->findData(selected);
    if (index >= 0)
        combo->setCurrentIndex(index);
}
void showPatientHistory(QWidget *parent, controller::ClinicManager &manager, const QString &id) {
    const auto *patient = manager.database().patient(id);
    if (!patient) {
```

```
143         QMessageBox::information(parent, "Patient history", "Select a patient first.");
144         return;
145     }
146     manager.access(id);
147     QDialog dialog(parent);
148     dialog.setObjectName("historyDialog");
149     dialog.setWindowTitle("Patient history - " + patient->details().name);
150     dialog.resize(950, 570);
151     auto *layout = new QVBoxLayout(&dialog);
152     auto *heading =
153         new QLabel(patient->details().name + " / " + id +
154                    (manager.database().archivedPatients.contains(id) ? " / Archived" : ""));
155     heading->setObjectName("sectionTitle");
156     layout->addWidget(heading);
157     auto *summary =
158         new QLabel(manager.canViewClinicalNotes()
159                        ? model::processPatient(*patient)
160                        : "Clinical details are available to clinicians and administrators.");
161     summary->setWordWrap(true);
162     layout->addWidget(summary);
163     auto *table = makeTable(
164         {"Appointment", "Date / time", "Service", "Status", "Consultation notes"}, "historyTable");
165     QList<QStringList> rows;
166     for (const auto &a : manager.database().appointments)
167         if (a.patientId == id)
168             rows.append({a.id, a.start.toLocalTime().toString("dd MMM yyyy HH:mm"), a.service,
169                          model::statusName(a.status),
170                          manager.canViewClinicalNotes() ? a.consultationNotes : "Restricted"});
171     fillTable(table, rows);
172     layout->addWidget(table);
173     auto *notes = new QTextEdit;
174     notes->setReadOnly(true);
175     notes->setMaximumHeight(140);
176     notes->setPlaceholderText("Select a visit to read its complete notes.");
177     layout->addWidget(notes);
178     QObject::connect(table, &QTableWidget::itemSelectionChanged, &dialog, [&] {
179         const auto *a = manager.database().appointment(selectedId(table));
180         notes->setPlainText(a && manager.canViewClinicalNotes()
181                                 ? a->consultationNotes + "\n\nChange reason: " + a->changeReason +
182                                       "\n\nCorrection history:\n" + a->corrections.join("\n\n")
183                                 : "Clinical notes restricted.");
184     });
185     auto *close = new QDialogButtonBox(QDialogButtonBox::Close);
186     layout->addWidget(close);
187     QObject::connect(close, &QDialogButtonBox::rejected, &dialog, &QDialog::reject);
188     dialog.exec();
189 }
190 } // namespace clinic::gui
```

5.43 SOURCE OR RESOURCE FILE

src/gui/panels/PatientsPanel.cpp

```
#include "PatientsPanel.h"
#include "gui/PatientDialog.h"
namespace clinic::gui {
PatientsPanel::PatientsPanel(controller::ClinicManager &manager, MutationHandler mutation,
                             QWidget *parent)
    : PanelBase(manager, std::move(mutation), parent) {
    auto *layout = panelLayout(
        this, "Patient records",
        "Review history, update details, and archive records without losing previous visits.");
    auto *row = new QHBoxLayout;
    auto *add = actionButton("+ Register patient", "addPatient", true);
    auto *edit = actionButton("Edit selected", "editPatient");
    auto *history = actionButton("View history", "patientHistory");
    auto *archive = actionButton("Archive selected", "deletePatient");
    auto *restore = actionButton("Restore selected", "restorePatient");
    archived_ = new QCheckBox("Include archived");
    archived_->setObjectName("includeArchived");
    for (auto *b : {add, edit, history, archive, restore})
        row->addWidget(b);
    row->addWidget(archived_);
    layout->addLayout(row);
    model_ = new PatientTableModel(this);
    table_ = new QTableView;
    table_->setObjectName("patientsTable");
    table_->setModel(model_);
    configureTable(table_);
    layout->addWidget(table_, 1);
    details_ = new QLabel("Select a record to see details.");
    details_->setWordWrap(true);
    details_->setMinimumHeight(50);
    layout->addWidget(details_);
    connect(add, &QPushButton::clicked, this, [this] { editPatient(false); });
    connect(edit, &QPushButton::clicked, this, [this] { editPatient(true); });
    connect(history, &QPushButton::clicked, this,
            [this] { showPatientHistory(this, manager_, selection()); });
    connect(table_, &QTableView::doubleClicked, this,
            [this] { showPatientHistory(this, manager_, selection()); });
    connect(archived_, &QCheckBox::toggled, this, [this] {
        beforeChange();
        refresh();
    });
    connect(archive, &QPushButton::clicked, this, [this] {
        const auto id = selection();
        if (id.isEmpty())
            return;
        const auto *p = manager_.database().patient(id);
        if (QMessageBox::question(this, "Archive patient",
                                  "Archive " + p->details().name +
                                      "? Active bookings will be cancelled; completed history is "
                                      "retained. Undo is available.") == QMessageBox::Yes)
            mutate_([&] { manager_.deletePatient(id); }, "Patient archived.");
    });
    connect(restore, &QPushButton::clicked, this, [this] {
        const auto id = selection();
        mutate_([&] { manager_.restorePatient(id); },
                "Patient restored. Cancelled appointments stay cancelled.");
    });
    connect(table_->selectionModel(), &QItemSelectionModel::selectionChanged, this, [this] {
        if (updating_)
            return;
        selected_ = selection();
        const auto *p = manager_.database().patient(selected_);
        if (p) {
            details_->setText(
                p->details().name + " / " +
                (manager_.database().archivedPatients.contains(p->id()) ? "Archived / " : "") +
                (manager_.canViewClinicalNotes() ? model::processPatient(*p)
                                                 : "Clinical details restricted."));
            manager_.access(p->id());
```

```
        }
    });
}
QString PatientsPanel::selection() const {
    return model_->idAt(table_->currentIndex().row());
}
void PatientsPanel::beforeChange() {
    selected_ = selection();
    scroll_ = table_->verticalScrollBar()->value();
    updating_ = true;
    model_->setPatients({});
}
void PatientsPanel::refresh() {
    model_->setClinicalAccess(manager_.canViewClinicalNotes());
    updating_ = true;
    algorithms::PatientView view;
    for (const auto &p : manager_.database().patients)
        if (archived_->isChecked() || !manager_.database().archivedPatients.contains(p->id()))
            view.push_back(p.get());
    model_->setPatients(std::move(view));
    for (int i = 0; i < model_->rowCount(); ++i)
        if (model_->idAt(i) == selected_)
            table_->selectRow(i);
    table_->verticalScrollBar()->setValue(scroll_);
    const auto *p = manager_.database().patient(selected_);
    details_->setText(p ? p->details().name + " / " +
                              (manager_.canViewClinicalNotes() ? model::processPatient(*p)
                                                               : "Clinical details restricted.")
                        : "Select a record to see details.");
    for (const auto &name : {"addPatient", "editPatient"})
        findChild<QPushButton *>(name)->setEnabled(manager_.canEdit());
    for (const auto &name : {"deletePatient", "restorePatient"})
        findChild<QPushButton *>(name)->setEnabled(manager_.isAdmin());
    updating_ = false;
}
void PatientsPanel::editPatient(bool edit) {
    const auto id = selection();
    const auto *p = edit ? manager_.database().patient(id) : nullptr;
    if (edit && !p) {
        QMessageBox::information(this, "Edit patient", "Select a patient first.");
        return;
    }
    PatientDialog dialog(manager_.suggestedPatientId(), p, this);
    dialog.configureAccess(manager_.canViewClinicalNotes(), manager_.database());
    if (dialog.exec() == QDialog::Accepted) {
        const auto d = dialog.details();
        const auto category = dialog.category();
        const auto matches = manager_.possibleDuplicates(d, edit ? id : QString());
        if (!matches.isEmpty() &&
            QMessageBox::question(this, "Possible duplicate patient",
                                  "These records share the name and age, or contact:\n" +
                                      matches.join("\n") + "\nContinue with this record?",
                                  QMessageBox::Yes | QMessageBox::No,
                                  QMessageBox::No) != QMessageBox::Yes)
            return;
        mutate_(
            [&] {
                if (edit)
                    manager_.editPatient(id, category, d);
                else
                    manager_.addPatient(category, d);
            },
            "Patient record saved.");
    }
}
} // namespace clinic::gui
```

5.44 SOURCE OR RESOURCE FILE

src/gui/panels/QueuePanel.cpp

```
#include "QueuePanel.h"
namespace clinic::gui {
QueuePanel::QueuePanel(controller::ClinicManager &m, MutationHandler f, QWidget *parent)
    : PanelBase(m, std::move(f), parent) {
    auto *layout = panelLayout(
        this, "Waiting queue",
        "FIFO order with appointment links and live waiting times. Walk-ins may also join.");
    auto *row = new QHBoxLayout;
    patient_ = new QComboBox;
    patient_->setObjectName("queuePatient");
    row->addWidget(patient_, 1);
    auto *add = actionButton("Add walk-in", "enqueuePatient", true);
    auto *next = actionButton("Call next patient", "callNext", true);
    auto *remove = actionButton("Remove selected", "removeQueued");
    for (auto *b : {add, next, remove})
        row->addWidget(b);
    layout->addLayout(row);
    table_ = makeTable({"Patient ID", "Patient", "Category", "Appointment", "Waiting", "Position"},
                       "queueTable");
    layout->addWidget(table_, 1);
    lastCalled_ = new QLabel;
    lastCalled_->setObjectName("lastCalled");
    lastCalled_->setWordWrap(true);
    layout->addWidget(lastCalled_);
    resetMessage();
    connect(add, &QPushButton::clicked, this, [this] {
        const auto id = patient_->currentData().toString();
        mutate_([&] { manager_.enqueue(id); }, "Walk-in added.");
    });
    connect(next, &QPushButton::clicked, this, [this] {
        QString summary;
        mutate_([&] { summary = manager_.callNext(); }, "Patient called.");
        if (!summary.isEmpty())
            lastCalled_->setText("LAST CALLED\n" + summary);
    });
    connect(remove, &QPushButton::clicked, this, [this] {
        const auto id = selectedId(table_);
        mutate_([&] { manager_.removeFromQueue(id); },
                "Removed from queue. Linked appointment returned to Scheduled.");
    });
}
void QueuePanel::refresh() {
    fillPatientChoices(patient_, manager_.database());
    auto queue = manager_.database().waiting;
    QList<QStringList> rows;
    int position = 1;
    while (!queue.empty()) {
        const auto &e = queue.front();
        const auto *p = manager_.database().patient(e.patientId);
        const auto minutes = qMax<qint64>(0, e.arrivedAt.secsTo(QDateTime::currentDateTime()) / 60)
 > ;
        rows.append({e.patientId, p->details().name, model::categoryName(p->category()),
                     e.appointmentId.isEmpty() ? "Walk-in" : e.appointmentId,
                     QString::number(minutes) + " min", QString::number(position++)});
        queue.pop();
    }
    fillTable(table_, rows);
    findChild<QPushButton *>("callNext")->setEnabled(manager_.canViewClinicalNotes());
    for (const auto &name : {"enqueuePatient", "removeQueued"})
        findChild<QPushButton *>(name)->setEnabled(manager_.canEdit());
}
void QueuePanel::resetMessage() {
    lastCalled_->setText("Select Call next to begin the next linked consultation.");
}
} // namespace clinic::gui
```

5.45 SOURCE OR RESOURCE FILE

src/gui/panels/ReportsPanel.cpp

```
#include "ReportsPanel.h"
#include "persistence/FileHandler.h"
#include <QPrintDialog>
#include <QPrinter>
namespace clinic::gui {
ReportsPanel::ReportsPanel(controller::ClinicManager &m, MutationHandler f, QWidget *parent)
    : PanelBase(m, std::move(f), parent) {
    auto *layout = panelLayout(
        this, "Clinic reports",
        "Filter appointment reports by date range, review totals, and export or print.");
    auto *row = new QHBoxLayout;
    type_ = new QComboBox;
    type_->setObjectName("reportType");
    type_->addItems({"Most frequently attended patients", "Patients with pending appointments",
                     "Patient category distribution", "Appointment status summary"});
    from_ = new QDateEdit(QDate::currentDate().addDays(-30));
    from_->setObjectName("reportDate");
    from_->setCalendarPopup(true);
    to_ = new QDateEdit(QDate::currentDate());
    to_->setObjectName("reportEndDate");
    to_->setCalendarPopup(true);
    for (auto *field : {from_, to_})
        field->setDisplayFormat("dd MMM yyyy");
    allDates_ = new QCheckBox("All dates");
    allDates_->setObjectName("reportAllDates");
    allDates_->setChecked(true);
    row->addWidget(type_, 1);
    row->addWidget(from_);
    row->addWidget(new QLabel("to"));
    row->addWidget(to_);
    row->addWidget(allDates_);
    layout->addLayout(row);
    auto *actions = new QHBoxLayout;
    auto *csv = actionButton("Export CSV", "exportReport");
    auto *pdf = actionButton("Export PDF", "exportReportPdf");
    auto *printer = actionButton("Print", "printReport");
    actions->addWidget(csv);
    actions->addWidget(pdf);
    actions->addWidget(printer);
    actions->addStretch();
    layout->addLayout(actions);
    title_ = new QLabel;
    title_->setObjectName("sectionTitle");
    summary_ = new QLabel;
    summary_->setObjectName("reportSummary");
    summary_->setWordWrap(true);
    layout->addWidget(title_);
    layout->addWidget(summary_);
    table_ = makeTable({}, "reportsTable");
    layout->addWidget(table_, 1);
    connect(type_, &QComboBox::currentIndexChanged, this, [this] { refresh(); });
    connect(from_, &QDateEdit::dateChanged, this, [this] { refresh(); });
    connect(to_, &QDateEdit::dateChanged, this, [this] { refresh(); });
    connect(allDates_, &QCheckBox::toggled, this, [this] { refresh(); });
    connect(csv, &QPushButton::clicked, this, [this] {
        if (!manager_.canExportReports())
            return;
        const auto path = QFileDialog::getSaveFileName(this, "Export clinic report",
                                                       "clinic-report.csv", "CSV (*.csv)");
        if (path.isEmpty())
            return;
        try {
            manager_.requireReportExport();
            persistence::FileHandler::writeText(path, controller::reportCsv(current_));
            manager_.recordExport("Export CSV report");
            QMessageBox::information(this, "Report exported", "Saved CSV report to " + path);
        } catch (const std::exception &e) {
            QMessageBox::warning(this, "Export failed", e.what());
        }
    });
```

```
    connect(pdf, &QPushButton::clicked, this, &ReportsPanel::exportPdf);
    connect(printer, &QPushButton::clicked, this, &ReportsPanel::print);
}
void ReportsPanel::refresh() {
    const bool dated = !allDates_->isChecked() && type_->currentIndex() != 2;
    from_->setEnabled(dated);
    to_->setEnabled(dated);
    try {
        current_ = controller::generateReport(
            manager_.database(), static_cast<controller::ReportType>(type_->currentIndex()),
            dated ? from_->date() : QDate{}, dated ? to_->date() : QDate{});
        title_->setText(current_.title);
        summary_->setText(current_.summary);
        table_->setColumnCount(current_.headers.size());
        table_->setHorizontalHeaderLabels(current_.headers);
        fillTable(table_, current_.rows);
        for (const auto &n : {"exportReport", "exportReportPdf", "printReport"})
            findChild<QPushButton *>(n)->setEnabled(manager_.canExportReports());
    } catch (const std::exception &e) {
        summary_->setText(e.what());
        table_->setRowCount(0);
        for (const auto &n : {"exportReport", "exportReportPdf", "printReport"})
            findChild<QPushButton *>(n)->setEnabled(false);
    }
}
void ReportsPanel::exportPdf() {
    if (!manager_.canExportReports())
        return;
    auto file = QFileDialog::getSaveFileName(this, "Export printable report", "clinic-report.pdf",
                                             "PDF (*.pdf)");
    if (file.isEmpty())
        return;
    if (!file.endsWith(".pdf", Qt::CaseInsensitive))
        file += ".pdf";
    QPrinter printer(QPrinter::HighResolution);
    printer.setOutputFormat(QPrinter::PdfFormat);
    QTemporaryDir temporary;
    if (!temporary.isValid()) {
        QMessageBox::warning(this, "Export failed", "Cannot create temporary report.");
        return;
    }
    const auto rendered = temporary.filePath("report.pdf");
    printer.setOutputFileName(rendered);
    printer.setPageOrientation(QPageLayout::Landscape);
    QTextDocument document;
    document.setHtml(controller::reportHtml(current_));
    document.print(&printer);
    QFile input(rendered);
    QSaveFile output(file);
    if (!input.open(QIODevice::ReadOnly) || input.size() == 0 ||
        !output.open(QIODevice::WriteOnly)) {
        QMessageBox::warning(this, "Export failed", "The PDF could not be written.");
        return;
    }
    const auto bytes = input.readAll();
    if (output.write(bytes) != bytes.size() || !output.commit()) {
        QMessageBox::warning(this, "Export failed", "The PDF could not be committed.");
        return;
    }
    manager_.recordExport("Export PDF report");
    QMessageBox::information(this, "Report exported", "Saved PDF report to " + file);
}
void ReportsPanel::print() {
    if (!manager_.canExportReports())
        return;
    QPrinter printer(QPrinter::HighResolution);
    printer.setPageOrientation(QPageLayout::Landscape);
    QPrintDialog dialog(&printer, this);
    if (dialog.exec() == QDialog::Accepted) {
        QTextDocument document;
        document.setHtml(controller::reportHtml(current_));
        document.print(&printer);
        manager_.recordExport("Print report");
    }
}
} // namespace clinic::gui
```

5.46 SOURCE OR RESOURCE FILE

src/gui/panels/SearchPanel.cpp

```
#include "SearchPanel.h"
#include <QElapsedTimer>
#include "algorithms/Benchmark.h"
#include <future>
#include <chrono>
namespace clinic::gui {
using namespace algorithms;
SearchPanel::SearchPanel(controller::ClinicManager &m, MutationHandler f, QWidget *parent)
    : PanelBase(m, std::move(f), parent) {
    auto *layout = panelLayout(this, "Search & sort",
                               "Choose a manual algorithm. Timings include the work described; "
                               "results survive record updates.");
    auto *row = new QHBoxLayout;
    field_ = new QComboBox;
    field_->setObjectName("searchField");
    field_->addItems({"Patient ID", "Patient name", "Category"});
    algorithm_ = new QComboBox;
    algorithm_->setObjectName("searchAlgorithm");
    algorithm_->addItems({"Linear search", "Binary search (ID)"});
    text_ = new QLineEdit;
    text_->setObjectName("searchText");
    text_->setPlaceholderText("ID, name or category");
    auto *search = actionButton("Search", "searchButton", true);
    auto *reset = actionButton("Show all", "resetSearch");
    row->addWidget(field_);
    row->addWidget(algorithm_);
    row->addWidget(text_, 1);
    row->addWidget(search);
    row->addWidget(reset);
    layout->addLayout(row);
    auto *sortRow = new QHBoxLayout;
    sortField_ = new QComboBox;
    sortField_->setObjectName("sortField");
    sortField_->addItems({"Patient name", "Patient ID", "Age"});
    sortAlgorithm_ = new QComboBox;
    sortAlgorithm_->setObjectName("sortAlgorithm");
    sortAlgorithm_->addItems({"Insertion sort", "Merge sort (recursive)"});
    auto *sort = actionButton("Sort all records", "sortButton");
    auto *compare = actionButton("Compare algorithms", "benchmarkButton");
    archived_ = new QCheckBox("Include archived");
    archived_->setObjectName("searchArchived");
    sortRow->addWidget(sortField_);
    sortRow->addWidget(sortAlgorithm_);
    sortRow->addWidget(sort);
    sortRow->addWidget(compare);
    sortRow->addWidget(archived_);
    layout->addLayout(sortRow);
    info_ = new QLabel;
    info_->setObjectName("searchInfo");
    info_->setWordWrap(true);
    info_->setMinimumHeight(45);
    layout->addWidget(info_);
    model_ = new PatientTableModel(this);
    table_ = new QTableView;
    table_->setObjectName("searchResults");
    table_->setModel(model_);
    configureTable(table_);
    layout->addWidget(table_, 1);
    auto execute = [this] {
        if (text_->text().trimmed().isEmpty()) {
            QMessageBox::warning(this, "Search", "Enter an ID, name or category.");
            return;
        }
        committedQuery_ = text_->text().trimmed();
        committedField_ = field_->currentIndex();
        committedAlgorithm_ = algorithm_->currentIndex();
        mode_ = ViewMode::Search;
        rebuild();
    };
```

```
    connect(search, &QPushButton::clicked, this, execute);
    connect(text_, &QLineEdit::returnPressed, this, execute);
    connect(sort, &QPushButton::clicked, this, [this] {
        committedSortField_ = sortField_->currentIndex();
        committedSortAlgorithm_ = sortAlgorithm_->currentIndex();
        mode_ = ViewMode::Sort;
        rebuild();
    });
    connect(reset, &QPushButton::clicked, this, [this] {
        mode_ = ViewMode::All;
        text_->clear();
        rebuild();
    });
    connect(archived_, &QCheckBox::toggled, this, [this] { rebuild(); });
    connect(field_, &QComboBox::currentIndexChanged, this, [this](int index) {
        if (index != 0)
            algorithm_->setCurrentIndex(0);
        algorithm_->setEnabled(index == 0);
    });
    connect(compare, &QPushButton::clicked, this, &SearchPanel::benchmark);
    connect(table_, &QTableView::clicked, this, [this](const QModelIndex &index) {
        selected_ = model_->idAt(index.row());
        manager_.access(selected_);
    });
    connect(table_, &QTableView::doubleClicked, this, [this](const QModelIndex &index) {
        showPatientHistory(this, manager_, model_->idAt(index.row()));
    });
}
void SearchPanel::beforeChange() {
    selected_ = model_->idAt(table_->currentIndex().row());
    scroll_ = table_->verticalScrollBar()->value();
    model_->setPatients({});
}
void SearchPanel::refresh() {
    rebuild();
}
void SearchPanel::rebuild() {
    model_->setClinicalAccess(manager_.canViewClinicalNotes());
    PatientView view;
    for (const auto &p : manager_.database().patients)
        if (archived_->isChecked() || !manager_.database().archivedPatients.contains(p->id()))
            view.push_back(p.get());
    Metrics metrics, preparation;
    QElapsedTimer timer;
    timer.start();
    QString explanation = "All active patient records.";
    if (mode_ == ViewMode::Search) {
        if (committedField_ == 0) {
            const model::PatientRecord *found = nullptr;
            if (committedAlgorithm_ == 1) {
                sortPatients(view, SortField::Id, SortAlgorithm::Merge, &preparation);
                found = binarySearchById(view, committedQuery_, &metrics);
                explanation = QString("Binary search: %1 comparisons; manual ID sort: %2 "
                                      "comparisons. Preparation O(n log n), lookup O(log n).")
                                  .arg(metrics.comparisons)
                                  .arg(preparation.comparisons);
            } else {
                for (const auto *p : view) {
                    ++metrics.comparisons;
                    if (p->id() == committedQuery_) {
                        found = p;
                        break;
                    }
                }
                explanation =
                    QString("Linear ID search: %1 comparisons; O(n).").arg(metrics.comparisons);
            }
            view.clear();
            if (found)
                view.push_back(found);
        } else {
            view = filterPatients(view, committedQuery_, committedField_ == 2);
            explanation =
                QString("Case-insensitive linear substring scan: %1 matches.").arg(view.size());
        }
```

```
145     } else if (mode_ == ViewMode::Sort) {
146         sortPatients(view, static_cast<SortField>(committedSortField_),
147                      static_cast<SortAlgorithm>(committedSortAlgorithm_), &metrics);
148         explanation =
149             QString("%1 | %2 comparisons / %3 moves / recursion depth %4.")
150                 .arg(committedSortAlgorithm_ == 0
151                          ? "Insertion sort: small or nearly sorted lists; worst case O(n^2)."
152                          : "Recursive merge sort: stable O(n log n); O(n) temporary space.")
153                 .arg(metrics.comparisons)
154                 .arg(metrics.moves)
155                 .arg(metrics.maxRecursionDepth);
156     }
157     const double elapsed = timer.nsecsElapsed() / 1000000.0;
158     info_->setText(explanation +
159                    QString("\n%1 ms including any sort preparation; %2 displayed records.")
160                        .arg(elapsed, 0, 'f', 3)
161                        .arg(view.size()));
162     model_->setPatients(std::move(view));
163     for (int i = 0; i < model_->rowCount(); ++i)
164         if (model_->idAt(i) == selected_)
165             table_->selectRow(i);
166     table_->verticalScrollBar()->setValue(scroll_);
167 }
168 void SearchPanel::benchmark() {
169     auto database = manager_.database();
170     if (database.patients.empty() || database.patients.size() > 10000) {
171         QMessageBox::information(
172             this, "Benchmark",
173             "Use 1-10,000 records. Load benchmark-2000.json to compare a larger dataset.");
174         return;
175     }
176     const auto field = static_cast<SortField>(sortField_->currentIndex());
177     QProgressDialog progress(
178         "Comparing three input orders with five measured runs per algorithm...", QString(), 0, 0,
179         this);
180     progress.setWindowModality(Qt::ApplicationModal);
181     progress.setMinimumDuration(0);
182     progress.show();
183     auto result = std::async(std::launch::async, [db = std::move(database), field] {
184         return benchmarkSorts(viewOf(db.patients), field, 5);
185     });
186     while (result.wait_for(std::chrono::milliseconds(20)) != std::future_status::ready)
187         QApplication::processEvents();
188     progress.close();
189     std::vector<BenchmarkResult> results;
190     try {
191         results = result.get();
192     } catch (const std::exception &e) {
193         QMessageBox::warning(this, "Benchmark failed", e.what());
194         return;
195     }
196     QDialog dialog(this);
197     dialog.setObjectName("benchmarkDialog");
198     dialog.setWindowTitle("Repeated sorting comparison");
199     dialog.resize(1000, 450);
200     auto *layout = new QVBoxLayout(&dialog);
201     auto *table = makeTable({"Input order", "Algorithm", "Records", "Median ms (5 runs)",
202                              "Comparisons", "Moves", "Depth"},
203                             "benchmarkTable");
204     QList<QStringList> rows;
205     for (const auto &r : results)
206         rows.append(
207             {r.order,
208              r.algorithm == SortAlgorithm::Insertion ? "Insertion sort" : "Recursive merge sort",
209              QString::number(r.records), QString::number(r.medianMs, 'f', 3),
210              QString::number(r.metrics.comparisons), QString::number(r.metrics.moves),
211              QString::number(r.metrics.maxRecursionDepth)});
212     fillTable(table, rows);
213     layout->addWidget(table);
214     auto *note =
215         new QLabel("Identical input per algorithm: sorted, reversed and seeded shuffled. One "
216                    "warm-up is excluded; five timed runs determine each median. Input copying is "
217                    "excluded. Counts are deterministic; timings depend on this computer.");
218     note->setWordWrap(true);
219     layout->addWidget(note);
```

```
220     auto *close = new QDialogButtonBox(QDialogButtonBox::Close);
221     layout->addWidget(close);
222     connect(close, &QDialogButtonBox::rejected, &dialog, &QDialog::reject);
223     dialog.exec();
224 }
225 } // namespace clinic::gui
```

5.47 SOURCE OR RESOURCE FILE

src/gui/PatientDialog.cpp

```
#include "PatientDialog.h"
#include <QtWidgets>
namespace clinic::gui {
PatientDialog::PatientDialog(const QString &suggestedId, const model::PatientRecord *patient,
                             QWidget *parent)
    : QDialog(parent) {
    setObjectName("patientDialog");
    setWindowTitle(patient ? "Edit patient" : "Register patient");
    setMinimumWidth(500);
    auto *layout = new QVBoxLayout(this);
    auto *title = new QLabel(patient ? "Update patient record" : "Register a new patient");
    title->setObjectName("sectionTitle");
    layout->addWidget(title);
    auto *subtitle = new QLabel("All visible fields are required.");
    layout->addWidget(subtitle);
    auto *form = new QFormLayout;
    form->setSpacing(14);
    id_ = new QLineEdit(suggestedId);
    id_->setObjectName("patientId");
    name_ = new QLineEdit;
    name_->setObjectName("patientName");
    name_->setMaxLength(120);
    age_ = new QSpinBox;
    age_->setRange(0, 120);
    age_->setObjectName("patientAge");
    contact_ = new QLineEdit;
    contact_->setObjectName("patientContact");
    contact_->setMaxLength(160);
    reason_ = new QLineEdit;
    reason_->setObjectName("patientReason");
    reason_->setMaxLength(1000);
    category_ = new QComboBox;
    category_->setObjectName("patientCategory");
    category_->addItems({"General", "Emergency", "Follow-up"});
    urgency_ = new QSpinBox;
    urgency_->setRange(1, 5);
    urgency_->setObjectName("patientUrgency");
    previous_ = new QLineEdit;
    previous_->setObjectName("previousVisit");
    previous_->setMaxLength(120);
    urgencyLabel_ = new QLabel("Urgency (5 = highest)");
    previousLabel_ = new QLabel("Previous visit reference");
    form->addRow("Patient ID", id_);
    form->addRow("Full name", name_);
    form->addRow("Age", age_);
    form->addRow("Contact", contact_);
    form->addRow("Patient category", category_);
    form->addRow("Reason for visit", reason_);
    form->addRow(urgencyLabel_, urgency_);
    form->addRow(previousLabel_, previous_);
    layout->addLayout(form);
    error_ = new QLabel;
    error_->setObjectName("formError");
    error_->setStyleSheet("color:#b43d34");
    error_->setWordWrap(true);
    layout->addWidget(error_);
    auto *buttons = new QDialogButtonBox(QDialogButtonBox::Save | QDialogButtonBox::Cancel);
    buttons->setObjectName("patientButtons");
    layout->addWidget(buttons);
    connect(buttons, &QDialogButtonBox::accepted, this, [this] {
        try {
            auto valid = model::makePatient(category(), details());
            Q_UNUSED(valid);
            accept();
        } catch (const std::exception &e) {
            error_->setText(QString::fromUtf8(e.what()));
        }
    });
    connect(buttons, &QDialogButtonBox::rejected, this, &QDialog::reject);
    connect(category_, &QComboBox::currentIndexChanged, this, [this] { updateFields(); });
```

```
    if (patient) {
        const auto &d = patient->details();
        id_->setText(d.id);
        id_->setReadOnly(true);
        name_->setText(d.name);
        age_->setValue(d.age);
        contact_->setText(d.contact);
        reason_->setText(d.reason);
        urgency_->setValue(d.urgency);
        previous_->setText(d.previousVisit);
        category_->setCurrentIndex(static_cast<int>(patient->category()));
    }
    updateFields();
}
void PatientDialog::updateFields() {
    const bool emergency = clinicalAccess_ && category_->currentIndex() == 1,
               followUp = clinicalAccess_ && category_->currentIndex() == 2;
    if (previousChoices_)
        previousChoices_->setVisible(followUp);
    urgency_->setVisible(emergency);
    urgencyLabel_->setVisible(emergency);
    previous_->setVisible(followUp);
    previousLabel_->setVisible(followUp);
}
model::PatientDetails PatientDialog::details() const {
    return {id_->text(),     name_->text(),     age_->value(),    contact_->text(),
            reason_->text(), urgency_->value(), previous_->text()};
}
model::PatientCategory PatientDialog::category() const {
    return static_cast<model::PatientCategory>(category_->currentIndex());
}
void PatientDialog::configureAccess(bool allowed, const model::ClinicDatabase &db) {
    clinicalAccess_ = allowed;
    auto *form = findChild<QFormLayout *>();
    for (auto *field : {static_cast<QWidget *>(reason_), static_cast<QWidget *>(category_)}) {
        field->setVisible(allowed);
        if (auto *label = form->labelForField(field))
            label->setVisible(allowed);
    }
    if (!allowed) {
        if (reason_->text().isEmpty())
            reason_->setText("Awaiting clinician assessment");
        setWindowTitle("Patient demographics");
    } else {
        previous_->setReadOnly(true);
        previous_->setPlaceholderText("Choose a completed visit below");
        previousChoices_ = new QComboBox;
        previousChoices_->setObjectName("previousVisitChoices");
        previousChoices_->addItem("Choose a completed visit", {});
        if (!previous_->text().isEmpty())
            previousChoices_->addItem("Existing reference: " + previous_->text(),
                                      previous_->text());
        for (const auto &visit : db.appointments)
            if (visit.patientId == id_->text() &&
                visit.status == model::AppointmentStatus::Completed)
                previousChoices_->addItem(visit.id + " / " + visit.start.toString("dd MMM yyyy") +
                                          " / " + visit.service,
                                          visit.id);
        auto selected = previousChoices_->findData(previous_->text());
        if (selected >= 0)
            previousChoices_->setCurrentIndex(selected);
        form->addRow(previousChoices_);
        connect(previousChoices_, &QComboBox::currentIndexChanged, this,
                [this] { previous_->setText(previousChoices_->currentData().toString()); });
        if (previousChoices_->count() == 1) {
            auto *items = qobject_cast<QStandardItemModel *>(category_->model());
            if (items)
                items->item(2)->setEnabled(false);
            previous_->setToolTip("Register first, complete a visit, then select Follow-up when "
                                  "editing this patient.");
        }
    }
    updateFields();
}
} // namespace clinic::gui
```

5.48 SOURCE OR RESOURCE FILE

src/gui/PatientTableModel.cpp

```
#include "PatientTableModel.h"
#include <QColor>
namespace clinic::gui {
int PatientTableModel::rowCount(const QModelIndex &parent) const {
    return parent.isValid() ? 0 : static_cast<int>(patients_.size());
}
int PatientTableModel::columnCount(const QModelIndex &parent) const {
    return parent.isValid() ? 0 : 5;
}
QVariant PatientTableModel::data(const QModelIndex &index, int role) const {
    if (!index.isValid() || index.row() < 0 || index.row() >= rowCount())
        return {};
    const auto *p = patients_[static_cast<size_t>(index.row())];
    if (role == Qt::ToolTipRole)
        return clinicalAccess_ ? model::processPatient(*p) : QString("Clinical details restricted")
 > ;
    if (role == Qt::ForegroundRole && index.column() == 3) {
        if (p->category() == model::PatientCategory::Emergency)
            return QColor("#b43d34");
        if (p->category() == model::PatientCategory::FollowUp)
            return QColor("#6652a3");
        return QColor("#147368");
    }
    if (role != Qt::DisplayRole)
        return {};
    switch (index.column()) {
    case 0:
        return p->id();
    case 1:
        return p->details().name;
    case 2:
        return p->details().age;
    case 3:
        return model::categoryName(p->category());
    case 4:
        return p->details().contact;
    }
    return {};
}
QVariant PatientTableModel::headerData(int section, Qt::Orientation orientation, int role) const {
    if (role != Qt::DisplayRole)
        return {};
    if (orientation == Qt::Vertical)
        return section + 1;
    const QStringList headers{"Patient ID", "Full name", "Age", "Category", "Contact"};
    if (section >= 0 && section < headers.size())
        return headers[section];
    return {};
}
void PatientTableModel::setPatients(algorithms::PatientView patients) {
    beginResetModel();
    patients_ = std::move(patients);
    endResetModel();
}
QString PatientTableModel::idAt(int row) const {
    return row >= 0 && row < rowCount() ? patients_[static_cast<size_t>(row)]->id() : QString();
}
} // namespace clinic::gui
```

5.49 SOURCE OR RESOURCE FILE

src/main.cpp

```
#include "gui/MainWindow.h"
#include <QApplication>
#include <QCommandLineParser>
#include <QMessageBox>
#include <QTimer>
#include <cstdio>
int main(int argc, char *argv[]) {
    QApplication app(argc, argv);
    QApplication::setOrganizationName("MivaStudy");
    QApplication::setApplicationName("SCAPMS");
    QApplication::setApplicationVersion("3.0");
    QCommandLineParser parser;
    parser.setApplicationDescription("Smart Clinic Appointment & Patient Management System");
    parser.addHelpOption();
    parser.addVersionOption();
    parser.addOption({{"d", "data"}, "Load a JSON workspace.", "file"});
    parser.addOption({"screenshots", "Capture all panels and exit.", "directory"});
    parser.process(app);
    clinic::gui::MainWindow window;
    try {
        // Native modal dialogs need a visible owner during startup.
        window.show();
        if (parser.isSet("data")) {
            if (parser.isSet("screenshots"))
                window.loadPath(parser.value("data"));
            else if (!window.openWorkspace(parser.value("data")))
                return 1;
        }
        if (!parser.isSet("data") && !parser.isSet("screenshots") && !window.welcome())
            return 0;
        if (parser.isSet("data") && !parser.isSet("screenshots"))
            window.offerRecovery();
        if (parser.isSet("screenshots"))
            QTimer::singleShot(100, &window, [&] {
                try {
                    window.captureScreenshots(parser.value("screenshots"));
                    app.exit(0);
                } catch (const std::exception &e) {
                    std::fprintf(stderr, "%s\n", e.what());
                    app.exit(1);
                }
            });
        return app.exec();
    } catch (const std::exception &e) {
        if (parser.isSet("screenshots"))
            std::fprintf(stderr, "%s\n", e.what());
        else
            QMessageBox::critical(nullptr, "SCAPMS startup error", QString::fromUtf8(e.what()));
        return 1;
    }
}
```

5.50 SOURCE OR RESOURCE FILE

src/model/ClinicDatabase.cpp

```
#include "ClinicDatabase.h"
#include "Errors.h"
#include "algorithms/Algorithms.h"
#include <QSet>
#include <limits>

namespace clinic::model {
ClinicDatabase::ClinicDatabase(const ClinicDatabase &other)
    : appointments(other.appointments), waiting(other.waiting), frequent(other.frequent),
      archivedPatients(other.archivedPatients), clinicians(other.clinicians), rooms(other.rooms),
      users(other.users), audit(other.audit), accessClock(other.accessClock),
      nextPatientId(other.nextPatientId), nextAppointmentId(other.nextAppointmentId) {
    for (const auto &patient : other.patients)
        patients.push_back(patient->clone());
}
ClinicDatabase &ClinicDatabase::operator=(const ClinicDatabase &other) {
    if (this != &other) {
        ClinicDatabase copy(other);
        *this = std::move(copy);
    }
    return *this;
}
const PatientRecord *ClinicDatabase::patient(const QString &id) const {
    return algorithms::linearSearchById(patients, id);
}
Appointment *ClinicDatabase::appointment(const QString &id) {
    for (auto &appointment : appointments)
        if (appointment.id == id)
            return &appointment;
    return nullptr;
}
const Appointment *ClinicDatabase::appointment(const QString &id) const {
    for (const auto &appointment : appointments)
        if (appointment.id == id)
            return &appointment;
    return nullptr;
}
void ClinicDatabase::recordAccess(const QString &id) {
    if (!patient(id) || archivedPatients.contains(id))
        return;
    ++accessClock;
    for (auto &entry : frequent)
        if (entry.patientId == id) {
            if (entry.accesses < std::numeric_limits<int>::max())
                ++entry.accesses;
            entry.lastAccess = accessClock;
            return;
        }
    size_t victim = 0;
    for (size_t i = 0; i < frequent.size(); ++i) {
        if (frequent[i].patientId.isEmpty()) {
            victim = i;
            break;
        }
        if (frequent[i].accesses < frequent[victim].accesses ||
            (frequent[i].accesses == frequent[victim].accesses &&
             frequent[i].lastAccess < frequent[victim].lastAccess))
            victim = i;
    }
    frequent[victim] = {id, 1, accessClock};
}
void ClinicDatabase::validate() const {
    QSet<QString> ids;
    for (const auto &p : patients) {
        if (!p)
            throw ValidationError("Null patient record.");
        if (ids.contains(p->id()))
            throw ValidationError("Patient ID already exists: " + p->id());
        ids.insert(p->id());
```

```
    }
    for (const auto &catalog : {clinicians, rooms}) {
        QSet<QString> names;
        for (const auto &name : catalog) {
            if (name.trimmed().isEmpty() || name.size() > 80 || names.contains(name.toCaseFolded())
 > )
                throw ValidationError("Resource names must be unique and 1-80 characters.");
            names.insert(name.toCaseFolded());
        }
    }
    QSet<QString> appointmentIds;
    for (const auto &a : appointments) {
        if (a.id.trimmed().isEmpty() || appointmentIds.contains(a.id))
            throw ValidationError("Missing or duplicate appointment ID.");
        appointmentIds.insert(a.id);
        if ((!a.clinician.isEmpty() && !clinicians.contains(a.clinician)) ||
            (!a.room.isEmpty() && !rooms.contains(a.room)))
            throw ValidationError("Appointment refers to an unknown clinician or room.");
        if (!ids.contains(a.patientId))
            throw ValidationError("Appointment refers to a missing patient: " + a.patientId);
        if (!a.start.isValid() || a.durationMinutes < 5 || a.durationMinutes > 240 ||
            a.service.trimmed().isEmpty())
            throw ValidationError(
                "Appointment requires a valid time, service and duration of 5-240 minutes.");
        if (a.status != AppointmentStatus::Scheduled && a.status != AppointmentStatus::Completed &&
            a.status != AppointmentStatus::Cancelled && a.status != AppointmentStatus::CheckedIn &&
            a.status != AppointmentStatus::InConsultation && a.status != AppointmentStatus::NoShow)
            throw ValidationError("Invalid appointment status.");
    }
    for (size_t i = 0; i < appointments.size(); ++i) {
        const auto &a = appointments[i];
        if (!isPending(a.status))
            continue;
        for (size_t j = i + 1; j < appointments.size(); ++j) {
            const auto &b = appointments[j];
            if (isPending(b.status) && a.start < b.end() && b.start < a.end()) {
                if (a.patientId == b.patientId)
                    throw ValidationError("This patient has overlapping scheduled appointments.");
                if (!a.clinician.isEmpty() && a.clinician == b.clinician)
                    throw ValidationError("The clinician is already booked at this time.");
                if (!a.room.isEmpty() && a.room == b.room)
                    throw ValidationError("The room is already booked at this time.");
            }
        }
    }
    QSet<QString> queued;
    auto copy = waiting;
    while (!copy.empty()) {
        const auto &e = copy.front();
        if (!ids.contains(e.patientId) || archivedPatients.contains(e.patientId) ||
            queued.contains(e.patientId) || !e.arrivedAt.isValid())
            throw ValidationError("Invalid or duplicate patient in the waiting queue.");
        if (!e.appointmentId.isEmpty()) {
            const auto *linked = appointment(e.appointmentId);
            if (!linked || linked->patientId != e.patientId ||
                linked->status != AppointmentStatus::CheckedIn)
                throw ValidationError("Invalid queue appointment link.");
        }
        queued.insert(e.patientId);
        copy.pop();
    }
    for (const auto &id : archivedPatients)
        if (!ids.contains(id))
            throw ValidationError("Archived patient is missing.");
    for (const auto &a : appointments) {
        if (archivedPatients.contains(a.patientId) && isPending(a.status))
            throw ValidationError("Archived patients cannot have active appointments.");
        if (a.status == AppointmentStatus::CheckedIn) {
            auto q = waiting;
            bool found = false;
            while (!q.empty()) {
                if (q.front().appointmentId == a.id)
                    found = true;
                q.pop();
            }
```

```
            if (!found)
                throw ValidationError("Checked-in appointment is missing from queue.");
        }
    }
    QSet<QString> inConsultation;
    for (const auto &a : appointments) {
        if (a.status == AppointmentStatus::InConsultation) {
            if (inConsultation.contains(a.patientId) || queued.contains(a.patientId))
                throw ValidationError(
                    "A patient cannot be waiting and in consultation simultaneously.");
            inConsultation.insert(a.patientId);
        }
    }
    QSet<QString> names;
    bool admin = users.empty();
    for (const auto &u : users) {
        if (u.username.isEmpty() || names.contains(u.username) || u.salt.size() != 16 ||
            u.passwordHash.size() != 32 || u.iterations < 100000 || u.iterations > 1000000)
            throw ValidationError("Invalid staff account.");
        names.insert(u.username);
        if (u.active && u.role == security::Role::Administrator)
            admin = true;
    }
    if (!admin)
        throw ValidationError("An active administrator is required.");
    if (nextPatientId < 1 || nextAppointmentId < 1)
        throw ValidationError("Invalid ID generator state.");
}
} // namespace clinic::model
```

5.51 SOURCE OR RESOURCE FILE

src/model/PatientRecord.cpp

```
#include "PatientRecord.h"
#include "EmergencyPatient.h"
#include "Errors.h"
#include "FollowUpPatient.h"
#include "GeneralPatient.h"
#include <QRegularExpression>

namespace clinic::model {
QString categoryName(PatientCategory category) {
    switch (category) {
    case PatientCategory::General:
        return "General";
    case PatientCategory::Emergency:
        return "Emergency";
    case PatientCategory::FollowUp:
        return "Follow-up";
    }
    throw ValidationError("Unknown patient category.");
}
PatientCategory parseCategory(const QString &value) {
    for (auto category :
         {PatientCategory::General, PatientCategory::Emergency, PatientCategory::FollowUp})
        if (value == categoryName(category))
            return category;
    throw ValidationError("Unknown patient category: " + value);
}
PatientRecord::PatientRecord(PatientDetails details) : details_(std::move(details)) {
    details_.id = details_.id.trimmed();
    details_.name = details_.name.trimmed();
    details_.contact = details_.contact.trimmed();
    details_.reason = details_.reason.trimmed();
    details_.previousVisit = details_.previousVisit.trimmed();
    static const QRegularExpression idPattern("^[A-Za-z0-9_-]{1,40}$");
    if (!idPattern.match(details_.id).hasMatch())
        throw ValidationError(
            "Patient ID must contain 1-40 letters, numbers, hyphens or underscores.");
    if (details_.name.isEmpty() || details_.name.size() > 120)
        throw ValidationError("Patient name must contain 1-120 characters.");
    if (details_.age < 0 || details_.age > 120)
        throw ValidationError("Patient age must be between 0 and 120.");
    if (details_.contact.isEmpty() || details_.contact.size() > 160)
        throw ValidationError("Enter contact information (up to 160 characters).");
    const QRegularExpression email(R"(^[^@\s]+@[^@\s]+\.[^@\s]+$)");
    const QRegularExpression phone(R"(^\+?[0-9 ()-]+$)");
    QString digits = details_.contact;
    digits.remove(QRegularExpression("[^0-9]"));
    if (!email.match(details_.contact).hasMatch() &&
        !(phone.match(details_.contact).hasMatch() && digits.size() >= 7 && digits.size() <= 15))
        throw ValidationError("Enter a valid email address or phone number with 7-15 digits.");
    if (details_.reason.isEmpty() || details_.reason.size() > 1000)
        throw ValidationError("Enter a reason for the visit (up to 1000 characters).");
}
std::unique_ptr<PatientRecord> makePatient(PatientCategory category, PatientDetails details) {
    switch (category) {
    case PatientCategory::General:
        return std::make_unique<GeneralPatient>(std::move(details));
    case PatientCategory::Emergency:
        return std::make_unique<EmergencyPatient>(std::move(details));
    case PatientCategory::FollowUp:
        return std::make_unique<FollowUpPatient>(std::move(details));
    }
    throw ValidationError("Unknown patient category.");
}
QString processPatient(const PatientRecord &patient) {
    return patient.consultationSummary();
}
} // namespace clinic::model
```

5.52 SOURCE OR RESOURCE FILE

src/model/PatientTypes.cpp

```
#include "EmergencyPatient.h"
#include "Errors.h"
#include "FollowUpPatient.h"
#include "GeneralPatient.h"

namespace clinic::model {
namespace {
void schedulingPolicy(const PatientRecord &patient, const Appointment &appointment) {
    if (appointment.patientId != patient.id())
        throw ValidationError("Appointment patient does not match.");
    if (!appointment.start.isValid())
        throw ValidationError("Choose a valid appointment date and time.");
}
void cancellationPolicy(const Appointment &appointment) {
    if (appointment.status != AppointmentStatus::Scheduled)
        throw ValidationError("Only scheduled appointments can be cancelled.");
}
} // namespace
GeneralPatient::GeneralPatient(PatientDetails details) : PatientRecord(std::move(details)) {}
QString GeneralPatient::consultationSummary() const {
    return "General consultation: " + details_.reason;
}
std::unique_ptr<PatientRecord> GeneralPatient::clone() const {
    return std::make_unique<GeneralPatient>(*this);
}
void GeneralPatient::validateScheduling(const Appointment &a) const {
    schedulingPolicy(*this, a);
}
void GeneralPatient::validateCancellation(const Appointment &a) const {
    cancellationPolicy(a);
}

EmergencyPatient::EmergencyPatient(PatientDetails details) : PatientRecord(std::move(details)) {
    if (details_.urgency < 1 || details_.urgency > 5)
        throw ValidationError("Urgency must be between 1 and 5.");
}
QString EmergencyPatient::consultationSummary() const {
    return QString("Urgent review (level %1/5): %2").arg(details_.urgency).arg(details_.reason);
}
std::unique_ptr<PatientRecord> EmergencyPatient::clone() const {
    return std::make_unique<EmergencyPatient>(*this);
}
void EmergencyPatient::validateScheduling(const Appointment &a) const {
    schedulingPolicy(*this, a);
}
void EmergencyPatient::validateCancellation(const Appointment &a) const {
    cancellationPolicy(a);
}

FollowUpPatient::FollowUpPatient(PatientDetails details) : PatientRecord(std::move(details)) {
    if (details_.previousVisit.isEmpty() || details_.previousVisit.size() > 120)
        throw ValidationError(
            "Follow-up patients require a previous visit reference (up to 120 characters).");
}
QString FollowUpPatient::consultationSummary() const {
    return "Review of " + details_.previousVisit + ": " + details_.reason;
}
std::unique_ptr<PatientRecord> FollowUpPatient::clone() const {
    return std::make_unique<FollowUpPatient>(*this);
}
void FollowUpPatient::validateScheduling(const Appointment &a) const {
    schedulingPolicy(*this, a);
}
void FollowUpPatient::validateCancellation(const Appointment &a) const {
    cancellationPolicy(a);
}
} // namespace clinic::model
```

5.53 SOURCE OR RESOURCE FILE

src/persistence/FileHandler.cpp

```
#include "FileHandler.h"
#include "model/Errors.h"
#include <QFile>
#include <QFileInfo>
#include <QJsonArray>
#include <QJsonDocument>
#include <QSaveFile>
#include <cmath>
#include <limits>

namespace clinic::persistence {
using namespace model;
namespace {
QString stringField(const QJsonObject &object, const QString &key) {
    if (!object.value(key).isString())
        throw FileError("Missing or invalid text field: " + key);
    return object.value(key).toString();
}
int integerField(const QJsonObject &object, const QString &key) {
    const auto value = object.value(key);
    if (!value.isDouble())
        throw FileError("Missing or invalid integer field: " + key);
    const double number = value.toDouble();
    if (!std::isfinite(number) || std::floor(number) != number || number < 0 ||
        number > std::numeric_limits<int>::max())
        throw FileError("Invalid integer field: " + key);
    return static_cast<int>(number);
}
QJsonArray arrayField(const QJsonObject &object, const QString &key) {
    if (!object.value(key).isArray())
        throw FileError("Missing or invalid list: " + key);
    return object.value(key).toArray();
}
QJsonObject objectValue(const QJsonValue &value) {
    if (!value.isObject())
        throw FileError("Expected an object in the data file.");
    return value.toObject();
}
} // namespace
QJsonObject FileHandler::serialize(const ClinicDatabase &db) {
    QJsonArray patients, appointments, waiting, archived, users, audit;
    for (const auto &id : db.archivedPatients)
        archived.append(id);
    for (const auto &u : db.users)
        users.append(QJsonObject{{"username", u.username},
                                 {"role", security::roleName(u.role)},
                                 {"salt", QString::fromLatin1(u.salt.toBase64())},
                                 {"passwordHash", QString::fromLatin1(u.passwordHash.toBase64())},
                                 {"iterations", u.iterations},
                                 {"active", u.active}});
    for (const auto &a : db.audit)
        audit.append(QJsonObject{{"time", a.time.toString(Qt::ISODateWithMs)},
                                 {"actor", a.actor},
                                 {"action", a.action}});
    for (const auto &p : db.patients) {
        const auto &d = p->details();
        patients.append(QJsonObject{{"id", d.id},
                                    {"name", d.name},
                                    {"age", d.age},
                                    {"contact", d.contact},
                                    {"category", categoryName(p->category())},
                                    {"reason", d.reason},
                                    {"urgency", d.urgency},
                                    {"previousVisit", d.previousVisit}});
    }
    for (const auto &a : db.appointments)
        appointments.append(
            QJsonObject{{"id", a.id},
                        {"patientId", a.patientId},
```

```
                        {"service", a.service},
                        {"start", a.start.toUTC().toString(Qt::ISODateWithMs)},
                        {"durationMinutes", a.durationMinutes},
                        {"status", statusName(a.status)},
                        {"consultationNotes", a.consultationNotes},
                        {"changeReason", a.changeReason},
                        {"clinician", a.clinician},
                        {"room", a.room},
                        {"walkIn", a.walkIn},
                        {"corrections", QJsonArray::fromStringList(a.corrections)},
                        {"consultationStarted", a.consultationStarted.toString(Qt::ISODateWithMs)},
                        {"completedAt", a.completedAt.toString(Qt::ISODateWithMs)}});
    auto copy = db.waiting;
    while (!copy.empty()) {
        waiting.append(
            QJsonObject{{"patientId", copy.front().patientId},
                        {"arrivedAt", copy.front().arrivedAt.toUTC().toString(Qt::ISODateWithMs)},
                        {"appointmentId", copy.front().appointmentId}});
        copy.pop();
    }
    return {{"schemaVersion", 3},
            {"clinicians", QJsonArray::fromStringList(db.clinicians)},
            {"rooms", QJsonArray::fromStringList(db.rooms)},
            {"archivedPatients", archived},
            {"users", users},
            {"audit", audit},
            {"nextPatientId", db.nextPatientId},
            {"nextAppointmentId", db.nextAppointmentId},
            {"patients", patients},
            {"appointments", appointments},
            {"waitingQueue", waiting}};
}
ClinicDatabase FileHandler::deserialize(const QJsonObject &root) {
    const auto version = integerField(root, "schemaVersion");
    if (version < 1 || version > 3)
        throw FileError("Unsupported data schema version.");
    ClinicDatabase db;
    if (version >= 3) {
        for (auto v : arrayField(root, "clinicians")) {
            if (!v.isString())
                throw FileError("Invalid clinician.");
            db.clinicians << v.toString();
        }
        for (auto v : arrayField(root, "rooms")) {
            if (!v.isString())
                throw FileError("Invalid room.");
            db.rooms << v.toString();
        }
    }
    db.nextPatientId = integerField(root, "nextPatientId");
    db.nextAppointmentId = integerField(root, "nextAppointmentId");
    for (const auto value : arrayField(root, "patients")) {
        const auto object = objectValue(value);
        PatientDetails d;
        d.id = stringField(object, "id");
        d.name = stringField(object, "name");
        d.age = integerField(object, "age");
        d.contact = stringField(object, "contact");
        d.reason = stringField(object, "reason");
        d.urgency = integerField(object, "urgency");
        d.previousVisit = stringField(object, "previousVisit");
        db.patients.push_back(makePatient(parseCategory(stringField(object, "category")), d));
    }
    for (const auto value : arrayField(root, "appointments")) {
        const auto object = objectValue(value);
        Appointment a;
        a.id = stringField(object, "id");
        a.patientId = stringField(object, "patientId");
        a.service = stringField(object, "service");
        a.start =
            QDateTime::fromString(stringField(object, "start"), Qt::ISODateWithMs).toLocalTime();
        a.durationMinutes = integerField(object, "durationMinutes");
        a.consultationNotes = stringField(object, "consultationNotes");
        if (version >= 3) {
            a.changeReason = stringField(object, "changeReason");
```

```
            a.clinician = stringField(object, "clinician");
            a.room = stringField(object, "room");
            if (!object.value("walkIn").isBool())
                throw FileError("Invalid walk-in flag.");
            a.walkIn = object.value("walkIn").toBool();
            for (auto v : arrayField(object, "corrections")) {
                if (!v.isString())
                    throw FileError("Invalid note correction.");
                a.corrections << v.toString();
            }
            a.consultationStarted = QDateTime::fromString(
                stringField(object, "consultationStarted"), Qt::ISODateWithMs);
            a.completedAt =
                QDateTime::fromString(stringField(object, "completedAt"), Qt::ISODateWithMs);
        }
        const auto status = stringField(object, "status");
        if (status == "Scheduled")
            a.status = AppointmentStatus::Scheduled;
        else if (status == "Checked in")
            a.status = AppointmentStatus::CheckedIn;
        else if (status == "In consultation")
            a.status = AppointmentStatus::InConsultation;
        else if (status == "Completed")
            a.status = AppointmentStatus::Completed;
        else if (status == "No-show")
            a.status = AppointmentStatus::NoShow;
        else if (status == "Cancelled")
            a.status = AppointmentStatus::Cancelled;
        else
            throw FileError("Unknown appointment status: " + status);
        db.appointments.push_back(a);
    }
    for (const auto value : arrayField(root, "waitingQueue")) {
        const auto object = objectValue(value);
        db.waiting.push({stringField(object, "patientId"),
                         QDateTime::fromString(stringField(object, "arrivedAt"), Qt::ISODateWithMs)
                             .toLocalTime(),
                         object.value("appointmentId").toString()});
    }
    if (version >= 2) {
        for (const auto value : arrayField(root, "archivedPatients")) {
            if (!value.isString())
                throw FileError("Invalid archived ID.");
            db.archivedPatients.insert(value.toString());
        }
        for (const auto value : arrayField(root, "users")) {
            const auto o = objectValue(value);
            security::UserAccount u;
            u.username = stringField(o, "username");
            u.role = security::parseRole(stringField(o, "role"));
            u.salt = QByteArray::fromBase64(stringField(o, "salt").toLatin1());
            u.passwordHash = QByteArray::fromBase64(stringField(o, "passwordHash").toLatin1());
            u.iterations = integerField(o, "iterations");
            if (!o.value("active").isBool())
                throw FileError("Invalid account status.");
            u.active = o.value("active").toBool();
            db.users.push_back(u);
        }
        for (const auto value : arrayField(root, "audit")) {
            const auto o = objectValue(value);
            auto when = QDateTime::fromString(stringField(o, "time"), Qt::ISODateWithMs);
            if (!when.isValid())
                throw FileError("Invalid audit timestamp.");
            db.audit.push_back({when, stringField(o, "actor"), stringField(o, "action")});
        }
    }
    db.validate();
    return db;
}
ClinicDatabase FileHandler::load(const QString &path) {
    QFile file(path);
    if (!file.open(QIODevice::ReadOnly))
        throw FileError("Could not open file: " + file.errorString());
    if (file.size() > 20 * 1024 * 1024)
        throw FileError("Data files must be smaller than 20 MB.");
```

```
    QJsonParseError error;
    auto bytes = file.readAll();
    if (bytes.startsWith("SCAPMS-PROTECTED-2\n"))
        bytes = security::unprotect(bytes.mid(QByteArray("SCAPMS-PROTECTED-2\n").size()));
    const auto document = QJsonDocument::fromJson(bytes, &error);
    if (error.error != QJsonParseError::NoError || !document.isObject())
        throw FileError("Invalid JSON data: " + error.errorString());
    return deserialize(document.object());
}
void FileHandler::save(const ClinicDatabase &db, const QString &path) {
    db.validate();
    const bool encrypted = QFileInfo(path).suffix().toLower() != "json";
    if (!encrypted && !db.users.empty())
        throw FileError("Staff-protected workspaces must be saved as .clinic files.");
    auto bytes = QJsonDocument(serialize(db)).toJson(QJsonDocument::Indented);
    if (encrypted)
        bytes = QByteArray("SCAPMS-PROTECTED-2\n") + security::protect(bytes);
    QSaveFile file(path);
    if (!file.open(QIODevice::WriteOnly) || file.write(bytes) != bytes.size() || !file.commit())
        throw FileError("Could not save workspace: " + file.errorString());
}
void FileHandler::writeText(const QString &path, const QString &text) {
    QSaveFile file(path);
    if (!file.open(QIODevice::WriteOnly))
        throw FileError("Could not save file: " + file.errorString());
    const auto bytes = text.toUtf8();
    if (file.write(bytes) != bytes.size() || !file.commit())
        throw FileError("Save failed: " + file.errorString());
}
} // namespace clinic::persistence
```

5.54 SOURCE OR RESOURCE FILE

src/persistence/RecoveryService.cpp

```
#include "RecoveryService.h"
#include "model/Errors.h"
#include <QDir>
#include <QFile>
#include <QFileInfo>
#include <QStandardPaths>
#include <QUuid>
namespace clinic::persistence {
QString RecoveryService::recoveryPath(const QString &workspace) {
    if (!workspace.isEmpty())
        return workspace + ".recovery.clinic";
    const auto directory =
        QStandardPaths::writableLocation(QStandardPaths::AppLocalDataLocation) + "/recovery";
    if (!QDir().mkpath(directory))
        throw model::FileError("Cannot create recovery directory.");
    static const auto session = QUuid::createUuid().toString(QUuid::WithoutBraces);
    return directory + "/unsaved-" + session + ".clinic";
}
void RecoveryService::autosave(const model::ClinicDatabase &db, const QString &workspace) {
    FileHandler::save(db, recoveryPath(workspace));
}
void RecoveryService::saveWithBackup(const model::ClinicDatabase &db, const QString &workspace) {
    if (QFileInfo::exists(workspace)) {
        // Validate the existing file before making a protected recovery copy.
        const auto previous = FileHandler::load(workspace);
        const QString folder = workspace + ".backups";
        if (!QDir().mkpath(folder))
            throw model::FileError("Cannot create backup folder.");
        const QString stamp = QDateTime::currentDateTimeUtc().toString("yyyyMMdd-HHmmss-zzz");
        FileHandler::save(previous, folder + "/" + stamp + ".clinic");
    }
    FileHandler::save(db, workspace);
    QDir folder(workspace + ".backups");
    const auto names = folder.entryList({"*.clinic"}, QDir::Files, QDir::Name);
    for (int i = 0; i < names.size() - 5; ++i)
        folder.remove(names[i]);
    discardRecovery(workspace);
}
QStringList RecoveryService::copies(const QString &workspace) {
    QStringList result;
    const auto directory = QFileInfo(recoveryPath({})).absoluteDir();
    for (const auto &name : directory.entryList({"unsaved*.clinic"}, QDir::Files, QDir::Time))
        result.append(directory.filePath(name));
    const auto recovery = recoveryPath(workspace);
    if (QFileInfo::exists(recovery) && !result.contains(recovery))
        result.append(recovery);
    QDir folder(workspace + ".backups");
    for (const auto &name :
         folder.entryList({"*.clinic"}, QDir::Files, QDir::Name | QDir::Reversed))
        result.append(folder.filePath(name));
    return result;
}
void RecoveryService::discardRecovery(const QString &workspace) {
    const auto file = recoveryPath(workspace);
    if (QFileInfo::exists(file) && !QFile::remove(file))
        throw model::FileError("Saved, but the recovery copy could not be removed.");
}
} // namespace clinic::persistence
```

5.55 SOURCE OR RESOURCE FILE

src/persistence/WorkspaceGuard.cpp

```
#include "WorkspaceGuard.h"
#include "model/Errors.h"
#include <QCryptographicHash>
#include <QDir>
#include <QFile>
#include <QFileInfo>
namespace clinic::persistence {
WorkspaceGuard::WorkspaceGuard(const QString &path) {
    QFileInfo info(path);
    path_ = info.exists() ? info.canonicalFilePath()
                          : QDir(info.absolutePath()).canonicalPath() + "/" + info.fileName();
    if (path_.isEmpty() || !QFileInfo(info.absolutePath()).isDir())
        throw model::FileError("Choose an existing writable folder.");
    lock_ = std::make_unique<QLockFile>(path_ + ".lock");
    lock_->setStaleLockTime(0);
    if (!lock_->tryLock(0))
        throw model::FileError("This workspace is open in another instance or cannot be locked. "
                               "Close the other instance or choose another file.");
    checkpoint();
}
QByteArray WorkspaceGuard::digest() const {
    QFile file(path_);
    if (!file.open(QIODevice::ReadOnly))
        throw model::FileError("Cannot verify workspace contents: " + file.errorString());
    QCryptographicHash hash(QCryptographicHash::Sha256);
    if (!hash.addData(&file))
        throw model::FileError("Cannot read workspace fingerprint.");
    return hash.result();
}
void WorkspaceGuard::checkpoint() {
    existed_ = QFileInfo::exists(path_);
    fingerprint_ = existed_ ? digest() : QByteArray();
}
void WorkspaceGuard::verifyUnchanged() const {
    if (QFileInfo::exists(path_) != existed_ || (existed_ && digest() != fingerprint_))
        throw model::FileError("This workspace changed outside this application. Save As to a new "
                               "file, or reload it; the original was not overwritten.");
}
} // namespace clinic::persistence
```

5.56 SOURCE OR RESOURCE FILE

src/security/AccessControl.cpp

```
#include "AccessControl.h"
#include "model/Errors.h"
#include <QCryptographicHash>
#include <QPasswordDigestor>
#include <QRandomGenerator>
#include <QRegularExpression>
#ifdef Q_OS_WIN
#ifndef NOMINMAX
#define NOMINMAX
#endif
#include <windows.h>

#include <dpapi.h>
#endif
namespace clinic::security {
using model::ValidationError;
QString roleName(Role r) {
    switch (r) {
    case Role::Administrator:
        return "Administrator";
    case Role::Clinician:
        return "Clinician";
    case Role::Receptionist:
        return "Receptionist";
    case Role::Viewer:
        return "Viewer";
    }
    throw ValidationError("Invalid staff role.");
}
Role parseRole(const QString &name) {
    for (auto r : {Role::Administrator, Role::Clinician, Role::Receptionist, Role::Viewer})
        if (roleName(r) == name)
            return r;
    throw ValidationError("Unknown staff role.");
}
UserAccount createAccount(const QString &username, const QString &password, Role role) {
    const auto name = username.trimmed().toLower();
    if (!QRegularExpression("^[a-z0-9._-]{3,40}$").match(name).hasMatch())
        throw ValidationError(
            "Username must be 3-40 letters, numbers, dots, hyphens or underscores.");
    if (password.size() < 12 || password.size() > 256)
        throw ValidationError("Use a password or passphrase of 12-256 characters.");
    UserAccount account;
    account.username = name;
    account.role = role;
    account.salt.resize(16);
    for (int i = 0; i < 16; ++i)
        account.salt[i] = static_cast<char>(QRandomGenerator::system()->generate() & 255);
    account.passwordHash = QPasswordDigestor::deriveKeyPbkdf2(
        QCryptographicHash::Sha256, password.toUtf8(), account.salt, account.iterations, 32);
    if (account.passwordHash.size() != 32)
        throw ValidationError("Password hashing failed.");
    return account;
}
bool verifyPassword(const UserAccount &account, const QString &password) {
    if (!account.active || password.size() > 256 || account.iterations < 100000 ||
        account.iterations > 1000000)
        return false;
    const auto digest = QPasswordDigestor::deriveKeyPbkdf2(
        QCryptographicHash::Sha256, password.toUtf8(), account.salt, account.iterations, 32);
    if (digest.size() != account.passwordHash.size())
        return false;
    unsigned char different = 0;
    for (int i = 0; i < digest.size(); ++i)
        different |= static_cast<unsigned char>(digest[i] ^ account.passwordHash[i]);
    return different == 0;
}
QByteArray protect(const QByteArray &plain) {
#ifdef Q_OS_WIN
```

```
    DATA_BLOB input{static_cast<DWORD>(plain.size()),
                    reinterpret_cast<BYTE *>(const_cast<char *>(plain.constData()))},
        output{};
    if (!CryptProtectData(&input, L"SCAPMS protected workspace", nullptr, nullptr, nullptr,
                          CRYPTPROTECT_UI_FORBIDDEN, &output))
        throw model::FileError("Windows could not protect this workspace.");
    QByteArray result(reinterpret_cast<char *>(output.pbData), output.cbData);
    SecureZeroMemory(output.pbData, output.cbData);
    LocalFree(output.pbData);
    return result;
#else
    Q_UNUSED(plain);
    throw model::FileError("Protected workspaces require Windows account encryption.");
#endif
}
QByteArray unprotect(const QByteArray &encrypted) {
#ifdef Q_OS_WIN
    DATA_BLOB input{static_cast<DWORD>(encrypted.size()),
                    reinterpret_cast<BYTE *>(const_cast<char *>(encrypted.constData()))},
        output{};
    if (!CryptUnprotectData(&input, nullptr, nullptr, nullptr, nullptr, CRYPTPROTECT_UI_FORBIDDEN,
                            &output))
        throw model::FileError("Cannot decrypt this workspace. Use the Windows account and "
                               "computer that saved it, or a valid backup.");
    QByteArray result(reinterpret_cast<char *>(output.pbData), output.cbData);
    SecureZeroMemory(output.pbData, output.cbData);
    LocalFree(output.pbData);
    return result;
#else
    Q_UNUSED(encrypted);
    throw model::FileError("Protected workspaces require Windows.");
#endif
}
} // namespace clinic::security
```

5.57 SOURCE OR RESOURCE FILE

src/security/PortableBackup.cpp

```
#include "PortableBackup.h"
#include "model/Errors.h"
#include <QCryptographicHash>
#include <QPasswordDigestor>
#include <QRandomGenerator>
#ifdef Q_OS_WIN
#ifndef NOMINMAX
#define NOMINMAX
#endif
#include <windows.h>
#include <bcrypt.h>
#endif
namespace clinic::security {
namespace {
const QByteArray magic("SCAPMS-BACKUP-1\n");
constexpr int iterations = 600000;
constexpr int maxBytes = 20 * 1024 * 1024;
QByteArray randomBytes(int count) {
    QByteArray out(count, Qt::Uninitialized);
    for (auto &c : out)
        c = static_cast<char>(QRandomGenerator::system()->generate() & 255);
    return out;
}
QByteArray crypt(const QByteArray &input, const QString &password, const QByteArray &salt,
                 QByteArray nonce, QByteArray &tag, bool encrypt) {
#ifdef Q_OS_WIN
    struct Handles {
        BCRYPT_ALG_HANDLE algorithm = nullptr;
        BCRYPT_KEY_HANDLE key = nullptr;
        QByteArray secret;
        ~Handles() {
            if (key)
                BCryptDestroyKey(key);
            if (algorithm)
                BCryptCloseAlgorithmProvider(algorithm, 0);
            if (!secret.isEmpty())
                SecureZeroMemory(secret.data(), secret.size());
        }
    } handles;
    auto check = [](NTSTATUS status) {
        if (status < 0)
            throw model::FileError("Backup authentication failed or encryption is unavailable. "
                                   "Check the password and backup file.");
    };
    handles.secret = QPasswordDigestor::deriveKeyPbkdf2(QCryptographicHash::Sha256,
                                                        password.toUtf8(), salt, iterations, 32);
    if (handles.secret.size() != 32)
        throw model::FileError("Backup key derivation failed.");
    check(BCryptOpenAlgorithmProvider(&handles.algorithm, BCRYPT_AES_ALGORITHM, nullptr, 0));
    check(BCryptSetProperty(handles.algorithm, BCRYPT_CHAINING_MODE,
                            reinterpret_cast<PUCHAR>(const_cast<wchar_t *>(BCRYPT_CHAIN_MODE_GCM)),
                            sizeof(BCRYPT_CHAIN_MODE_GCM), 0));
    check(BCryptGenerateSymmetricKey(handles.algorithm, &handles.key, nullptr, 0,
                                     reinterpret_cast<PUCHAR>(handles.secret.data()),
                                     handles.secret.size(), 0));
    BCRYPT_AUTHENTICATED_CIPHER_MODE_INFO info;
    BCRYPT_INIT_AUTH_MODE_INFO(info);
    auto aad = magic + salt + nonce;
    info.pbNonce = reinterpret_cast<PUCHAR>(nonce.data());
    info.cbNonce = nonce.size();
    info.pbTag = reinterpret_cast<PUCHAR>(tag.data());
    info.cbTag = tag.size();
    info.pbAuthData = reinterpret_cast<PUCHAR>(aad.data());
    info.cbAuthData = aad.size();
    QByteArray out(input.size(), Qt::Uninitialized);
    ULONG written = 0;
    auto operation = encrypt ? BCryptEncrypt : BCryptDecrypt;
    const auto status = operation(
        handles.key, reinterpret_cast<PUCHAR>(const_cast<char *>(input.constData())), input.size(),
```

```
        &info, nullptr, 0, reinterpret_cast<PUCHAR>(out.data()), out.size(), &written, 0);
    if (status < 0) {
        SecureZeroMemory(out.data(), out.size());
        check(status);
    }
    out.resize(written);
    return out;
#else
    Q_UNUSED(input);
    Q_UNUSED(password);
    Q_UNUSED(salt);
    Q_UNUSED(nonce);
    Q_UNUSED(tag);
    Q_UNUSED(encrypt);
    throw model::FileError("Portable backup encryption requires the Windows build.");
#endif
}
} // namespace
QByteArray encryptBackup(const QByteArray &plain, const QString &password) {
    if (password.size() < 12 || password.size() > 256)
        throw model::ValidationError("Use a backup passphrase of 12-256 characters.");
    if (plain.isEmpty() || plain.size() > maxBytes)
        throw model::FileError("Backup data must be 1 byte to 20 MB.");
    const auto salt = randomBytes(16), nonce = randomBytes(12);
    QByteArray tag(16, 0);
    const auto cipher = crypt(plain, password, salt, nonce, tag, true);
    return magic + salt + nonce + tag + cipher;
}
QByteArray decryptBackup(const QByteArray &data, const QString &password) {
    const int header = magic.size() + 44;
    if (!data.startsWith(magic) || data.size() <= header || data.size() > maxBytes + header ||
        password.size() < 12 || password.size() > 256)
        throw model::FileError("Invalid backup format or passphrase.");
    const auto salt = data.mid(magic.size(), 16), nonce = data.mid(magic.size() + 16, 12);
    auto tag = data.mid(magic.size() + 28, 16);
    return crypt(data.mid(header), password, salt, nonce, tag, false);
}
} // namespace clinic::security
```

5.58 SOURCE OR RESOURCE FILE

tests/ClinicTests.cpp

```
#include "algorithms/Algorithms.h"
#include "algorithms/Benchmark.h"
#include "security/PortableBackup.h"
#include "persistence/WorkspaceGuard.h"
#include "gui/PatientTableModel.h"
#include "controller/ClinicManager.h"
#include "controller/ReportService.h"
#include "gui/MainWindow.h"
#include "gui/PatientDialog.h"
#include "gui/panels/PatientsPanel.h"
#include "model/EmergencyPatient.h"
#include "model/Errors.h"
#include "model/FollowUpPatient.h"
#include "persistence/FileHandler.h"
#include "persistence/RecoveryService.h"
#include "security/AccessControl.h"
#include <QtTest>
#include <QtWidgets>
#include <random>
using namespace clinic;
using namespace clinic::model;
using namespace clinic::controller;
using namespace clinic::algorithms;
using persistence::FileHandler;
namespace {
PatientDetails detail(QString id, QString name = "Test Patient", int age = 25) {
    return {id,
            name,
            age,
            (id.isEmpty() ? QString("test") : id.toLower()) + "@example.test",
            "Health review",
            4,
            "VIS-001"};
}
QJsonObject stateOnly(const ClinicDatabase &db) {
    auto json = FileHandler::serialize(db);
    json.remove("audit");
    return json;
}
ClinicDatabase fixture() {
    ClinicDatabase db;
    db.patients.push_back(makePatient(PatientCategory::General, detail("P1", "Ada", 23)));
    db.patients.push_back(makePatient(PatientCategory::Emergency, detail("P2", "Tunde", 31)));
    db.patients.push_back(makePatient(PatientCategory::FollowUp, detail("P3", "Zainab", 27)));
    const auto now = QDateTime::currentDateTime();
    db.appointments.push_back(
        {"A1", "P1", "Review", now.addSecs(-3600), 30, AppointmentStatus::Scheduled, {}});
    db.appointments.push_back({"A2", "P2", "Consultation", now.addSecs(-7200), 30,
                               AppointmentStatus::Completed, "Reviewed"});
    db.appointments.push_back(
        {"A3", "P1", "Consultation", now.addSecs(7200), 30, AppointmentStatus::Cancelled, {}});
    db.validate();
    return db;
}
} // namespace
class ClinicTests : public QObject {
    Q_OBJECT
  private slots:
    void patientValidationAndPolymorphism() {
        QVERIFY_EXCEPTION_THROWN(makePatient(PatientCategory::General, detail("P1", "", 25)),
                                 ValidationError);
        QVERIFY_EXCEPTION_THROWN(makePatient(PatientCategory::General, detail("P1", "Ada", 121)),
                                 ValidationError);
        auto d = detail("P1");
        d.previousVisit.clear();
        QVERIFY_EXCEPTION_THROWN(makePatient(PatientCategory::FollowUp, d), ValidationError);
        auto p = makePatient(PatientCategory::Emergency, detail("E1"));
        QVERIFY(processPatient(*p).contains("level 4"));
        QVERIFY(dynamic_cast<EmergencyPatient *>(p->clone().get()) != nullptr);
```

```
        auto f = makePatient(PatientCategory::FollowUp, detail("F1"));
        QVERIFY(processPatient(*f).contains("VIS-001"));
    }
    void duplicateAndFailedOperationAreAtomic() {
        ClinicManager m;
        m.addPatient(PatientCategory::General, detail("P1"));
        const auto before = stateOnly(m.database());
        QVERIFY_EXCEPTION_THROWN(m.addPatient(PatientCategory::General, detail("P1")),
                                 ValidationError);
        QCOMPARE(stateOnly(m.database()), before);
        m.undo();
        QCOMPARE(m.database().patients.size(), size_t(0));
        QVERIFY(!m.canUndo());
    }
    void cascadeDeleteUndoRestoresSubtypeAndQueue() {
        ClinicManager m;
        m.replace(fixture());
        m.enqueue("P2");
        m.enqueue("P1");
        m.access("P2");
        const auto before = stateOnly(m.database());
        m.deletePatient("P2");
        QVERIFY(m.database().archivedPatients.contains("P2"));
        QVERIFY(m.database().appointment("A2"));
        QCOMPARE(m.database().waiting.front().patientId, QString("P1"));
        m.undo();
        QCOMPARE(stateOnly(m.database()), before);
        QVERIFY(dynamic_cast<const EmergencyPatient *>(m.database().patient("P2")));
        QCOMPARE(m.database().waiting.front().patientId, QString("P2"));
        QVERIFY(!m.database().frequent[0].patientId.isEmpty());
    }
    void editUndoPreservesRelationships() {
        ClinicManager m;
        m.replace(fixture());
        m.enqueue("P1");
        m.editPatient("P1", PatientCategory::FollowUp, detail("OTHER", "Updated", 55));
        QCOMPARE(m.database().patient("P1")->details().name, QString("Updated"));
        QCOMPARE(m.database().appointment("A1")->patientId, QString("P1"));
        QCOMPARE(m.database().waiting.front().patientId, QString("P1"));
        m.undo();
        QCOMPARE(m.database().patient("P1")->details().name, QString("Ada"));
        QCOMPARE(m.database().patient("P1")->category(), PatientCategory::General);
    }
    void fifoAndDuplicateQueue() {
        ClinicManager m;
        m.replace(fixture());
        QVERIFY_EXCEPTION_THROWN(m.callNext(), ValidationError);
        m.enqueue("P3");
        m.enqueue("P1");
        QVERIFY_EXCEPTION_THROWN(m.enqueue("P3"), ValidationError);
        QVERIFY(m.callNext().startsWith("Zainab"));
        QCOMPARE(m.database().waiting.front().patientId, QString("P1"));
        m.undo();
        QCOMPARE(m.database().waiting.front().patientId, QString("P3"));
        m.removeFromQueue("P3");
        QCOMPARE(m.database().waiting.front().patientId, QString("P1"));
    }
    void appointmentLifecycleAndConflicts() {
        ClinicManager m;
        m.replace(fixture());
        const auto future = QDateTime::currentDateTime().addDays(1);
        const auto id = m.schedule("P1", "Review", future, 30);
        QVERIFY_EXCEPTION_THROWN(m.schedule("P1", "Overlap", future.addSecs(60), 30),
                                 ValidationError);
        const auto adjacent = m.schedule("P1", "Next slot", future.addSecs(1800), 30);
        QVERIFY(!adjacent.isEmpty());
        QVERIFY_EXCEPTION_THROWN(m.complete(id, "Notes"), ValidationError);
        m.cancel(id, "Patient requested cancellation");
        QCOMPARE(m.database().appointment(id)->status, AppointmentStatus::Cancelled);
        m.undo();
        QCOMPARE(m.database().appointment(id)->status, AppointmentStatus::Scheduled);
        m.checkIn("A1");
        m.callNext();
        m.complete("A1", "Review complete");
        QCOMPARE(m.database().appointment("A1")->status, AppointmentStatus::Completed);
```

```
        QVERIFY(m.database().waiting.empty());
        QVERIFY_EXCEPTION_THROWN(m.cancel("A1"), ValidationError);
        QVERIFY_EXCEPTION_THROWN(m.schedule("missing", "Review", future, 30), ValidationError);
        QVERIFY_EXCEPTION_THROWN(m.schedule("P1", "Review", future, 0), ValidationError);
        QVERIFY_EXCEPTION_THROWN(m.schedule("P1", "Review", future.addDays(-2), 30),
                                 ValidationError);
    }
    void manualAlgorithmsEdgeCasesAndStability() {
        for (auto algorithm : {SortAlgorithm::Insertion, SortAlgorithm::Merge}) {
            PatientView empty;
            sortPatients(empty, SortField::Id, algorithm);
            QVERIFY(!binarySearchById(empty, "P1"));
            ClinicDatabase db;
            db.patients.push_back(makePatient(PatientCategory::General, detail("P3", "same", 30)));
            db.patients.push_back(makePatient(PatientCategory::General, detail("P1", "SAME", 10)));
            db.patients.push_back(makePatient(PatientCategory::General, detail("P2", "Alpha", 20)))
  > ;
            auto view = viewOf(db.patients);
            sortPatients(view, SortField::Name, algorithm);
            QCOMPARE(view[0]->id(), QString("P2"));
            QCOMPARE(view[1]->id(), QString("P3"));
            QCOMPARE(view[2]->id(), QString("P1"));
            sortPatients(view, SortField::Age, algorithm);
            QCOMPARE(view[0]->details().age, 10);
            QCOMPARE(view[2]->details().age, 30);
            sortPatients(view, SortField::Id, algorithm);
            QCOMPARE(binarySearchById(view, "P1")->id(), QString("P1"));
            QCOMPARE(binarySearchById(view, "P3")->id(), QString("P3"));
            QVERIFY(!binarySearchById(view, "P9"));
            QVERIFY(!binarySearchById(view, "P0"));
            QCOMPARE(linearSearchById(db.patients, "P2")->id(), QString("P2"));
            QCOMPARE(filterPatients(view, "same", false).size(), size_t(2));
            QCOMPARE(filterPatients(view, "general", true).size(), size_t(3));
        }
        std::mt19937 random(42);
        std::vector<int> numbers;
        for (int i = 0; i < 1000; ++i)
            numbers.push_back(static_cast<int>(random() % 50));
        auto copy = numbers;
        Metrics metrics;
        mergeSort(numbers, [](int a, int b) { return a < b; }, &metrics);
        insertionSort(copy, [](int a, int b) { return a < b; });
        QCOMPARE(numbers, copy);
        QVERIFY(metrics.maxRecursionDepth > 1);
        for (size_t i = 1; i < numbers.size(); ++i)
            QVERIFY(numbers[i - 1] <= numbers[i]);
    }
    void persistenceRoundTripAndMalformedData() {
        QTemporaryDir dir;
        QVERIFY(dir.isValid());
        auto db = fixture();
        db.waiting.push({"P3", QDateTime::currentDateTime()});
        const auto path = dir.filePath("clinic.json");
        FileHandler::save(db, path);
        auto loaded = FileHandler::load(path);
        QCOMPARE(FileHandler::serialize(db), FileHandler::serialize(loaded));
        QVERIFY(dynamic_cast<const FollowUpPatient *>(loaded.patient("P3")));
        auto invalid = FileHandler::serialize(db);
        invalid["schemaVersion"] = 99;
        QVERIFY_EXCEPTION_THROWN(FileHandler::deserialize(invalid), FileError);
        invalid = FileHandler::serialize(db);
        auto patients = invalid["patients"].toArray();
        patients.append(patients[0]);
        invalid["patients"] = patients;
        QVERIFY_EXCEPTION_THROWN(FileHandler::deserialize(invalid), ValidationError);
        invalid = FileHandler::serialize(db);
        auto appointments = invalid["appointments"].toArray();
        auto a = appointments[0].toObject();
        a["patientId"] = "missing";
        appointments[0] = a;
        invalid["appointments"] = appointments;
        QVERIFY_EXCEPTION_THROWN(FileHandler::deserialize(invalid), ValidationError);
        FileHandler::writeText(path, "{invalid json");
        QVERIFY_EXCEPTION_THROWN(FileHandler::load(path), FileError);
        QVERIFY_EXCEPTION_THROWN(FileHandler::save(db, dir.filePath("missing/folder/data.json")),
```

```
                                FileError);
        ClinicManager m;
        m.replace(fixture());
        const auto before = FileHandler::serialize(m.database());
        try {
            m.replace(FileHandler::load(path));
        } catch (const ClinicError &) {
        }
        QCOMPARE(FileHandler::serialize(m.database()), before);
    }
    void idGenerationAfterLoadAndCacheCapacity() {
        ClinicManager m;
        m.addPatient(PatientCategory::General, detail("P0001"));
        auto blank = detail("");
        m.addPatient(PatientCategory::General, blank);
        QVERIFY(m.database().patient("P0002"));
        m.replace(FileHandler::deserialize(FileHandler::serialize(m.database())));
        m.addPatient(PatientCategory::General, blank);
        QVERIFY(m.database().patient("P0003"));
        for (int i = 4; i <= 12; ++i)
            m.addPatient(PatientCategory::General, detail("X" + QString::number(i)));
        m.access("P0001");
        m.access("P0001");
        for (const auto &p : m.database().patients)
            m.access(p->id());
        size_t count = 0;
        bool retained = false;
        for (const auto &entry : m.database().frequent) {
            if (!entry.patientId.isEmpty())
                ++count;
            if (entry.patientId == "P0001")
                retained = true;
        }
        QCOMPARE(count, size_t(10));
        QVERIFY(retained);
    }
    void reportsAndCsv() {
        auto db = fixture();
        auto pending = generateReport(db, ReportType::Pending);
        QCOMPARE(pending.rows.size(), 1);
        auto attendance = generateReport(db, ReportType::Attendance);
        QCOMPARE(attendance.rows[0][0], QString("P2"));
        auto categories = generateReport(db, ReportType::Categories);
        QCOMPARE(categories.rows.size(), 3);
        for (const auto &row : categories.rows)
            QCOMPARE(row[1], QString("1"));
        auto daily = generateReport(db, ReportType::Daily, db.appointments[0].start.date());
        int expected = 0;
        for (const auto &a : db.appointments)
            if (a.start.date() == db.appointments[0].start.date())
                ++expected;
        QCOMPARE(daily.rows[0][1].toInt(), expected);
        Report r{"CSV", {"Name"}, {{"A, \"B\""}, {"=SUM(A1:A2)"}}};
        QVERIFY(reportCsv(r).contains("\"A, \"\"B\"\"\""));
        QVERIFY(reportCsv(r).contains("'=SUM"));
    }
    void guiRegistrationSearchSortQueueAndSave() {
        QTemporaryDir dir;
        gui::MainWindow window;
        window.show();
        QApplication::processEvents();
        auto *nav = window.findChild<QListWidget *>("navigation");
        QVERIFY(nav);
        nav->setCurrentRow(1);
        QTimer::singleShot(30, [&] {
            auto *dialog = window.findChild<QDialog *>("patientDialog");
            QVERIFY(dialog);
            dialog->findChild<QLineEdit *>("patientName")->setText("GUI Patient");
            dialog->findChild<QLineEdit *>("patientContact")->setText("gui@example.test");
            dialog->findChild<QLineEdit *>("patientReason")->setText("Routine review");
            QTest::mouseClick(dialog->findChild<QDialogButtonBox *>("patientButtons")
                                  ->button(QDialogButtonBox::Save),
                              Qt::LeftButton);
        });
        QTest::mouseClick(window.findChild<QPushButton *>("addPatient"), Qt::LeftButton);
```

```
294         QCOMPARE(window.manager().database().patients.size(), size_t(1));
295         nav->setCurrentRow(4);
296         window.findChild<QComboBox *>("searchAlgorithm")->setCurrentIndex(1);
297         window.findChild<QLineEdit *>("searchText")->setText("P0001");
298         QTest::mouseClick(window.findChild<QPushButton *>("searchButton"), Qt::LeftButton);
299         QCOMPARE(window.findChild<QTableView *>("searchResults")->model()->rowCount(), 1);
300         window.findChild<QComboBox *>("sortAlgorithm")->setCurrentIndex(1);
301         QTest::mouseClick(window.findChild<QPushButton *>("sortButton"), Qt::LeftButton);
302         QVERIFY(window.findChild<QLabel *>("searchInfo")->text().contains("Recursive merge"));
303         nav->setCurrentRow(3);
304         QTest::mouseClick(window.findChild<QPushButton *>("enqueuePatient"), Qt::LeftButton);
305         QCOMPARE(window.manager().database().waiting.size(), size_t(1));
306         QTest::mouseClick(window.findChild<QPushButton *>("callNext"), Qt::LeftButton);
307         QVERIFY(window.manager().database().waiting.empty());
308         QTest::mouseClick(window.findChild<QPushButton *>("undoButton"), Qt::LeftButton);
309         QCOMPARE(window.manager().database().waiting.size(), size_t(1));
310         const auto file = dir.filePath("gui.json");
311         FileHandler::save(window.manager().database(), file);
312         window.loadPath(file);
313         QVERIFY(!window.manager().dirty());
314         nav->setCurrentRow(5);
315         for (int i = 0; i < 4; ++i)
316             window.findChild<QComboBox *>("reportType")->setCurrentIndex(i);
317         window.close();
318     }
319     void guiAppointmentDeleteUndoAndSave() {
320         QTemporaryDir dir;
321         gui::MainWindow window;
322         const auto path = dir.filePath("clinic.json");
323         FileHandler::save(fixture(), path);
324         window.loadPath(path);
325         window.show();
326         auto *nav = window.findChild<QListWidget *>("navigation");
327         nav->setCurrentRow(2);
328         auto *appointments = window.findChild<QTableWidget *>("appointmentsTable");
329         window.findChild<QDateTimeEdit *>("appointmentTime")
330             ->setDateTime(QDateTime::currentDateTime().addDays(2));
331         QTest::mouseClick(window.findChild<QPushButton *>("scheduleAppointment"), Qt::LeftButton);
332         QCOMPARE(window.manager().database().appointments.size(), size_t(4));
333         appointments->selectRow(3);
334         QTimer::singleShot(30, [&] {
335             auto *input = window.findChild<QInputDialog *>();
336             QVERIFY(input);
337             input->setTextValue("Patient requested cancellation");
338             input->accept();
339         });
340         QTest::mouseClick(window.findChild<QPushButton *>("cancelAppointment"), Qt::LeftButton);
341         QCOMPARE(window.manager().database().appointments[3].status, AppointmentStatus::Cancelled);
342         QTest::mouseClick(window.findChild<QPushButton *>("undoButton"), Qt::LeftButton);
343         QCOMPARE(window.manager().database().appointments[3].status, AppointmentStatus::Scheduled);
344         window.manager().checkIn("A1");
345         window.manager().callNext();
346         window.refresh();
347         appointments->selectRow(0);
348         QTimer::singleShot(30, [&] {
349             auto *input = window.findChild<QInputDialog *>();
350             QVERIFY(input);
351             input->setTextValue("GUI consultation completed");
352             input->accept();
353         });
354         QTest::mouseClick(window.findChild<QPushButton *>("completeAppointment"), Qt::LeftButton);
355         QCOMPARE(window.manager().database().appointment("A1")->status,
356                  AppointmentStatus::Completed);
357         nav->setCurrentRow(1);
358         window.findChild<QTableView *>("patientsTable")->selectRow(1);
359         QTimer::singleShot(30, [&] {
360             auto *box = window.findChild<QMessageBox *>();
361             QVERIFY(box);
362             box->button(QMessageBox::Yes)->click();
363         });
364         QTest::mouseClick(window.findChild<QPushButton *>("deletePatient"), Qt::LeftButton);
365         QVERIFY(window.manager().database().archivedPatients.contains("P2"));
366         QTest::mouseClick(window.findChild<QPushButton *>("undoButton"), Qt::LeftButton);
367         QVERIFY(!window.manager().database().archivedPatients.contains("P2"));
368         QTest::mouseClick(window.findChild<QPushButton *>("saveButton"), Qt::LeftButton);
```

```
        QVERIFY(!window.manager().dirty());
        QCOMPARE(FileHandler::serialize(FileHandler::load(path)),
                 FileHandler::serialize(window.manager().database()));
        window.close();
    }

    void rescheduleConflictAndUndo() {
        ClinicManager m;
        m.replace(fixture());
        const auto future = QDateTime::currentDateTime().addDays(1);
        const auto first = m.schedule("P1", "Review", future, 30);
        const auto second = m.schedule("P1", "Review", future.addSecs(3600), 30);
        const auto before = stateOnly(m.database());
        QVERIFY_EXCEPTION_THROWN(m.reschedule(second, future.addSecs(60), 30), ValidationError);
        QCOMPARE(stateOnly(m.database()), before);
        m.reschedule(second, future.addSecs(7200), 45);
        QCOMPARE(m.database().appointment(second)->durationMinutes, 45);
        m.undo();
        QCOMPARE(stateOnly(m.database()), before);
        m.cancel(first, "Changed plans");
        QVERIFY_EXCEPTION_THROWN(m.reschedule(first, future.addSecs(9000), 30), ValidationError);
    }
    void linkedCheckInConsultationLifecycle() {
        ClinicManager m;
        auto db = fixture();
        db.appointments[0].start = QDateTime::currentDateTime().addSecs(-60);
        m.replace(db);
        m.checkIn("A1");
        QCOMPARE(m.database().appointment("A1")->status, AppointmentStatus::CheckedIn);
        QCOMPARE(m.database().waiting.front().appointmentId, QString("A1"));
        QVERIFY_EXCEPTION_THROWN(m.checkIn("A1"), ValidationError);
        QVERIFY_EXCEPTION_THROWN(m.enqueue("P1"), ValidationError);
        QVERIFY(m.callNext().contains("Ada"));
        QCOMPARE(m.database().appointment("A1")->status, AppointmentStatus::InConsultation);
        m.complete("A1", "Linked workflow complete");
        QCOMPARE(m.database().appointment("A1")->status, AppointmentStatus::Completed);
        m.undo();
        QCOMPARE(m.database().appointment("A1")->status, AppointmentStatus::InConsultation);
        m.undo();
        QCOMPARE(m.database().appointment("A1")->status, AppointmentStatus::CheckedIn);
        m.removeFromQueue("P1");
        QCOMPARE(m.database().appointment("A1")->status, AppointmentStatus::Scheduled);
        m.checkIn("A1");
        m.cancel("A1", "Cannot attend");
        QVERIFY(m.database().waiting.empty());
    }
    void archiveRetainsHistoryAndAudit() {
        ClinicManager m;
        m.replace(fixture());
        const auto oldCount = m.database().appointments.size();
        m.deletePatient("P1");
        QVERIFY(m.database().archivedPatients.contains("P1"));
        QCOMPARE(m.database().appointments.size(), oldCount);
        QCOMPARE(m.database().appointment("A1")->status, AppointmentStatus::Cancelled);
        QVERIFY_EXCEPTION_THROWN(m.enqueue("P1"), ValidationError);
        QVERIFY_EXCEPTION_THROWN(
            m.schedule("P1", "Visit", QDateTime::currentDateTime().addDays(1), 30),
            ValidationError);
        const auto auditSize = m.database().audit.size();
        m.undo();
        QVERIFY(!m.database().archivedPatients.contains("P1"));
        QCOMPARE(m.database().audit.size(), auditSize + 1);
        QVERIFY(m.database().audit.back().action.startsWith("Undo:"));
        m.deletePatient("P1");
        m.restorePatient("P1");
        QVERIFY(!m.database().archivedPatients.contains("P1"));
        QCOMPARE(m.database().appointment("A1")->status, AppointmentStatus::Cancelled);
    }
    void staffRolesPasswordsAndLocking() {
        ClinicManager m;
        m.replace(fixture());
        m.configureAdministrator("admin", "a long admin passphrase");
        m.addUser("desk", "a long desk passphrase", security::Role::Receptionist);
        m.addUser("doctor", "a long doctor passphrase", security::Role::Clinician);
        m.addUser("reader", "a long reader passphrase", security::Role::Viewer);
```

```
        const auto secured = m.database();
        QVERIFY(!secured.users[0].passwordHash.contains("passphrase"));
        m.lock();
        QVERIFY(!m.canEdit());
        QVERIFY_EXCEPTION_THROWN(m.addPatient(PatientCategory::General, detail("P9")),
                                 ValidationError);
        m.login("desk", "a long desk passphrase");
        QVERIFY(m.canEdit());
        QVERIFY(!m.canViewClinicalNotes());
        QVERIFY_EXCEPTION_THROWN(m.complete("A1", "Notes"), ValidationError);
        QVERIFY_EXCEPTION_THROWN(m.deletePatient("P1"), ValidationError);
        QVERIFY_EXCEPTION_THROWN(
            m.addUser("other", "a long other password", security::Role::Viewer), ValidationError);
        m.addPatient(PatientCategory::General, detail("P9"));
        m.lock();
        m.login("reader", "a long reader passphrase");
        QVERIFY_EXCEPTION_THROWN(m.enqueue("P1"), ValidationError);
        QVERIFY(!m.canUndo());
        m.lock();
        m.login("doctor", "a long doctor passphrase");
        m.checkIn("A1");
        m.callNext();
        m.complete("A1", "Clinician completed");
        m.lock();
        m.login("admin", "a long admin passphrase");
        m.setUserActive("reader", false);
        QVERIFY_EXCEPTION_THROWN(m.setUserActive("admin", false), ValidationError);
        m.changePassword("a long admin passphrase", "a different long admin password");
        m.lock();
        QVERIFY_EXCEPTION_THROWN(m.login("reader", "a long reader passphrase"), ValidationError);
        m.login("admin", "a different long admin password");
        ClinicManager target;
        target.replace(fixture());
        const auto before = stateOnly(target.database());
        QVERIFY_EXCEPTION_THROWN(target.replace(secured, "admin", "wrong password"),
                                 ValidationError);
        QCOMPARE(stateOnly(target.database()), before);
        target.replace(secured, "admin", "a long admin passphrase");
        QVERIFY(target.isAdmin());
    }
    void protectedStorageTamperAndRecovery() {
#ifdef Q_OS_WIN
        QTemporaryDir dir;
        QVERIFY(dir.isValid());
        ClinicManager m;
        m.replace(fixture());
        m.configureAdministrator("admin", "a long admin passphrase");
        const auto file = dir.filePath("secure.clinic");
        persistence::RecoveryService::saveWithBackup(m.database(), file);
        QFile contents(file);
        QVERIFY(contents.open(QIODevice::ReadOnly));
        auto bytes = contents.readAll();
        contents.close();
        QVERIFY(bytes.startsWith("SCAPMS-PROTECTED-2\n"));
        QVERIFY(!bytes.contains("Ada"));
        QVERIFY(!bytes.contains("passphrase"));
        auto restored = FileHandler::load(file);
        QCOMPARE(FileHandler::serialize(restored), FileHandler::serialize(m.database()));
        QVERIFY_EXCEPTION_THROWN(FileHandler::save(m.database(), dir.filePath("leak.json")),
                                 FileError);
        for (int i = 0; i < 7; ++i) {
            m.editPatient("P1", PatientCategory::General,
                          detail("P1", "Ada " + QString::number(i)));
            persistence::RecoveryService::saveWithBackup(m.database(), file);
            QTest::qWait(2);
        }
        QCOMPARE(persistence::RecoveryService::copies(file).size(), 5);
        m.editPatient("P1", PatientCategory::General, detail("P1", "Recovered Ada"));
        persistence::RecoveryService::autosave(m.database(), file);
        auto recovery = FileHandler::load(persistence::RecoveryService::recoveryPath(file));
        QCOMPARE(recovery.patient("P1")->details().name, QString("Recovered Ada"));
        bytes[bytes.size() / 2] = static_cast<char>(bytes[bytes.size() / 2] ^ 1);
        QVERIFY(contents.open(QIODevice::WriteOnly));
        contents.write(bytes);
        contents.close();
```

```
        QVERIFY_EXCEPTION_THROWN(FileHandler::load(file), FileError);
        persistence::RecoveryService::discardRecovery(file);
        QVERIFY(!QFileInfo::exists(persistence::RecoveryService::recoveryPath(file)));
#endif
    }
    void dateRangeReportTotalsAndEscaping() {
        auto db = fixture();
        db.appointments[0].start = QDateTime(QDate(2026, 1, 1), QTime(9, 0));
        db.appointments[1].start = QDateTime(QDate(2026, 1, 2), QTime(9, 0));
        auto report = generateReport(db, ReportType::Daily, QDate(2026, 1, 1), QDate(2026, 1, 2));
        QCOMPARE(report.rows[0][1], QString("2"));
        QVERIFY(report.summary.contains("2 visits"));
        QVERIFY_EXCEPTION_THROWN(
            generateReport(db, ReportType::Daily, QDate(2026, 1, 3), QDate(2026, 1, 1)),
            ValidationError);
        Report html{"<script>", {"A&B"}, {{"<b>literal</b>"}}, "<summary>"};
        const auto output = reportHtml(html);
        QVERIFY(!output.contains("<script>"));
        QVERIFY(output.contains("&lt;b&gt;literal"));
    }
    void guiRetainsSearchSortAndSelectionAfterEdit() {
        gui::MainWindow window;
        window.manager().replace(fixture());
        window.refresh();
        window.show();
        auto *nav = window.findChild<QListWidget *>("navigation");
        nav->setCurrentRow(4);
        window.findChild<QComboBox *>("searchField")->setCurrentIndex(1);
        window.findChild<QLineEdit *>("searchText")->setText("Ada");
        QTest::mouseClick(window.findChild<QPushButton *>("searchButton"), Qt::LeftButton);
        QCOMPARE(window.findChild<QTableView *>("searchResults")->model()->rowCount(), 1);
        nav->setCurrentRow(1);
        auto *table = window.findChild<QTableView *>("patientsTable");
        table->selectRow(0);
        QTimer::singleShot(30, [&] {
            auto *dialog = window.findChild<QDialog *>("patientDialog");
            QVERIFY(dialog);
            dialog->findChild<QLineEdit *>("patientName")->setText("Ada Updated");
            dialog->findChild<QDialogButtonBox *>("patientButtons")
                ->button(QDialogButtonBox::Save)
                ->click();
        });
        window.findChild<QPushButton *>("editPatient")->click();
        QCOMPARE(table->currentIndex().row(), 0);
        auto *results = window.findChild<QTableView *>("searchResults")->model();
        QCOMPARE(results->rowCount(), 1);
        QCOMPARE(results->index(0, 1).data().toString(), QString("Ada Updated"));
        nav->setCurrentRow(4);
        window.findChild<QComboBox *>("sortField")->setCurrentIndex(2);
        window.findChild<QPushButton *>("sortButton")->click();
        window.findChild<QPushButton *>("undoButton")->click();
        QCOMPARE(results->rowCount(), 3);
        QCOMPARE(results->index(0, 2).data().toInt(), 23);
        window.manager().markSaved();
        window.close();
    }
    void guiUnsavedPromptAndRescheduling() {
        gui::MainWindow window;
        window.manager().replace(fixture());
        window.refresh();
        window.show();
        window.findChild<QListWidget *>("navigation")->setCurrentRow(2);
        auto *table = window.findChild<QTableWidget *>("appointmentsTable");
        table->selectRow(0);
        QTimer::singleShot(30, [&] {
            auto *dialog = window.findChild<QDialog *>("rescheduleDialog");
            QVERIFY(dialog);
            dialog->findChild<QDateTimeEdit *>("rescheduleTime")
                ->setDateTime(QDateTime::currentDateTime().addDays(2));
            dialog->findChild<QDialogButtonBox *>()->button(QDialogButtonBox::Save)->click();
        });
        window.findChild<QPushButton *>("rescheduleAppointment")->click();
        QVERIFY(window.manager().database().appointment("A1")->start >
                QDateTime::currentDateTime());
        QTimer::singleShot(30, [&] {
```

```
            auto *box = window.findChild<QMessageBox *>();
            QVERIFY(box);
            box->button(QMessageBox::Cancel)->click();
        });
        window.findChild<QAction *>("newWorkspaceAction")->trigger();
        QCOMPARE(window.manager().database().patients.size(), size_t(3));
        QVERIFY(window.manager().dirty());
        QTimer::singleShot(30, [&] {
            auto *box = window.findChild<QMessageBox *>();
            QVERIFY(box);
            box->button(QMessageBox::Discard)->click();
        });
        window.findChild<QAction *>("newWorkspaceAction")->trigger();
        QVERIFY(window.manager().database().patients.empty());
        window.close();
    }

    void walkInNoShowAndCorrections() {
        ClinicManager m;
        m.replace(fixture());
        m.enqueue("P3");
        const auto visit = m.database().waiting.front().appointmentId;
        QVERIFY(!visit.isEmpty());
        QVERIFY(m.database().appointment(visit)->walkIn);
        QVERIFY_EXCEPTION_THROWN(m.complete(visit, "Too soon"), ValidationError);
        m.callNext();
        m.complete(visit, "Initial notes");
        QVERIFY(m.database().appointment(visit)->completedAt.isValid());
        m.correctNotes(visit, "Corrected notes", "Clarified wording");
        QCOMPARE(m.database().appointment(visit)->corrections.size(), 1);
        QVERIFY(m.database().appointment(visit)->corrections[0].contains("Initial notes"));
        m.noShow("A1", "Patient did not attend");
        QCOMPARE(m.database().appointment("A1")->status, AppointmentStatus::NoShow);
        QVERIFY_EXCEPTION_THROWN(m.noShow("A3", "Already cancelled"), ValidationError);
        const auto future = m.schedule("P1", "Future", QDateTime::currentDateTime().addDays(1), 30)
  > ;
        QVERIFY_EXCEPTION_THROWN(m.noShow(future, "Early"), ValidationError);
        QVERIFY_EXCEPTION_THROWN(m.cancel(future), ValidationError);
        const auto report = generateReport(m.database(), ReportType::Daily);
        QCOMPARE(report.rows.last(), QStringList({"No-show", "1"}));
        const auto restored = FileHandler::deserialize(FileHandler::serialize(m.database()));
        QCOMPARE(restored.appointment(visit)->corrections,
                 m.database().appointment(visit)->corrections);
        QVERIFY(restored.appointment(visit)->walkIn);
    }
    void clinicianRoomConflictsAndPersistence() {
        ClinicManager m;
        m.replace(fixture());
        m.addResource("Dr Ada", false);
        m.addResource("Dr Tunde", false);
        m.addResource("Room 1", true);
        m.addResource("Room 2", true);
        QVERIFY_EXCEPTION_THROWN(m.addResource("room 1", true), ValidationError);
        auto start = QDateTime::currentDateTime().addDays(1);
        const auto a = m.schedule("P1", "Review", start, 30, "Dr Ada", "Room 1");
        auto before = stateOnly(m.database());
        QVERIFY_EXCEPTION_THROWN(m.schedule("P2", "Review", start, 30, "Dr Ada", "Room 2"),
                                 ValidationError);
        QVERIFY_EXCEPTION_THROWN(m.schedule("P2", "Review", start, 30, "Dr Tunde", "Room 1"),
                                 ValidationError);
        QCOMPARE(stateOnly(m.database()), before);
        const auto b = m.schedule("P2", "Review", start, 30, "Dr Tunde", "Room 2");
        QVERIFY_EXCEPTION_THROWN(m.assignResources(b, "Dr Ada", "Room 2"), ValidationError);
        m.reschedule(b, start.addSecs(1800), 30);
        m.assignResources(b, "Dr Ada", "Room 1");
        QCOMPARE(m.database().appointment(a)->room, QString("Room 1"));
        const auto restored = FileHandler::deserialize(FileHandler::serialize(m.database()));
        QCOMPARE(restored.clinicians, m.database().clinicians);
        QCOMPARE(restored.appointment(b)->clinician, QString("Dr Ada"));
    }
    void workspaceLockAndExternalChange() {
        QTemporaryDir dir;
        const auto file = dir.filePath("workspace.json");
        auto db = fixture();
        FileHandler::save(db, file);
```

```
        {
            persistence::WorkspaceGuard guard(file);
            guard.verifyUnchanged();
            QVERIFY_EXCEPTION_THROWN(persistence::WorkspaceGuard second(file), FileError);
            db.nextPatientId += 10;
            FileHandler::save(db, file);
            QVERIFY_EXCEPTION_THROWN(guard.verifyUnchanged(), FileError);
            guard.checkpoint();
            guard.verifyUnchanged();
        }
        persistence::WorkspaceGuard reopened(file);
        reopened.verifyUnchanged();
        gui::MainWindow window;
        QVERIFY_EXCEPTION_THROWN(window.loadPath(file), FileError);
    }
    void portableBackupAuthentication() {
#ifdef Q_OS_WIN
        // Independent fixture generated with Python cryptography AESGCM and hashlib PBKDF2.
        const auto external =
            QByteArray::fromBase64("U0NBUE1TLUJBQ0tVUC0xCgABAgMEBQYHCAkKCwwNDg8AAQIDBAUGBwgJCgvhpxT
  > "
                                   "wL9As2ZdaFaEjkqKTcWsk8lmPFd/kqzk3u8w+C7lZC3vHQbagOxWEsr4ABI4n")
  > ;
        QCOMPARE(security::decryptBackup(external, "interoperability test password"),
                 QByteArray("portable backup interoperability\n"));
        const auto plain = QJsonDocument(FileHandler::serialize(fixture())).toJson();
        const auto password = QString("a separate long backup passphrase");
        const auto encrypted = security::encryptBackup(plain, password);
        QVERIFY(encrypted.startsWith("SCAPMS-BACKUP-1\n"));
        QVERIFY(!encrypted.contains("Health review"));
        QCOMPARE(security::decryptBackup(encrypted, password), plain);
        QVERIFY_EXCEPTION_THROWN(security::decryptBackup(encrypted, "wrong but sufficiently long"),
                                 FileError);
        auto tampered = encrypted;
        tampered[tampered.size() - 1] ^= 1;
        QVERIFY_EXCEPTION_THROWN(security::decryptBackup(tampered, password), FileError);
        QVERIFY_EXCEPTION_THROWN(security::encryptBackup(plain, "short"), ValidationError);
        QVERIFY_EXCEPTION_THROWN(security::decryptBackup(encrypted.left(20), password), FileError);
#endif
    }
    void clinicalPrivacyAndExportPermissions() {
        ClinicManager m;
        m.replace(fixture());
        m.configureAdministrator("admin", "long administrator password");
        m.addUser("desk", "long receptionist password", security::Role::Receptionist);
        m.addUser("reader", "long viewer password", security::Role::Viewer);
        m.lock();
        m.login("desk", "long receptionist password");
        const auto old = m.database().patient("P1")->details().reason;
        auto change = detail("P1", "Updated demographics");
        change.reason = "Unauthorized clinical change";
        m.editPatient("P1", PatientCategory::Emergency, change);
        QCOMPARE(m.database().patient("P1")->details().reason, old);
        QCOMPARE(m.database().patient("P1")->category(), PatientCategory::General);
        QVERIFY_EXCEPTION_THROWN(m.requireReportExport(), ValidationError);
        gui::PatientTableModel model;
        model.setPatients(viewOf(m.database().patients));
        QVERIFY(!model.data(model.index(0, 0), Qt::ToolTipRole).toString().contains(old));
        model.setClinicalAccess(true);
        QVERIFY(model.data(model.index(0, 0), Qt::ToolTipRole).toString().contains(old));
        gui::PatientDialog dialog("P1", m.database().patient("P1"));
        dialog.configureAccess(false, m.database());
        QVERIFY(dialog.findChild<QLineEdit *>("patientReason")->isHidden());
        QVERIFY(dialog.findChild<QComboBox *>("patientCategory")->isHidden());
        gui::MainWindow window;
        window.manager().replace(m.database(), "reader", "long viewer password");
        window.refresh();
        QVERIFY(!window.findChild<QPushButton *>("exportReport")->isEnabled());
        QVERIFY(!window.findChild<QPushButton *>("exportReportPdf")->isEnabled());
        QVERIFY(!window.findChild<QPushButton *>("printReport")->isEnabled());
        QVERIFY(!window.findChild<QPushButton *>("addClinician")->isEnabled());
    }
    void contactDuplicatesAndPreviousVisitPicker() {
        auto invalid = detail("P9");
        invalid.contact = "not a contact";
```

```
        QVERIFY_EXCEPTION_THROWN(makePatient(PatientCategory::General, invalid), ValidationError);
        invalid.contact = "+234 801 234 5678";
        QVERIFY(makePatient(PatientCategory::General, invalid));
        ClinicManager m;
        m.replace(fixture());
        auto duplicate = detail("P8", "Ada", 23);
        QVERIFY(!m.possibleDuplicates(duplicate).isEmpty());
        QVERIFY(m.possibleDuplicates(detail("P1", "Ada", 23), "P1").isEmpty());
        gui::PatientDialog dialog("P2", m.database().patient("P2"));
        dialog.configureAccess(true, m.database());
        auto *choices = dialog.findChild<QComboBox *>("previousVisitChoices");
        QVERIFY(choices);
        QVERIFY(choices->findData("A2") >= 0);
        QVERIFY(dialog.findChild<QLineEdit *>("previousVisit")->isReadOnly());
    }
    void repeatedBenchmarksUseIdenticalCases() {
        ClinicDatabase db;
        for (int i = 0; i < 10; ++i)
            db.patients.push_back(
                makePatient(PatientCategory::General, detail("P" + QString::number(i))));
        const auto results = benchmarkSorts(viewOf(db.patients), SortField::Id, 3);
        QCOMPARE(results.size(), size_t(6));
        QCOMPARE(results[0].metrics.comparisons, size_t(9));
        QCOMPARE(results[2].metrics.comparisons, size_t(45));
        for (const auto &r : results) {
            QCOMPARE(r.records, 10);
            QVERIFY(r.medianMs >= 0);
        }
        QVERIFY_EXCEPTION_THROWN(benchmarkSorts({}, SortField::Id), ValidationError);
    }
    void welcomeAndCalendarUi() {
        gui::MainWindow window;
        QTimer::singleShot(30, [&] {
            auto *button = window.findChild<QPushButton *>("welcomeCreate");
            QVERIFY(button);
            button->click();
        });
        QVERIFY(window.welcome());
        QVERIFY(window.manager().database().patients.empty());
        window.manager().replace(fixture());
        window.manager().addResource("Dr Calendar", false);
        window.manager().addResource("Room Calendar", true);
        window.refresh();
        auto *nav = window.findChild<QListWidget *>("navigation");
        QCOMPARE(nav->count(), 8);
        nav->setCurrentRow(7);
        auto *calendar = window.findChild<QCalendarWidget *>("resourceCalendar");
        QVERIFY(calendar);
        calendar->setSelectedDate(QDate::currentDate());
        QVERIFY(window.findChild<QComboBox *>("calendarClinician")->findData("Dr Calendar") >= 0);
        window.manager().markSaved();
    }

    void guiDynamicFieldsAndValidation() {
        gui::PatientDialog dialog("P1");
        dialog.show();
        auto *category = dialog.findChild<QComboBox *>("patientCategory");
        category->setCurrentIndex(1);
        QVERIFY(!dialog.findChild<QSpinBox *>("patientUrgency")->isHidden());
        QVERIFY(dialog.findChild<QLineEdit *>("previousVisit")->isHidden());
        category->setCurrentIndex(2);
        QVERIFY(!dialog.findChild<QLineEdit *>("previousVisit")->isHidden());
        QVERIFY(dialog.findChild<QSpinBox *>("patientUrgency")->isHidden());
        QTest::mouseClick(
            dialog.findChild<QDialogButtonBox *>("patientButtons")->button(QDialogButtonBox::Save),
            Qt::LeftButton);
        QCOMPARE(dialog.result(), 0);
        QVERIFY(!dialog.findChild<QLabel *>("formError")->text().isEmpty());
    }
};
QTEST_MAIN(ClinicTests)
#include "ClinicTests.moc"
```

5.59 SOURCE OR RESOURCE FILE

scripts/build.ps1

```
param([switch]$Test, [switch]$Package)
$ErrorActionPreference = 'Stop'
$projectRoot = Split-Path -Parent $PSScriptRoot
$qtRoot = Join-Path $projectRoot '.tools/Qt/6.8.3/mingw_64'
$compilerRoot = Join-Path $projectRoot '.tools/Qt/Tools/mingw1310_64/bin'
$cmake = Join-Path $projectRoot '.tools/python/cmake/data/bin/cmake.exe'
$ninja = Join-Path $projectRoot '.tools/python/bin/ninja.exe'
if (!(Test-Path $cmake) -or !(Test-Path $qtRoot) -or !(Test-Path $compilerRoot)) {
    throw 'Local toolchain missing. Run scripts/setup.ps1, or use Qt Creator with CMakeLists.txt.'
}
$env:QT_QPA_FONTDIR = "$env:WINDIR/Fonts"
$env:PATH = "$qtRoot/bin;$compilerRoot;$(Split-Path $ninja);$env:PATH"
& $cmake -S $projectRoot -B "$projectRoot/build" -G Ninja "-DCMAKE_MAKE_PROGRAM=$ninja" "-DCMAKE_PR
 > EFIX_PATH=$qtRoot" "-DCMAKE_CXX_COMPILER=$compilerRoot/g++.exe" -DCMAKE_BUILD_TYPE=Release
if ($LASTEXITCODE -ne 0) { throw 'CMake configuration failed.' }
& $cmake --build "$projectRoot/build" --parallel 4
if ($LASTEXITCODE -ne 0) { throw 'Build failed.' }
if ($Test) {
    & (Join-Path (Split-Path $cmake) 'ctest.exe') --test-dir "$projectRoot/build" --output-on-failu
 > re
    if ($LASTEXITCODE -ne 0) { throw 'Tests failed.' }
}
if ($Package) {
    $packageDir = Join-Path $projectRoot 'dist/SCAPMS-v3'
    New-Item -ItemType Directory -Force -Path $packageDir | Out-Null
    Copy-Item -LiteralPath "$projectRoot/build/scapms.exe" -Destination $packageDir -Force
    & "$qtRoot/bin/windeployqt.exe" --release --no-translations --no-opengl-sw --no-system-d3d-comp
 > iler "$packageDir/scapms.exe"
    if ($LASTEXITCODE -ne 0) { throw 'Qt deployment failed.' }
    Copy-Item -LiteralPath "$qtRoot/plugins/platforms/qoffscreen.dll" -Destination "$packageDir/pla
 > tforms" -Force
    if (Test-Path "$projectRoot/licenses") { Copy-Item -LiteralPath "$projectRoot/licenses" -Destin
 > ation $packageDir -Recurse -Force }
    Copy-Item -LiteralPath "$projectRoot/data/sample-clinic.json" -Destination $packageDir -Force
    Copy-Item -LiteralPath "$projectRoot/data/benchmark-2000.json" -Destination $packageDir -Force
    Copy-Item -LiteralPath "$projectRoot/README.md" -Destination $packageDir -Force
    Copy-Item -LiteralPath "$projectRoot/THIRD_PARTY_NOTICES.md" -Destination $packageDir -Force
    Copy-Item -LiteralPath "$projectRoot/Launch-SCAPMS.cmd" -Destination $packageDir -Force
}
```

5.60 SOURCE OR RESOURCE FILE

scripts/setup.ps1

```
param([string]$Python = 'python')
$ErrorActionPreference = 'Stop'
$projectRoot = Split-Path -Parent $PSScriptRoot
Push-Location $projectRoot
try {
    & $Python -m pip install --target .tools/python aqtinstall==3.3.0 cmake ninja
    if ($LASTEXITCODE -ne 0) { throw 'Python dependency installation failed.' }
    $env:PYTHONPATH = Join-Path $projectRoot '.tools/python'
    & $Python -m aqt install-qt windows desktop 6.8.3 win64_mingw -O .tools/Qt --archives qtbase
    if ($LASTEXITCODE -ne 0) { throw 'Qt installation failed.' }
    & $Python -m aqt install-tool windows desktop tools_mingw1310 qt.tools.win64_mingw1310 -O .tool
 > s/Qt
    if ($LASTEXITCODE -ne 0) { throw 'Compiler installation failed.' }
} finally { Pop-Location }
```

5.61 SOURCE OR RESOURCE FILE

scripts/make_sample.py

```
"""Generate fictional clinic data. Run again to refresh dates for a demonstration."""
```

```
import json
from datetime import datetime, timedelta, timezone
from pathlib import Path
root = Path(__file__).resolve().parents[1]
now = datetime.now(timezone.utc)
def iso(value):
    return value.isoformat(timespec="milliseconds").replace("+00:00", "Z")
people = [
    ("Ada Okafor", 23, "General", "Routine health review", 1, ""),
    ("Tunde Bello", 31, "Emergency", "Urgent assessment", 4, ""),
    ("Zainab Musa", 27, "Follow-up", "Review of previous consultation", 1, "VIS-2026-014"),
    ("Chidi Eze", 20, "General", "Wellness screening", 1, ""),
    ("Fatima Ibrahim", 34, "Follow-up", "Scheduled review", 1, "VIS-2026-018"),
    ("Emeka Nwosu", 26, "General", "General consultation", 1, ""),
    ("Amina Yusuf", 22, "Emergency", "Same-day assessment", 3, ""),
    ("Samuel Adeyemi", 29, "General", "Health screening", 1, ""),
]
patients = [dict(id=f"P{i:04}", name=p[0], age=p[1], contact=f"patient{i}@example.test",
    category=p[2], reason=p[3], urgency=p[4], previousVisit=p[5]) for i,p in enumerate(people,1)]
appointments = []
def appointment(patient, offset, status, service):
    appointments.append(dict(id=f"A{len(appointments)+1:04}", patientId=f"P{patient:04}", service=s
> ervice,
        start=iso(now+timedelta(minutes=offset)), durationMinutes=30, status=status,
        consultationNotes="Sample consultation completed; routine follow-up recorded." if status=="
> Completed" else ""))
appointment(1,-150,"Completed","General consultation")
appointment(3,-100,"Completed","Follow-up review")
appointment(1,-1440,"Completed","Health screening")
appointment(2,-35,"Scheduled","Emergency review")
appointment(5,20,"Scheduled","Follow-up review")
appointment(4,60,"Scheduled","Health screening")
appointment(6,120,"Cancelled","General consultation")
appointment(8,1440,"Scheduled","Health screening")
data = dict(schemaVersion=3,nextPatientId=9,nextAppointmentId=9,patients=patients,appointments=appo
> intments,
    waitingQueue=[dict(patientId=f"P{i:04}",arrivedAt=iso(now-timedelta(minutes=m))) for i,m in [(2
> ,20),(5,10),(4,5)]])

data.update(clinicians=["Dr Demo A","Dr Demo B"], rooms=["Room 1","Room 2"],
            archivedPatients=[], users=[], audit=[])
for index,visit in enumerate(appointments):
    visit.update(changeReason="Sample cancellation" if visit["status"]=="Cancelled" else "",
                 clinician=data["clinicians"][index%2], room=data["rooms"][index%2],
                 walkIn=False, corrections=[], consultationStarted="", completedAt="")
for entry in data["waitingQueue"]:
    visit=next(a for a in appointments if a["patientId"]==entry["patientId"] and a["status"]=="Sche
> duled")
    visit["status"]="Checked in"
    entry["appointmentId"]=visit["id"]
(root/"data").mkdir(exist_ok=True)
(root/"data"/"sample-clinic.json").write_text(json.dumps(data,indent=2),encoding="utf-8")
print("Created fictional sample-clinic.json")
```

5.62 SOURCE OR RESOURCE FILE

scripts/make_benchmark.py

```
"""Create 2,000 deterministic, fictional records for comparing manual algorithms."""
from pathlib import Path
import json
import random

randomizer = random.Random(2026)
records = []
for index in range(1, 2001):
    category = randomizer.choice(["General", "Emergency", "Follow-up"])
    records.append({
        "id": f"B{index:05d}",
        "name": f"Demo Patient {randomizer.randrange(100000):05d}",
        "age": randomizer.randrange(121),
        "contact": f"benchmark{index}@example.test",
        "category": category,
        "reason": "Fictional algorithm demonstration",
```

```
        "urgency": randomizer.randrange(1, 6),
        "previousVisit": f"DEMO-{index}" if category == "Follow-up" else ""
    })
randomizer.shuffle(records)
data = dict(schemaVersion=2, nextPatientId=1, nextAppointmentId=1, patients=records,
            appointments=[], waitingQueue=[], archivedPatients=[], users=[], audit=[])
target = Path(__file__).resolve().parents[1] / "data" / "benchmark-2000.json"
target.write_text(json.dumps(data, indent=2), encoding="utf-8")
print(target)
```

5.63 SOURCE OR RESOURCE FILE

Launch-SCAPMS.cmd

```
@echo off
cd /d "%~dp0"
if exist "scapms.exe" (
    start "" "scapms.exe"
) else if exist "dist\SCAPMS-v3\scapms.exe" (
    start "" "dist\SCAPMS-v3\scapms.exe"
) else (
    echo Build the application first: powershell -File scripts\build.ps1 -Test -Package
    pause
)
```

5.64 SOURCE OR RESOURCE FILE

data/sample-clinic.json

```
{
  "schemaVersion": 3,
  "nextPatientId": 9,
  "nextAppointmentId": 9,
  "patients": [
    {
      "id": "P0001",
      "name": "Ada Okafor",
      "age": 23,
      "contact": "patient1@example.test",
      "category": "General",
      "reason": "Routine health review",
      "urgency": 1,
      "previousVisit": ""
    },
    {
      "id": "P0002",
      "name": "Tunde Bello",
      "age": 31,
      "contact": "patient2@example.test",
      "category": "Emergency",
      "reason": "Urgent assessment",
      "urgency": 4,
      "previousVisit": ""
    },
    {
      "id": "P0003",
      "name": "Zainab Musa",
      "age": 27,
      "contact": "patient3@example.test",
      "category": "Follow-up",
      "reason": "Review of previous consultation",
      "urgency": 1,
      "previousVisit": "VIS-2026-014"
    },
    {
      "id": "P0004",
      "name": "Chidi Eze",
      "age": 20,
      "contact": "patient4@example.test",
      "category": "General",
      "reason": "Wellness screening",
      "urgency": 1,
      "previousVisit": ""
```

```
    },
    {
      "id": "P0005",
      "name": "Fatima Ibrahim",
      "age": 34,
      "contact": "patient5@example.test",
      "category": "Follow-up",
      "reason": "Scheduled review",
      "urgency": 1,
      "previousVisit": "VIS-2026-018"
    },
    {
      "id": "P0006",
      "name": "Emeka Nwosu",
      "age": 26,
      "contact": "patient6@example.test",
      "category": "General",
      "reason": "General consultation",
      "urgency": 1,
      "previousVisit": ""
    },
    {
      "id": "P0007",
      "name": "Amina Yusuf",
      "age": 22,
      "contact": "patient7@example.test",
      "category": "Emergency",
      "reason": "Same-day assessment",
      "urgency": 3,
      "previousVisit": ""
    },
    {
      "id": "P0008",
      "name": "Samuel Adeyemi",
      "age": 29,
      "contact": "patient8@example.test",
      "category": "General",
      "reason": "Health screening",
      "urgency": 1,
      "previousVisit": ""
    }
  ],
  "appointments": [
    {
      "id": "A0001",
      "patientId": "P0001",
      "service": "General consultation",
      "start": "2026-11-04T07:18:56.610Z",
      "durationMinutes": 30,
      "status": "Completed",
      "consultationNotes": "Sample consultation completed; routine follow-up recorded.",
      "changeReason": "",
      "clinician": "Dr Demo A",
      "room": "Room 1",
      "walkIn": false,
      "corrections": [],
      "consultationStarted": "",
      "completedAt": ""
    },
    {
      "id": "A0002",
      "patientId": "P0003",
      "service": "Follow-up review",
      "start": "2026-11-04T08:08:56.610Z",
      "durationMinutes": 30,
      "status": "Completed",
      "consultationNotes": "Sample consultation completed; routine follow-up recorded.",
      "changeReason": "",
      "clinician": "Dr Demo B",
      "room": "Room 2",
      "walkIn": false,
      "corrections": [],
      "consultationStarted": "",
      "completedAt": ""
    },
```

```
    {
      "id": "A0003",
      "patientId": "P0001",
      "service": "Health screening",
      "start": "2026-11-03T09:48:56.610Z",
      "durationMinutes": 30,
      "status": "Completed",
      "consultationNotes": "Sample consultation completed; routine follow-up recorded.",
      "changeReason": "",
      "clinician": "Dr Demo A",
      "room": "Room 1",
      "walkIn": false,
      "corrections": [],
      "consultationStarted": "",
      "completedAt": ""
    },
    {
      "id": "A0004",
      "patientId": "P0002",
      "service": "Emergency review",
      "start": "2026-11-04T09:13:56.610Z",
      "durationMinutes": 30,
      "status": "Checked in",
      "consultationNotes": "",
      "changeReason": "",
      "clinician": "Dr Demo B",
      "room": "Room 2",
      "walkIn": false,
      "corrections": [],
      "consultationStarted": "",
      "completedAt": ""
    },
    {
      "id": "A0005",
      "patientId": "P0005",
      "service": "Follow-up review",
      "start": "2026-11-04T10:08:56.610Z",
      "durationMinutes": 30,
      "status": "Checked in",
      "consultationNotes": "",
      "changeReason": "",
      "clinician": "Dr Demo A",
      "room": "Room 1",
      "walkIn": false,
      "corrections": [],
      "consultationStarted": "",
      "completedAt": ""
    },
    {
      "id": "A0006",
      "patientId": "P0004",
      "service": "Health screening",
      "start": "2026-11-04T10:48:56.610Z",
      "durationMinutes": 30,
      "status": "Checked in",
      "consultationNotes": "",
      "changeReason": "",
      "clinician": "Dr Demo B",
      "room": "Room 2",
      "walkIn": false,
      "corrections": [],
      "consultationStarted": "",
      "completedAt": ""
    },
    {
      "id": "A0007",
      "patientId": "P0006",
      "service": "General consultation",
      "start": "2026-11-04T11:48:56.610Z",
      "durationMinutes": 30,
      "status": "Cancelled",
      "consultationNotes": "",
      "changeReason": "Sample cancellation",
      "clinician": "Dr Demo A",
      "room": "Room 1",
```

```
      "walkIn": false,
      "corrections": [],
      "consultationStarted": "",
      "completedAt": ""
    },
    {
      "id": "A0008",
      "patientId": "P0008",
      "service": "Health screening",
      "start": "2026-11-05T09:48:56.610Z",
      "durationMinutes": 30,
      "status": "Scheduled",
      "consultationNotes": "",
      "changeReason": "",
      "clinician": "Dr Demo B",
      "room": "Room 2",
      "walkIn": false,
      "corrections": [],
      "consultationStarted": "",
      "completedAt": ""
    }
  ],
  "waitingQueue": [
    {
      "patientId": "P0002",
      "arrivedAt": "2026-11-04T09:28:56.610Z",
      "appointmentId": "A0004"
    },
    {
      "patientId": "P0005",
      "arrivedAt": "2026-11-04T09:38:56.610Z",
      "appointmentId": "A0005"
    },
    {
      "patientId": "P0004",
      "arrivedAt": "2026-11-04T09:43:56.610Z",
      "appointmentId": "A0006"
    }
  ],
  "clinicians": [
    "Dr Demo A",
    "Dr Demo B"
  ],
  "rooms": [
    "Room 1",
    "Room 2"
  ],
  "archivedPatients": [],
  "users": [],
  "audit": []
}
```

6.0 REFERENCES

The Qt Company. (n.d.). Model/view programming. https://doc.qt.io/qt-6/model-view-programming.html

The Qt Company. (n.d.). QPasswordDigestor namespace. https://doc.qt.io/qt-6/qpassworddigestor.html

The Qt Company. (n.d.). QSaveFile class. https://doc.qt.io/qt-6/qsavefile.html

Microsoft. (n.d.). CryptProtectData function. https://learn.microsoft.com/en-us/windows/win32/api/dpapi/nf-dpapi-cryptprotectdata

Microsoft. (n.d.). BCRYPT_AUTHENTICATED_CIPHER_MODE_INFO. https://learn.microsoft.com/en-us/windows/win32/api/bcrypt/ns-bcrypt-bcrypt_authenticated_cipher_mode_info

The Qt Company. (n.d.). QLockFile class. https://doc.qt.io/qt-6/qlockfile.html

Miva Open University. (n.d.). COS 202 Programming II lab assessment project: Smart Clinic Appointment and Patient Management System. Supplied assessment brief.